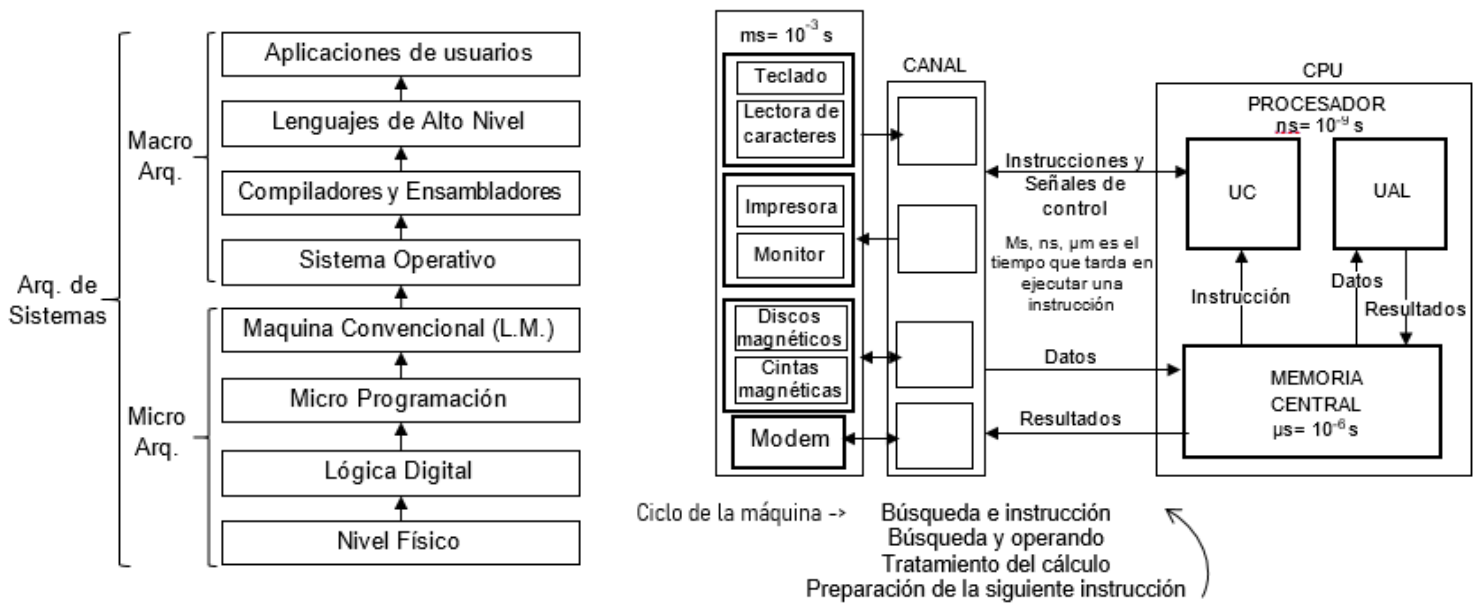


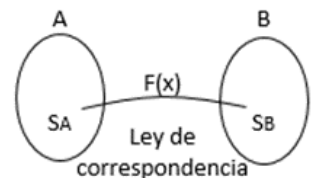
Evolución de las Computadoras

0 Hasta 1945	Sistemas mecánicos y electromecánicos Desde épocas remotas el hombre trató de liberarse de hacer cálculos en forma manual. Un ejemplo es el ábaco y la pascalina La tarjeta perforada desempeñó un papel importante en el procesamiento de datos y reemplazó en gran medida al procesamiento manual (1930-1970 aprox)
1 1945-1955	Tubos al vacío, Tableros Las computadoras estaban constituidas por válvulas de vacío, que disipan gran cantidad de calor y ocupaban una amplia superficie. Los medios de entrada/salida eran tarjetas o cintas perforadas. Las tareas se ejecutaban de manera puramente secuencial. ENIGMA (Thomas Jefferson), COLOSSUS (Turing)
2 1955-1965	Transistores y Sistemas por Lotes Estaban constituidas por transistores, más confiables, de menor tamaño, menor disipación de calor y más rápidos que las válvulas de vacío para cambiar de estado. Posibilitaron la simultaneidad entre un cálculo y una operación de E/S, aunque el encadenamiento de los trabajos seguía siendo secuencial. Las operaciones de E/S se realizaban usando como soporte de almacenamiento unidades de cinta magnética, más rápidas que las lectoras de tarjetas y las impresoras. Acá surge el concepto de procesamiento por lotes, ya que era necesario esperar a que un lote de trabajos cargados en la cinta magnética se procese por completo para obtener los resultados de cada trabajo.
3 1965-1980	Circuitos integrados y Multiprogramación Se introducen los discos duros y el tiempo compartido entre usuarios. Es posible el procesamiento a distancia y se desarrolla el concepto de máquina virtual. En esta etapa se pudo explotar la multiprogramación, método que consiste en que varios programas residan en forma simultánea en la memoria en estado de ejecución. La "carga por lotes" se sustituyó por la "carga continua". Los trabajos se ponen en cola de espera en un disco magnético y el sistema operativo es el que se encarga de ejecutarlos según un nivel de prioridad. Los resultados, que son transferidos al disco, luego son extraídos por la impresora.
4 1980	Computadores personales Se introducen sistemas operativos en red (varios computadores conectados) y sistemas operativos distribuidos (múltiples procesadores). La tecnología aplicada en la fabricación de circuitos pertenece a la clasificación LSI, que permitió incluir una CPU completa en un microprocesador. Se desarrollan nuevas utilidades a partir de la sencilla comunicación usuario-máquina Hasta esta etapa, los avances tecnológicos estuvieron concentrados en lograr mejorar el hardware de la computadora y así obtener equipos más pequeños, menos costosos y más rápidos. A partir de entonces, también se puso atención en la necesidad de mejorar el software para que permitiera una mayor velocidad de procesamiento.
5 Actualidad	Computación cuántica Tiene su origen en uno de los principios fundamentales de la mecánica cuántica, conocida como principio de superposición de estados. La unidad de información es el qubit, o bit cuántico → puede tener múltiples estados simultáneamente en un instante determinado, así reduciendo el tiempo de ejecución de algunos algoritmos de miles de años a segundos ⇒ Para evitar que la energía térmica introduzca perturbaciones que provoquen un cambio de estado espontáneo en los qubits, es necesario mantener los ordenadores cuánticos a una temperatura cercana al cero absoluto. Aplicaciones a las que apunta: IA, Simulación Biomédica, Sistemas Financieros, Transmisión y encriptación de datos, etc.



Codificación de la información

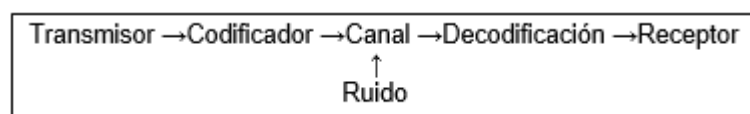
La **codificación** es un proceso, u sistema, que permite establecer una **ley de correspondencia** entre dos alfabetos A y B; entre un símbolo de A y un símbolo de B. Consiste en establecer una ley de correspondencia, llamada código, entre las informaciones por representar y las posibles configuraciones binarias, de tal manera que a cada información corresponda una y generalmente solo una, configuración binaria. **Es decir, este proceso convierte el alfabeto del usuario a una cadena o configuración binaria entendible para la computadora.**



La codificación de la información puede ser:

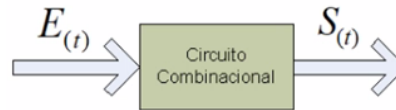
- **Númerica:**
 - Conversión de números en diferentes bases.
 - Conversión de números en número flotante.
- **No numérica:**
 - Alfanumérica.
 - Instrucciones.
 - Redundantes → detección y corrección de errores.

El cambio de un bit en el almacenamiento o la manipulación de datos origina resultados erróneos. Para detectar e incluso corregir estas alteraciones se usan códigos que agregan "**bits redundantes**". De todas formas, la cantidad de errores que puedan producirse son mensurables probabilísticamente. El resultado de "medir" la cantidad de "ruido" que puede afectar la información se representa con un coeficiente conocido como tasa de error. De acuerdo con el valor de la tasa de error se selecciona un código para descubrir e incluso corregir los errores. Estos códigos reciben el nombre de códigos de paridad.

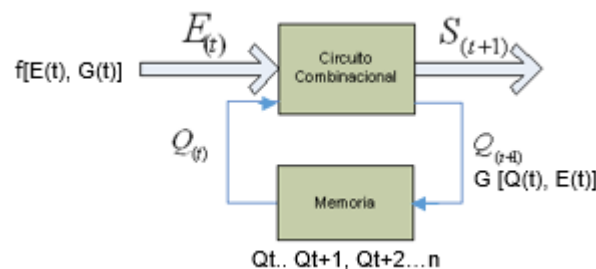


Circuitos

Decimos que un **circuito es combinacional** cuando las señales de salida no dependen más que de las señales de entrada, por lo que no necesita de módulos de memoria. Este tipo de circuitos no tiene en cuenta la variable tiempo, es decir, a menos que se redefina su estructura interna, **un circuito combinacional entrega siempre el mismo valor de salida para una combinación dada de entradas.**



Y será **secuencial** si las señales de salida **dependen de las de señales de entrada y del tiempo**, estos poseen **elementos de memoria** que recojan la información de la “historia pasada” del sistema. Se basa en la teoría de autómatas finitos: tiene una cantidad de estados determinados, que no es infinita.



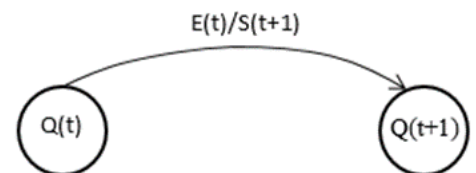
Funciones de transición

$S(t+1) = F [H(t), E(t)]$ *expresión genérica*

$S(t+1) = F [Q(t), E(t)]$

$Q(t+1) = G [Q(t), E(t)]$

/Qt= Estado AF en t; Et= Entradas AF en t; F= función de salida en t+1



Biestables o flip-flops

Un **biestable es una celda binaria capaz de almacenar un bit**. Tiene dos salidas, una para el valor del bit almacenado y otra que representa su complemento. Según en qué instante pueden actuar sus entradas y cambiar de estado (y en correspondencia cambiar el valor de sus salidas), los flip – flops pueden ser:

- **Asincrónico:** sus entradas pueden actuar en cualquier instante para cambiar el estado del circuito, y en conformidad cambiar el valor de sus salidas.
- **Sincrónico:** sus entradas pueden actuar sobre el estado del circuito sólo en determinados instantes, definidos en relación a pulsos que llegan por otra entrada de sincronismo conocida como reloj o “clock”.

Flip – Flop RS { Asincrónico
Sincrónico

Flip – Flop D { Sincrónico

Flip – Flop JK { Asincrónico
Sincrónico

Flip – Flop T { Asincrónico
Sincrónico

R-S

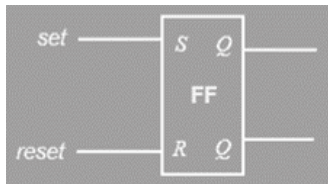
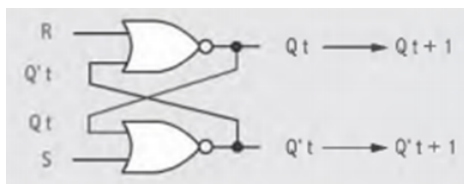


Tabla de funcionamiento

R	S	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	?
1	1	1	?

Tabla de Excitación

$Q(t)$	$Q(t+1)$	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

D

Es una modificación del RS síncrono. **Es básicamente un flip-flop RS de una sola entrada con un inversor en la entrada R**, lo que ofrece una manera de eliminar el estado indefinido en el flip-flop RS, ya que asegura que las entradas S y R nunca sean iguales a 1 al mismo tiempo.

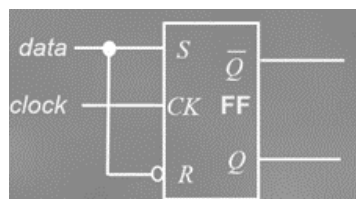
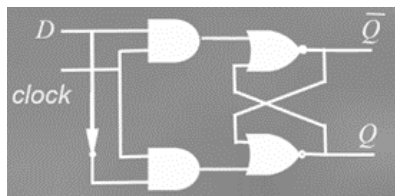


Tabla de Comportamiento

D	Q_t	Q_{t+1}
0	0	0
0	1	0
1	0	1
1	1	1

Tabla de Excitación

Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

J-K

Rediseño de RS. Evita el caso de la indeterminación donde $J=K=1$ y complementa el bit almacenado.

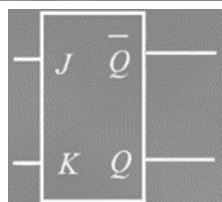
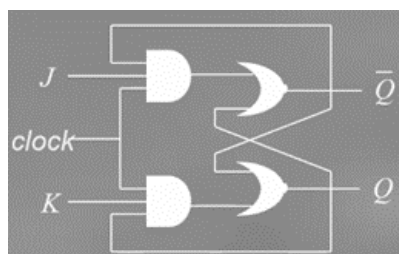


Tabla de Comportamiento

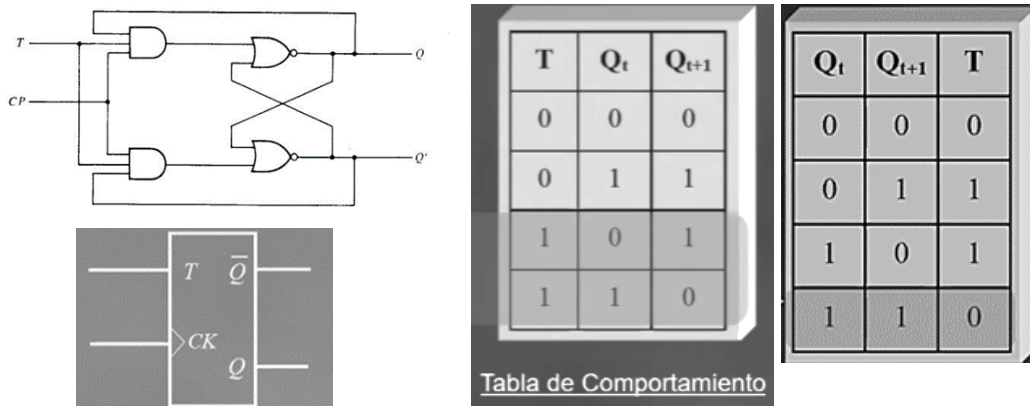
J	K	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Tabla de Excitación

Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

T

Versión de una sola entrada del biestable JK, se obtiene uniendo las entradas J y K de dicho flip-flop. El flip-flop complementador es útil para diseñar contadores binarios.



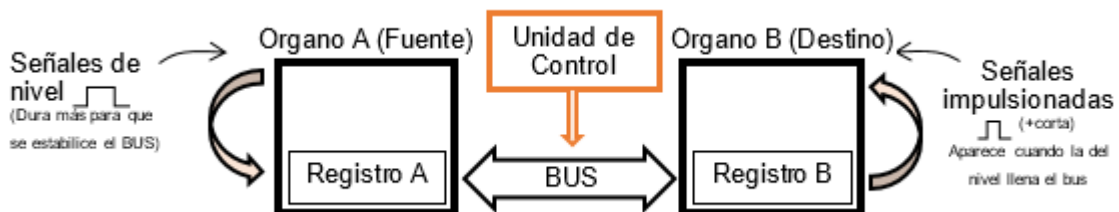
Registros y Buses

Un registro representa un **arreglo de flip-flops** organizados a través de una lógica combinacional y/o secuencial que lo capacita para realizar operaciones específicas sobre la información binaria contenida. **Suele utilizarse para retener información binaria temporalmente.** La principal característica de los registros es que poseen una **gran velocidad de acceso a los datos y permiten almacenar los datos rápidamente**, pero cuentan con **poca capacidad de almacenamiento**.

Como mínimo debe contener la lógica necesaria para la su lectura y grabación con el objeto de poder inicializarse. Las lógicas más sencillas permiten operaciones básicas sobre todo el conjunto de flip-flops tales como la puesta a cero, el complemento, y la transferencia.

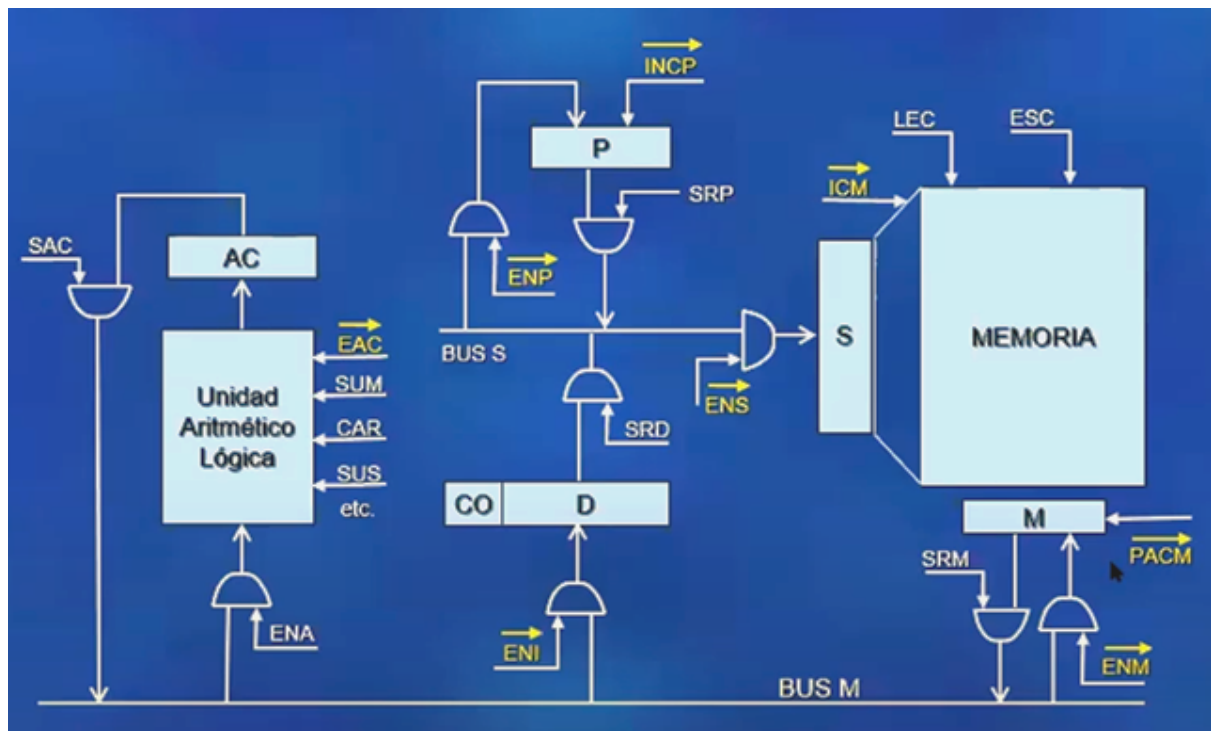
- **Registros de memoria:** son usados para guardar exclusivamente direcciones de memoria. Muy usados en Arq. Harvard, ya que muchas veces las direcciones tienen un tamaño de palabra distinto que la de datos.
- **Registros de propósito general (GPR):** pueden guardar tanto datos como direcciones. Son fundamentales en la Arq. Von Neumann, la mayor parte de las computadoras modernas lo usan.
- **Registros de coma flotante:** son usados para guardar datos en formato de coma flotante,
- **Registros de constantes:** no son editables, son solo de tipo lectura.
- **Registros de propósito específico:** guardan información específica del estado del sistema, como el puntero de pila o el registro de estado.

BUS es una palabra que significa transporte y consiste en un arreglo de líneas lógicas dedicadas a interconectar Órganos de computador entre sí, con el objeto de transportar las configuraciones binarias presentes en sus registros. **Es decir, se utilizan para interconectar varios registros** (unos considerados como registros fuente de información y otros como destinatarios de información).



La UC gobierna el BUS, las señales de gobierno de nivel permiten estabilizar los valores lógicos del Registro A sobre el BUS. ALU controla y manda las señales de gobierno de nivel.

Abacus - Von Newman



Abacus es una máquina organizada en torno a dos buses, que son el bus de memoria (bus M) que transfiere las palabras leídas de, o escritas en memoria y el bus de selección (bus S) que transfiere las direcciones.

Resumen de los registros de Abacus:

S:	Registro de Selección de Memoria	<i>m bits.</i>
P:	Contador Ordinal o de Programa	<i>m bits.</i>
M:	Registro de Intercambio o de Palabra	<i>n bits.</i>
I:	Registro de instrucción con dos partes:	
	CO: Código de Operación	<i>p bits.</i>
	D: Dirección	<i>m bits.</i>
AC:	Acumulador	<i>n bits.</i>

Unidad de procesamiento

AC: Registro acumulador de n bits. Opera contra el registro M y almacena el resultado sobrescribiendo su contenido. Ejemplo: si CO es SUMA entonces $(M) + (AC) \rightarrow AC$

Unidad de control

Registro P: Contador de programa - m bits.

Bus S Bus de direcciones: m bits. Lleva el contenido de Reg P o Reg D para la siguiente instrucción o encontrar un operando que será utilizado contra el acumulador.

Registro S Registro Selección de memoria: m bits. Permite direccionar 2^m palabras de memoria.

Registro de instrucción: n bits. Dividido en Registro D - m bits, y Código de operación (CO) - p bits.

Unidad de Almacenamiento

Memoria: Compuesta por 2^m palabras de n bits.

Registro M: Registro Palabra de memoria - n bits. Pone una palabra de memoria a disposición de la unidad de control o de procesamiento, según corresponda.

Recordando señales de impulso y de nivel se describirán las señales de gobierno (característica más importante) de esta arquitectura Abacus, las de amarillo son de impulso y las blancas de nivel. La unidad de control es aquella que controla las señales de gobierno para que se realicen las respectivas instrucciones. Los cálculos deben ser de nivel porque no alcanza con un pulso para realizar la operación.

ALU

Es la encargada de ejecutar todo tipo de instrucciones de cálculo. Para cumplir esta función dispone de operadores o unidades funcionales, destinadas a realizar una o varias operaciones aritméticas o lógicas. Es un órgano compuesto de muchísimos niveles de circuitos lógicos combinacionales y secuenciales dedicados a resolver operaciones de cálculos aritméticos y lógicos entre las unidades funcionales dedicadas a esta tarea.

✿ **Última operación realizada.** Permite informar si fue realizada o no la última operación, sirve para saber si se encuentra disponible o no el operador.

✿ **Error de Desbordamiento.** Indican que el valor, el resultado del acumulador se excede por un dígito y es necesario hacer un desplazamiento para acomodarlo.

✿ **Error de Desborde de Capacidad.** Es cuando directamente el registro se ve totalmente desbordado, entones la unidad de control debe tomar alguna acción para poder preservar ese desbordamiento que excede a más de un bit.

✿ **Error de División por Cero.** La ALU entra en error si intenta de dividir algo por 0.

✿ **Estado Acumulador Positivo.**

✿ **Estado Acumulador Negativo.**

✿ **Estado Acumulador Igual a 0.**

Son fundamentales para tomar decisiones de salto por positivo, por negativo o por cero. Es la única base para actuar con inferencias lógicas. Son importantísimos al tener en cuenta su diseño

Semi-Sumador

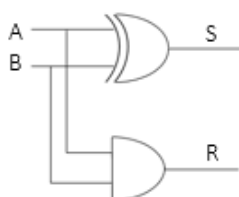
El semisumador es un circuito lógico que realiza la suma de dos bits del mismo peso sin acarreo. Sirve para hacer la suma de los primeros bits de una cadena.

Entradas		Suma	
A	B	R	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Ecuaciones:

$$S = (\bar{A} + B) \cdot (A + \bar{B}) = A \oplus B$$

$$R = A \cdot B$$



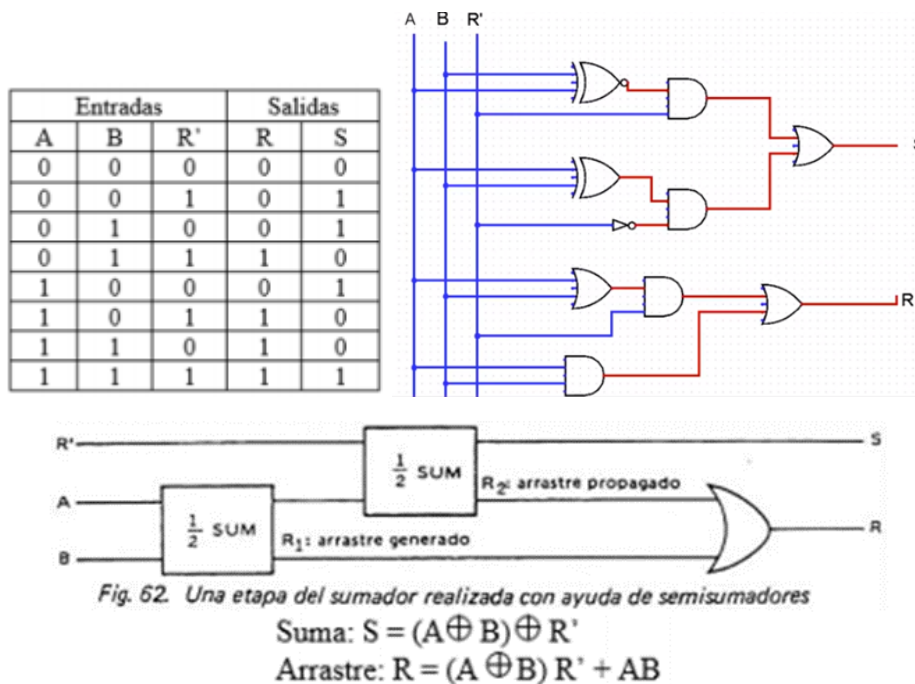
Forma simplificada



Símbolo

Sumador-Completo

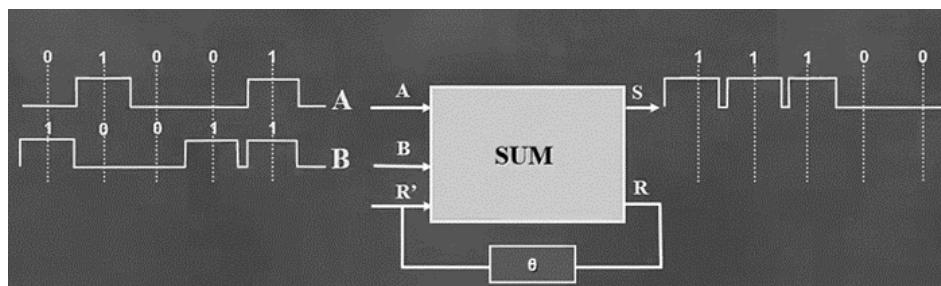
Para realizar la suma de dos dígitos binarios es preciso tener en cuenta, en cada posición n , los dos dígitos de dicha posición y el arrastre de la posición $n - 1$.



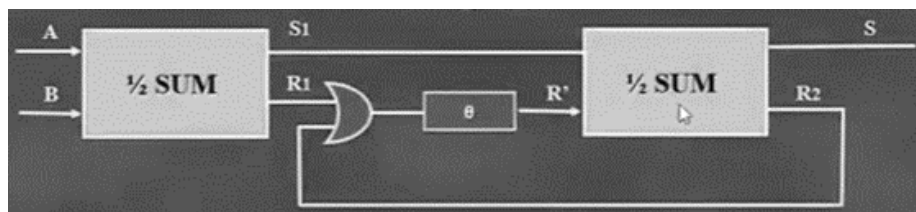
Sumador-Serial

Las operaciones en serie son más lentas y requieren menos equipo. El circuito sumador en serie usa solamente un circuito sumador completo y un elemento acumulador para conservar el arrastre de salida generado. El método en paralelo usa n circuitos sumadores completos y todos los bits de A y B se aplican simultáneamente.

Las entradas A y B son trenes de pulso, las cuales contienen las cadenas que se pretenden sumar.



Ventaja: Simpleza y economía del circuito con un solo sumador de dos compuertas (ORX, ORX) más una suma y multiplicación lógica (AND y OR) logramos hacer un sumador completo con un circuito que permita llevar el acarreo. De una forma más económica según Meinadier, también puede evitar el lag.

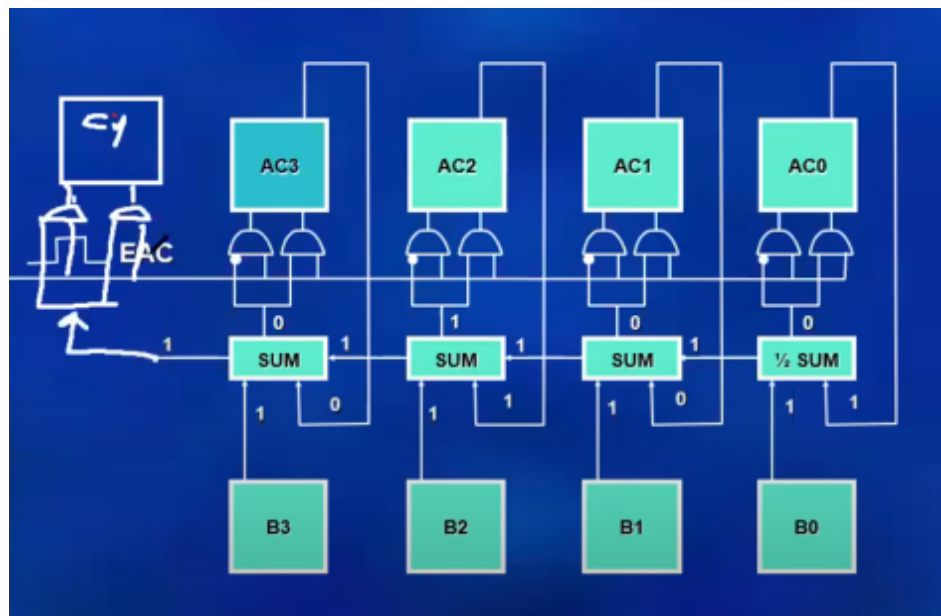


Desventaja: Consume muchos ciclos de CPU y mantiene ocupado al procesador lo que dure la suma, dependiendo del tren de entrada N ciclos y si lo guardo $N+1$.

Sumador-Paralelo

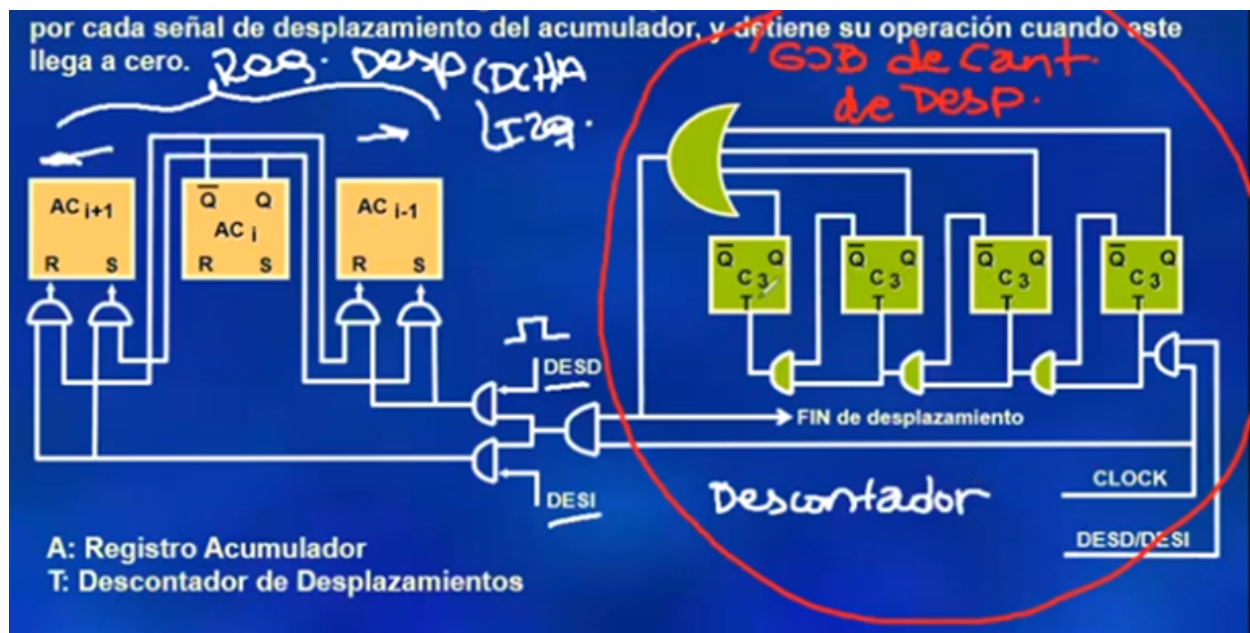
Este consiste en sumadores completos conectados en cascada con la salida de arrastre de un sumador completo conectado al arrastre de entrada del siguiente sumador completo ubicado a la

izquierda. Un sumador completo de n bits requiere n sumadores completos conectados en cascada. Es mucho más caro, pero puede resolver la suma en un batido.

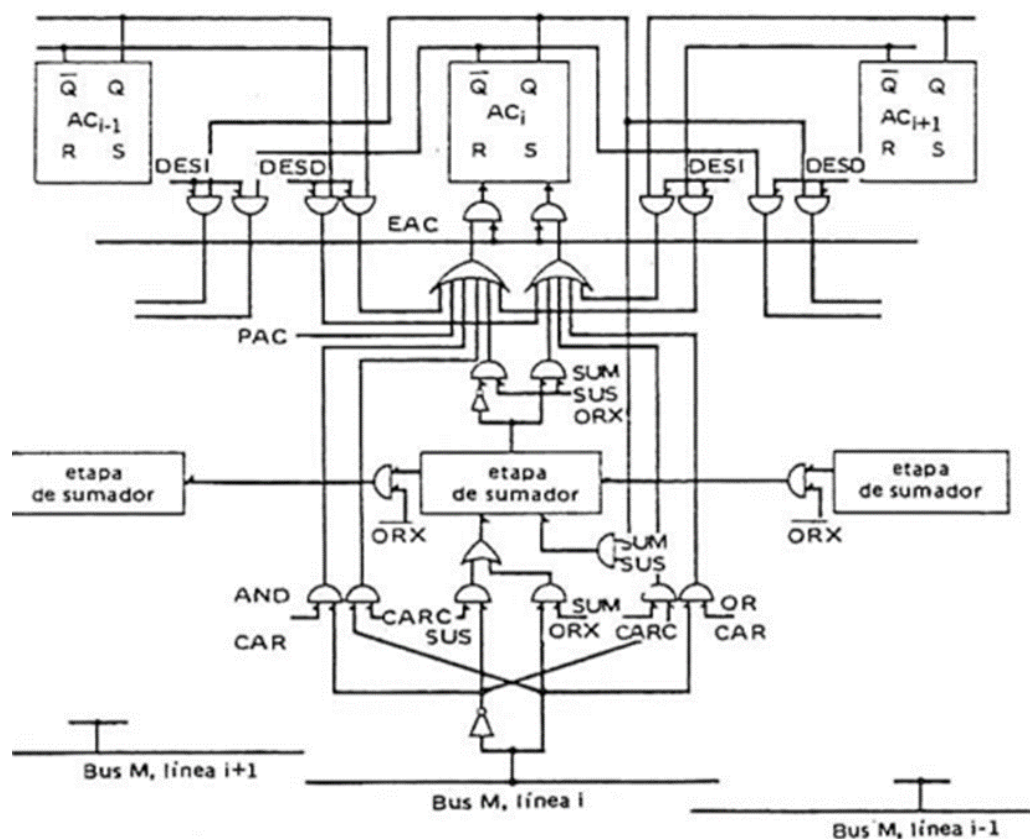


Gobierno Secuencial de Desplazamiento

Consiste en un registro de desplazamiento le permite ir a derecha o a izquierda, depende desde dónde se active (DESD o DESI). El valor binario en el descontador le dice cuántas veces se puede desplazar. Es un acumulador montado como un registro desplazamiento. El contador se ve retrasado por cada señal de desplazamiento del acumulador, y detiene su operación cuando este llega a 0.



ALU-Abacus



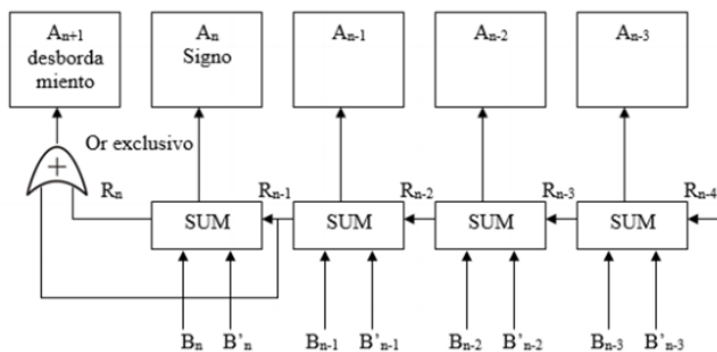
SUM/SUS en Punto Fijo

Para poder desarrollar un circuito sumador/sustractor es necesario:

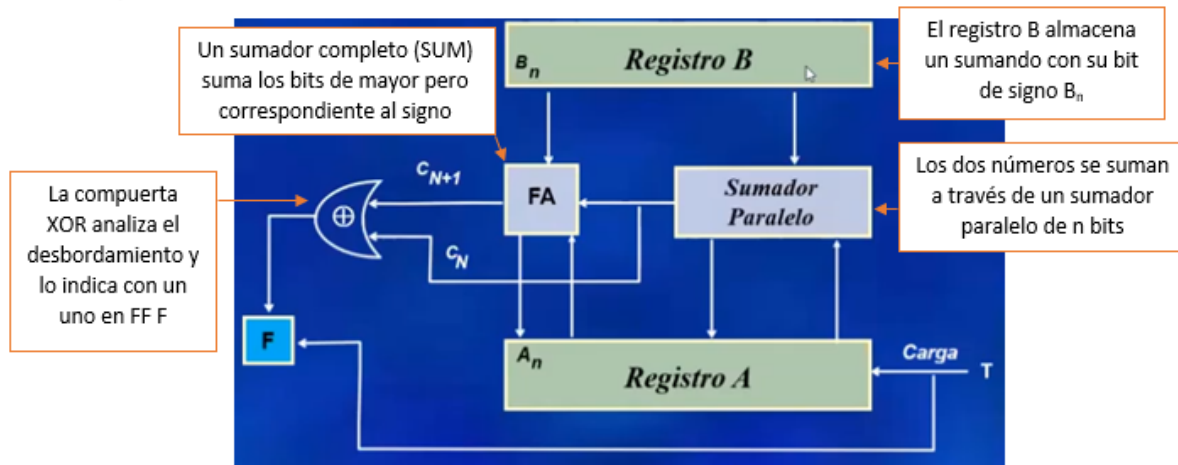
- Expresar las magnitudes positivas en valor absoluto indicar el bit de mayor peso su signo a través de un 0
- Expresar las magnitudes negativas por complemento absoluto e indicar el bit de mayor peso su signo a través de un 1
- Verificar situaciones de desborde a través de analizar el arrastre del dígito de mayor peso de la magnitud junto del arrastre de la suma de los signos, detectando situaciones donde alguno de ellos sea igual a 1

Sustraer un número de otro equivale a sumarle su opuesto. La sustracción se reduce a la suma algebraica siempre que se sepa calcular el opuesto de un número, lo que generalmente se hace por complementación. Este método evita tener que situar un Sustractor junto al sumador.

Supongamos que nuestra unidad aritmética procesa números representados por n bits, con el bit de signo situado en la primera posición a la izquierda. Asociaremos al acumulador un $n+1$ -ésimo bit, que actuara de indicador de desbordamiento.



Condiciones de signo	2 operandos > 0 El resultado debe ser positivo	1 operando > 0 1 operando < 0. No puede haber desbordamiento	2 operandos < 0 El resultado debe ser negativo
Representación inicial (registros de 5 bits)	0---- 0----	1---- 0----	1---- 1----
Resultado si no hay arrastre procedente de $n - 1$	0---- Válido: resultado positivo	1---- Válido: resultado negativo	10--- No válido puesto que resultado positivo, hay desbordamiento.
Resultado si hay arrastre procedente de $n - 1$	1---- No válido: puesto que resultado negativo, hay desbordamiento	10--- Válido: olvidándose del arrastre. Resultado positivo	11--- Válido: olvidándose del arrastre. Resultado negativo



Debido a la longitud fija del tamaño de los registros del computador, estos deben verificar situaciones de sobre capacidad e indicar esta situación en un biestable diseñado a tal efecto. La condición de Sobrecapacidad o Desbordamiento se detecta observando el arrastre producido por el bit más significativo de la Magnitud y el del Bit del Signo.

NO Puede haber Sobrecapacidad o Desbordamiento después de una suma si un número es positivo y el otro es negativo.

SI Puede haber Sobrecapacidad o Desbordamiento después de una suma si ambos números son positivos o negativos.

Coma Flotante

Recordemos que los números en coma flotante se representan en la forma:

$$SM\alpha^E$$

S es el signo del número

M la mantisa

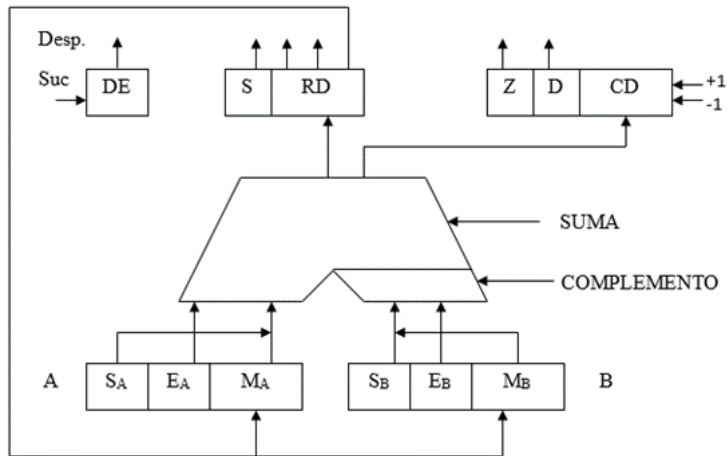
α la base (generalmente 2 o 16 en aritmética binaria).

E el exponente

S	E	M
---	---	---

Las operaciones se llevan a cabo sobre **números normalizados**, esto es, sobre números cuyo valor de exponente está ajustado para que la mantisa tenga el mayor número posible de dígitos significativos (**cuando el valor de binario es su signo y el bit más significativo de su mantisa son distintos**). Por consiguiente, los operadores en aritmética flotante deben, no solamente ejecutar la operación correspondiente, sino también **normalizar el resultado obtenido**.

SUM/SUS



S: Signo del operando (0 es + y 1 es -)

E: Exponente

M: Mantisa

CD: Contador/Descontador con:

(D) Desborde / (Z) Indicador de Cero

Si Z=1 indica resultado 0

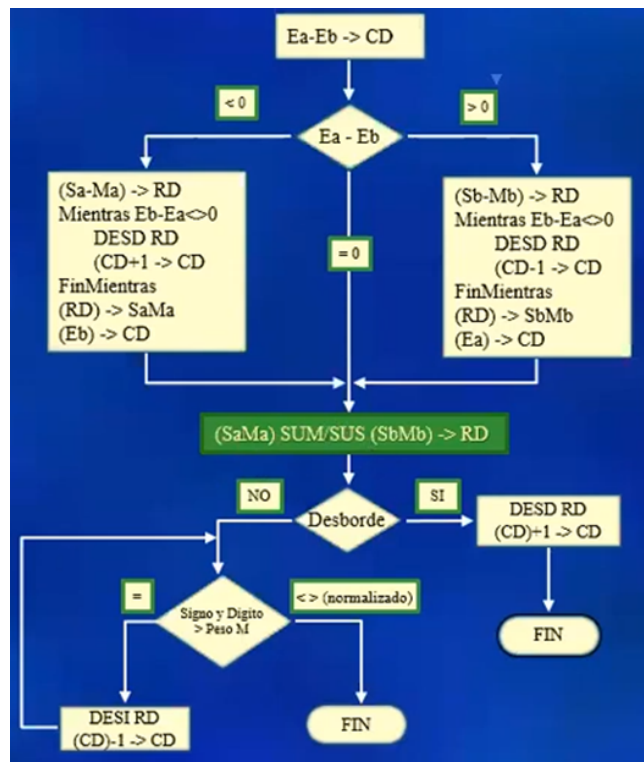
Si Z=0 se analiza campo D

Si D=1 resultado $E_a - E_b < 0$

(resultados negativos se caracterizan por tener desborde)

Si D=0 resultado $E_a - E_b > 0$

RD: Registro de Desplazamiento derecha o izquierda con
(DE) Bit de Desborde.



La operación se divide en varias fases:

Este operador utilizado comprende dos registros A y B para los operandos y un sumador $m + 1$ bits que ejecuta las sustracciones por complementación previa del segundo operando.

1. Comparación de los exponentes: Se resta E_B de E_A (sumando a E_A E_B complementado), y el resultado va a CD. Si el resultado es nulo, las dos mantisas están correctamente alineadas y se pasa inmediatamente a la fase de suma o sustracción de las mantisas. Si no, el contenido de CD permitirá contar los desplazamientos necesarios para alinear las mantisas.

2. Alineamiento de mantisas: Esta operación se realiza sobre el operando cuyo exponente es menor. Si $E_A < E_B$ el resultado de la sustracción de los exponentes es negativo y se caracteriza por el valor 1 del bit de desbordamiento. El complemento del contenido de CD indica el número de desplazamientos por efectuar.

El desplazamiento de la mantisa se efectúa en RD $\Rightarrow$ se envían los contenidos de S_A y M_A yuxtapuestos a RD y se opera por desplazamiento aritmético hacia la derecha (lo que permite reproducir el signo en los bits no significativos y por lo tanto conservar la representación complementada).

En el caso $E_A > E_B$, el desplazamiento aritmético a la derecha afecta a S_B y M_B , pero en esta ocasión el registro CD se emplea como descontador, deteniéndose los desplazamientos cuando $CD=0$ (nótese que después de todo, pueden detenerse al cabo del m -ésimo desplazamiento ya que a partir de entonces la mantisa pierde todo su significado).

Esta fase finaliza preservando el mayor exponente en CD y devolviendo la mantisa contenida en RD al registro A o B que corresponda.

3. Suma-sustracción de las mantisas: Estas operaciones se ejecutan sobre el conjunto formado por el signo y mantisa, yuxtapuestos. La sustracción se obtiene sumando S_A M_A el complemento auténtico de S_B M_B . El resultado a RD, tomando en cuenta el signo y un eventual desbordamiento.

Las operaciones de normalización se realizan mediante desplazamientos sobre el registro RD, que contiene el signo y la mantisa del resultado, y sobre el registro CD, que contiene el exponente.

4. Normalización en el caso de desbordamiento: Consiste en desplazar la mantisa del resultado en una posición a la derecha, siendo sustituido el bit de mayor peso por el de desbordamiento. El signo queda inalterado. El contenido del registro CD se ve aumentado en una unidad.

5. Normalización en caso que la mantisa del resultado contenga dígitos no significativos a su izquierda: esta eventualidad es detectable haciendo un test sobre el signo y el dígito de mayor peso de la mantisa. Si son diferentes, el número ya estaba normalizado. Si no, es preciso realizar desplazamientos aritméticos de la mantisa hacia la izquierda hasta que sean diferentes. A cada desplazamiento se resta una unidad del contenido de CD. El proceso se detiene incondicionalmente al cabo de m desplazamientos.

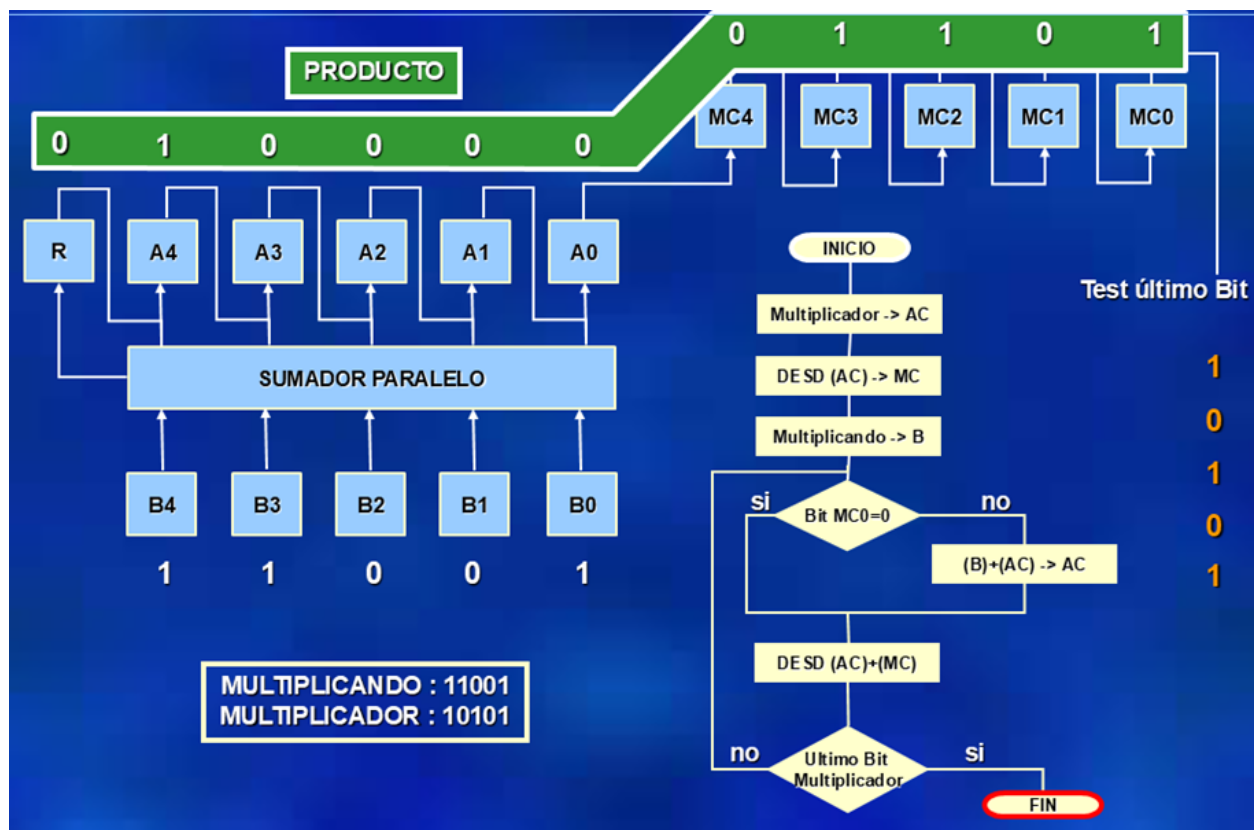
MULT

La multiplicación flotante es reducible a la multiplicación de las mantisas y a la suma de los exponentes. No hay operación preliminar de comparación de exponentes o de alineamiento de las mantisas.

La operación se inicializa de la siguiente manera:

- 1) Carga del multiplicador en el acumulador.
- 2) Desplazamiento a derecha del contenido del conjunto formado por el acumulador y el registro MC, cuyo resultado es poner 0 en el acumulador y el multiplicador en MC.
- 3) Carga del multiplicando en B (que, eventualmente, pudiera ser sustituido por el RPM).

Al final de la operación, el resultado de la multiplicación ocupa en doble longitud el conjunto acumulador + MC.



La ventaja de este modelo es que es preciso, relativamente económico y lo más importante es que es muy sencillo. Es solamente un circuito de control añadido a la ALU para realizar la operación.

La desventaja es que es poco eficiente, muy serial por los repetidos desplazamientos a derecha lo que requiere muchos batidos de reloj a medida se aumenta el tamaño de la operación.

HAY DOS TÉCNICAS MÁS RÁPIDAS:

(1) Basándose en la técnica tradicional de suma-desplazamiento y disminuyendo el número de operaciones parciales sucesivas.

- **Caso de una sucesión de 0.** La mejora consiste en dotar al acumulador de circuitos para el desplazamiento de varias posiciones en una sola operación. En estas condiciones, la sucesión de 0 será tratada por un solo desplazamiento.

- **Caso de una sucesión de 1.** Se emplea el método llamado de “suma y sustracción”. En el esquema tradicional una serie de 1 contiguos da lugar a una serie de adiciones, cada una seguida de un desplazamiento.

(2) Realizando un operador celular para una multiplicación casi paralela.

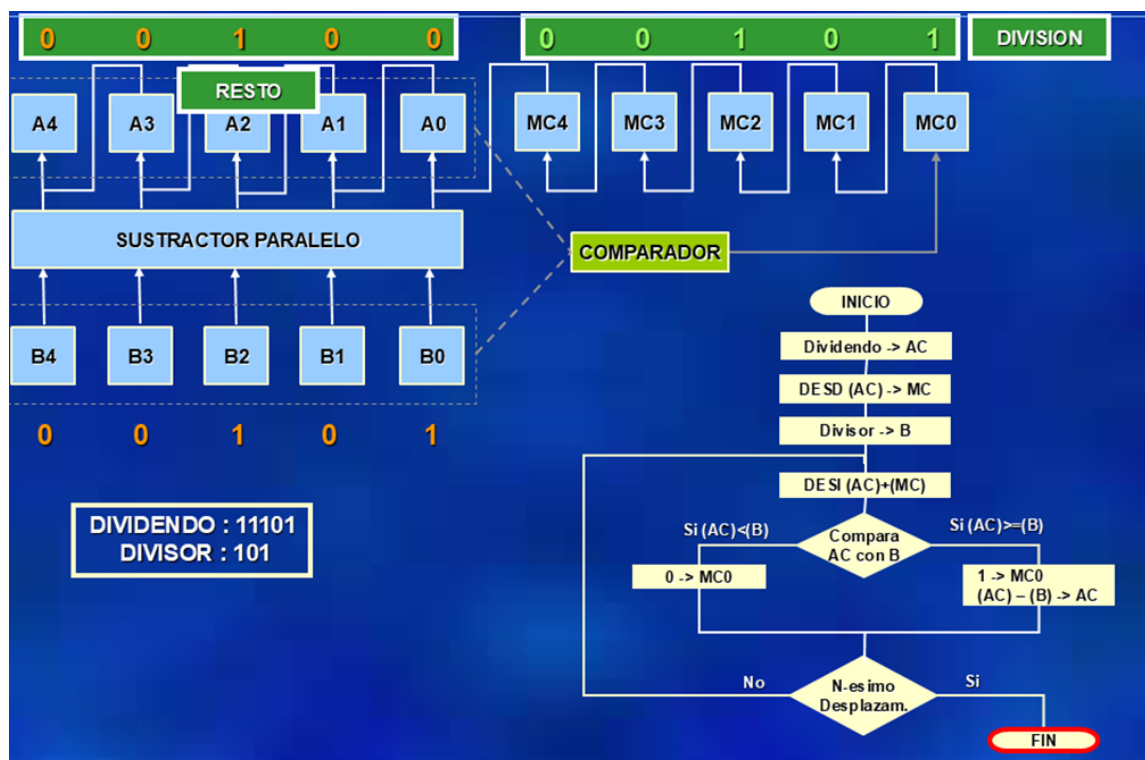
Es factible un circuito combinacional con dos entradas (el multiplicando y el multiplicador) que de el resultado a la salida en una sola operación. Esta sucesión, mucho más onerosa que la anterior, puesto que se multiplica por n el número de etapas de sumador, ofrece la ventaja de ser muy rápida: aproximadamente el doble del tiempo de una suma de n dígitos, ya que un arrastre atraviesa como máximo $2n+1$ etapas de sumador

DIV

La **división flotante** se reduce a la división de las mantisas y a la sustracción de los exponentes. No hay operación preliminar de comparación de exponentes o de alineamiento de las mantisas.

La multiplicación y división son operaciones muy semejantes. Basta cambiar sumas en sustracciones e invertir el sentido de los desplazamientos. **Sin embargo, en la división aparece una dificultad complementaria:** es necesario añadir a cada etapa una operación de comparación entre los bits de mayor peso de dividendo y divisor **a fin de determinar el bit correspondiente del cociente**

⇒ Para efectuar una división tomaremos de nuevo la unidad aritmética utilizada con la multiplicación (casi siempre es común la unidad para ambas operaciones); pero transformaremos el sumador en sustractor, invertiremos el sentido del desplazamiento y añadiremos el dispositivo de comparación.



Generalmente la operación se inicializa de la manera siguiente:

- 1) Carga del dividendo en el Acumulador
- 2) Desplazamiento a derecha del contenido del conjunto acumulador + MC, con objeto de poner a cero el acumulador y cargar el dividendo en el MC.
- 3) Carga del divisor en B.

La operación comenzará por un **DESI del AC + MC**, lo que tiene por consecuencia poner el bit de mayor peso del dividendo en A₀ y liberar el biestable MC₀. Se **compara el contenido del AC con el de B**.

Si $(AC) < (B)$, se pone MC₀ a cero sin tocar el acumulador.

Si $(AC) \geq (B)$, se resta del acumulador el divisor, se pone MC₀ a 1, etc.

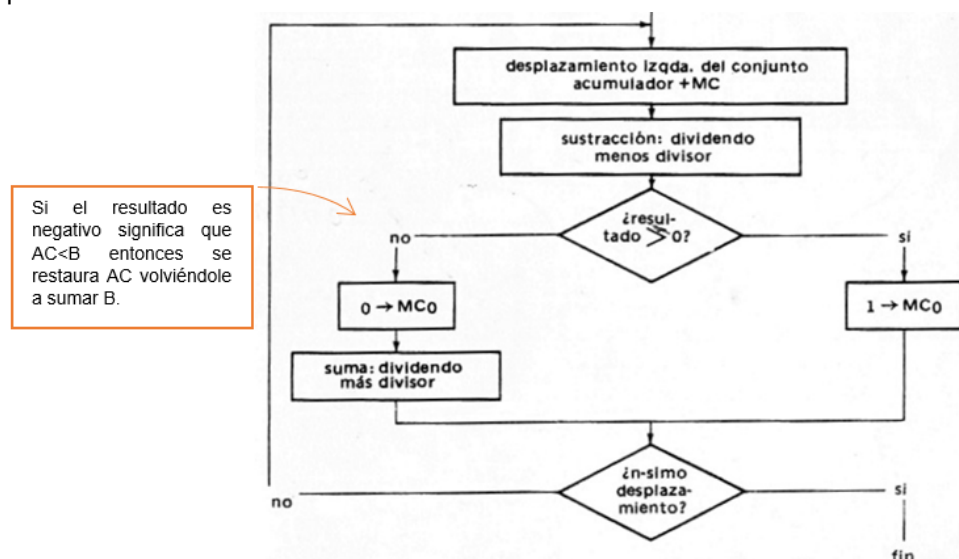
Al final de la operación, el registro multiplicador cociente alberga al cociente y el resto aparece en el acumulador. **Pero obtuvimos un dividendo de doble longitud** ⇒ Basta cargar al principio los bits de mayor peso del dividendo en el acumulador, y los bits inferiores al MC. En estas condiciones, se corre el riesgo de cometer un error si el contenido del acumulador es mayor que el divisor: el cociente excedería entonces a la capacidad del MC y se perderían sus bits de mayor peso.

Se evita este error comparando inicialmente los contenidos de acumulador y registro B. A este nivel es cuando pueden detectarse las divisiones por cero. También puede convenirse en no dividir más que por números enmarcados a izquierda, de manera que siempre el bit de mayor peso del divisor sea 1. (Este será el caso en la división de números flotantes).

División CON Restauración (sin comparador)

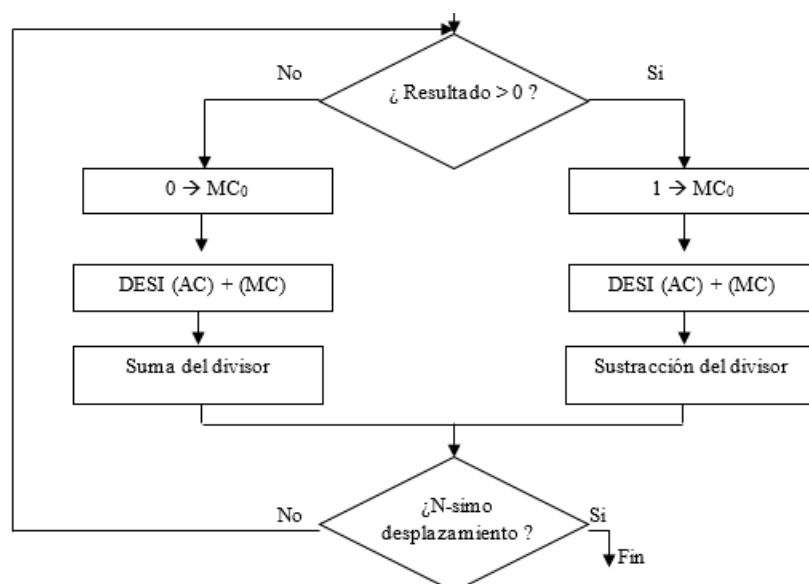
En el punto anterior hemos dado por supuesto que contábamos con un dispositivo para comparar directamente los valores numéricos de los contenidos del acumulador y del registro B. En realidad, casi nunca se cuenta con él por razones de precio, por lo que puede hacerse la comparación intentando restar el divisor del dividendo y comprobando el signo del resultado.

Pero, al contrario de lo que sucede en el papel, la unidad aritmética pierde el antiguo valor del dividendo cuando el intento no surte efecto $\Rightarrow$ es necesario restaurarlo añadiendo el divisor al dividendo antes de hacer un nuevo intento. El problema es que a mayor número de restauraciones, mayor cantidad de ciclos requeridos



División SIN Restauración

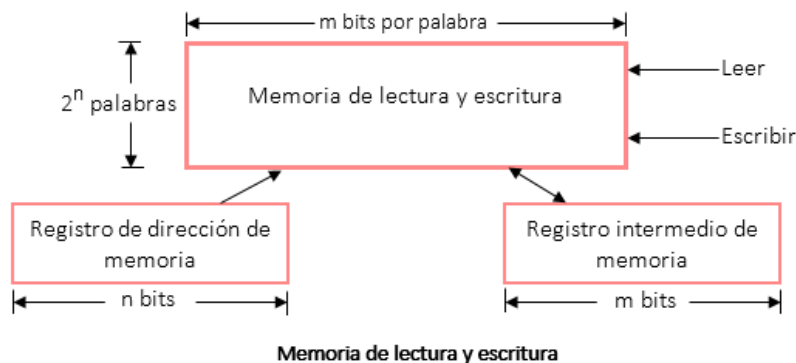
Corresponde a un algoritmo que permite ahorrar la fase de restauración después de una sustracción con resultado negativo, lo que supone una ganancia de tiempo. **Un intento dando resto positivo** irá seguido de la sustracción del divisor desplazado, mientras que **un intento resultando en resto negativo** deberá seguirse de la suma del divisor desplazado.



Memoria

Es un dispositivo electrónico capaz de almacenar de manera temporal una información binaria de tal forma que todo órgano que requiera servirse de él, pueda acceder a la información solicitada en cualquier momento $\Rightarrow$ **La memoria está formada por un conjunto de células o registros, capaces de almacenar información (en general una palabra). La información guardada podrá ser un dato o una instrucción**, sin embargo, las instrucciones están en una zona y los datos en otra. Los registros de almacenamiento en una unidad de memoria se llaman registros de memoria.

Cada célula de memoria tiene un número que la identifica (su dirección), que es habilitada por la unidad de control, ya que conoce a cada célula por su número. Esta habilitación nos permite leer o escribir información contenida en una dirección determinada.



Punto de memoria

Es el soporte físico que permite almacenar la mínima unidad de información binaria. Deberá contar con la lógica necesaria para poder leer o escribir información binaria en él, tal el caso que los antiguos Núcleos de Ferrita o el Biestable Electrónico.

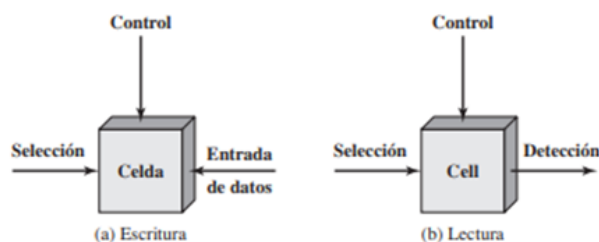


Figura 5.1. Funcionamiento de una celda de memoria.

Palabra de Memoria

Es el conjunto de bits que la máquina trata como unidad básica de información. Mínima unidad de información que la máquina puede almacenar, transferir y transformar (definición de Vicente). Una palabra de memoria puede representar un operando, una instrucción, o un grupo de caracteres alfanuméricos o cualquier información codificada binariamente.

Jerarquía de Memorias

En un ordenador hay una jerarquía de memorias atendiendo al tiempo de acceso y a la capacidad que normalmente son factores contrapuestos por razones económicas y en muchos casos también físicas. Las restricciones de diseño de la memoria de un computador se pueden resumir en tres cuestiones:

1. ¿Cuánta capacidad?
2. ¿Cómo de rápido?
3. ¿De qué coste?

La última de las cuestiones anteriores también debe tenerse en cuenta. *En la práctica, el coste de la memoria debe ser razonable con relación a los otros componentes.* Como es de esperar, existe un compromiso entre las tres características claves de coste, capacidad y tiempo de acceso.

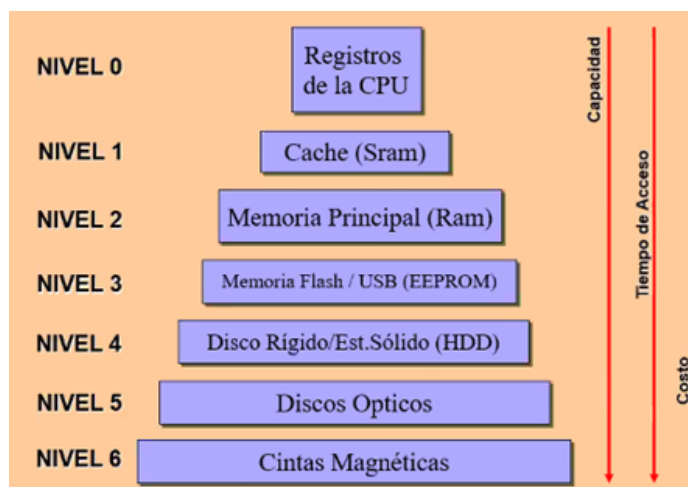
En todo el espectro de posibles tecnologías se cumplen las siguientes relaciones:

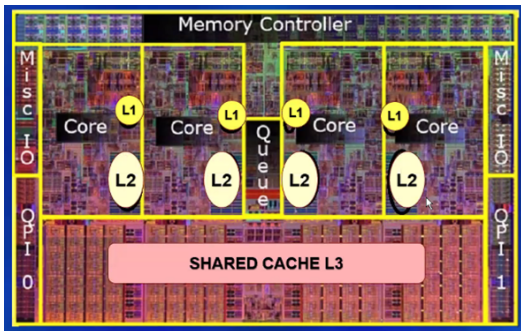
A mayor capacidad, menor
coste por bit.

A mayor capacidad, mayor
tiempo de acceso.

A menor tiempo de acceso,
mayor coste por bit.

Así pues, las memorias más pequeñas, costosas y más rápidas, se complementan con otras más grandes, más económicas y más lentas.



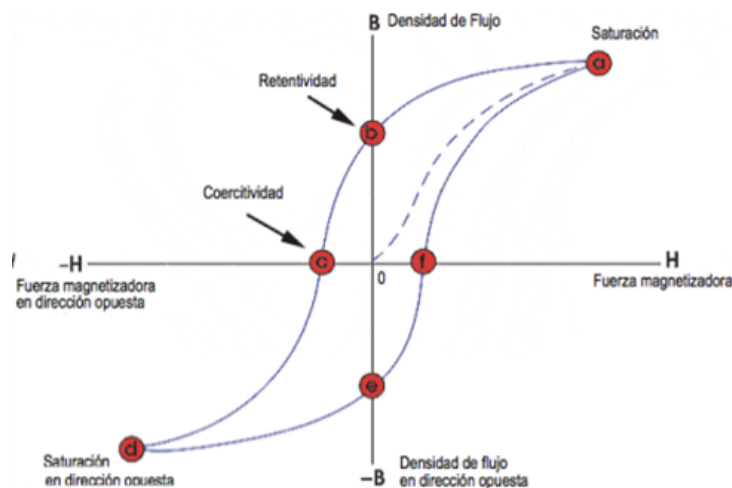
0	Hardware → Registros de la arquitectura Son caros, tienen un tiempo de acceso muy pequeño y una capacidad mínima. Interactúan con los buses, permiten mover y operar con la información. Intermediario entre procesador y memoria.		
1	Memoria caché → SRAM Tiene una altísima velocidad, y menor capacidad de memoria, que permite amortiguar las asimetrías de velocidad y tiempos de accesos. Libera la RAM de estar leyendo continuamente, sirven como amortiguador ⇒ Su función es mejorar el rendimiento del Procesador Central sin aumentar el costo del sistema. Normalmente almacena datos e instrucciones que el procesador va a necesitar para completar diferentes tareas a corto plazo. 		
	L1 Es el nivel más básico, rápido y caro, y el de menos capacidad, está más cerca del procesador. Se caracteriza por cachear cálculos e información, libera al núcleo el tener que hacer el cálculo o acceder frecuentemente la memoria.	L2 Presenta un buen equilibrio en cercanía, capacidad y velocidad, pero el tiempo de acceso es más lento.	Shared L3 Tiene una memoria por cada núcleo, guarda cálculos recurrentes; entonces se evita hacer cálculos (o un acceso de memoria) hechos anteriormente. Guarda información de memoria compartida de los núcleos ⇒ Permite integrar el caché a nivel de NÚCLEOS. Si el núcleo 1 quiere determinada información que el núcleo 3 ya lo hizo, L3 le permite acceder a esa data.

2	Memoria Principal → RAM Está altamente vinculada con la arquitectura e interactúa de manera directa con la unidad central de proceso. En este nivel residen los programas y los datos. La CPU lee y escribe datos en él aunque con menos frecuencia que en los niveles inferiores. Las memorias RAM son volátiles, es decir que si se interrumpe la energía se pierden los datos y sólo pueden utilizarse como almacenamiento temporal. Existen dos tipos de RAM: las estáticas y las dinámicas.
3	Memorias Flash Sobre todo lectura porque se desempeñan muy bien en lectura, pero en escritura produce un desgaste en el punto de memoria. Se borran y graban en bloques.
4	Discos Magnéticos Son memorias que residen en dispositivos externos al ordenador, en ellas se archivan programas y datos para su uso posterior. También se usan estas memorias para el apoyo de la MC en caso de que esta sea insuficiente (memoria virtual). Estas memorias suelen tener una gran capacidad pero pueden tener un tiempo de acceso muy lento. Dentro de ellas también se pueden establecer varios niveles de jerarquía.
5	Discos Ópticos Es básicamente un pedazo de plástico lleno de marcas o huecos que un rayo láser puede leer. Cuando más marcas podemos hacer (y leer) en su superficie, mayor será la capacidad de datos que podremos almacenar en él.
6	Cintas Magnéticas Los sistemas de cinta usan las mismas técnicas de lectura y grabación que los discos. El medio es una cinta de poliéster flexible (parecido al usado en ropa) cubierta por un material magnetizable. Las cintas magnéticas fueron el primer tipo de memorias secundarias.

Almacenamiento en Núcleos Magnéticos

El dispositivo básico de almacenamiento en una memoria de núcleos magnéticos consiste en una pequeña pieza toroidal (en forma de anillo), llamada núcleo magnético o anillo de ferrita. El núcleo magnético, almacena información binaria, **es generalmente una pieza sólida de material cerámico ferromagnético.**

Cuando se magnetiza en una dirección un **material ferromagnético**, no vuelve de nuevo a magnetización cero cuando cesa el campo magnético impulsor. **Debe ser impulsado hacia atrás de nuevo a cero mediante un campo con dirección opuesta. Si se aplica al material un campo magnético alterno, su magnetización trazará un bucle llamado ciclo de histéresis.**

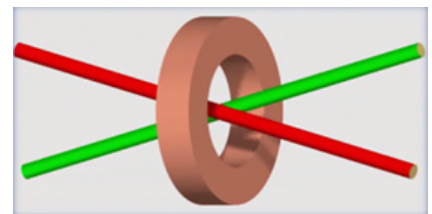
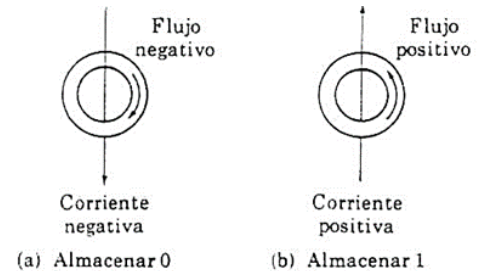


⇒ La corriente que transita a través del alambre produce una inducción eléctrica que produce un B (densidad de flujo). Este ciclo, plantea que si aumenta H (fuerza magnética), entonces el campo B comienza a magnetizarse; este tiene un límite, el cual está dado en el propio material (punto de saturación)

A medida que bajamos H, B baja pero **en una nueva línea** → **el material quedó imantado por la inducción** ⇒ **invertimos**. Para poder llevarlo a su posición original de no magnetismo, tenemos que ejercer una fuerza que lo obligue a ir a su estado normal (el punto de inducción remanente). **Si vamos a H=0 ahora volvemos a una inducción remanente que tuvimos al principio.**

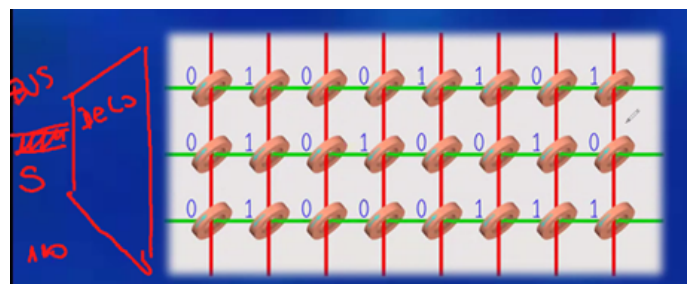
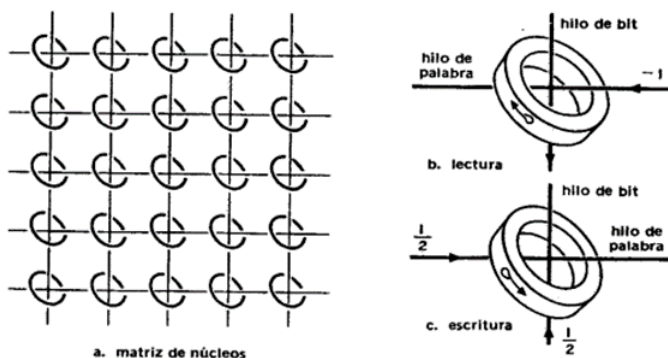
FeO ⇒ El anillo puede magnetizarse. La forma de anillo es para que las cargas recorren una forma propicia que mantengan una carga magnética. Si lo queremos magnetizar ⇒ se le enhebra un hilo conductor y se le hace pasar por él una corriente eléctrica. **ESCRITURA.**

Para averiguar el estado o sentido de magnetización del anillo se enhebra un segundo hilo (que no debe ser paralelo al hilo de escritura para así evitar inducciones indirectas entre ambos) que hará las veces de hilo de lectura. Ese hilo tiene una intensidad que intente magnetizar el anillo ⇒ lo obliga a llevar a B=0 (lo desmagnetiza). Eso es para el proceso de **LECTURA**. PERO → Para leer el contenido, hay que borrarlo. Se arregla mandando otro pulso, pero son memorias muy lentas ya que ocupa dos ciclos. Destruir y escribir de nuevo, es decir: si dicho anillo estaba en el estado, al pasar al 0 su magnetización cambia bruscamente, lo cual induce una corriente apreciable en el hilo de lectura. Si el anillo estaba ya en 0, la magnetización apenas cambia y la corriente inducida es mucho menor. Midiendo esa corriente se puede saber cuál era el estado del anillo.



Implementación 2D

Va a haber un hilo de palabra x cada fila, no por cada anillo como antes (Gobierna una palabra completa), y un hilo por columna (gobierna el bit de la palabra seleccionada). *Menos hilos ⇒ menos compleja la memoria.*



Limitación → El plano

Ventaja → Simplicidad de construcción de la memoria. El plano nos permite hacerla materialmente de manera sencilla (anillos en la misma dirección hilos verticales y horizontales)

PUNTO DE MEMORIA EN 2D

Es una memoria de tipo volátil cuyo punto de memoria está compuesto por un FF. Para que esta celda sea útil, deberá poder ser seleccionada mediante el registro de dirección de memoria y se debe tener un método para controlar cuando las celdas seleccionadas deban ser leídas o escritas. Es decir,

tiene que presentar diferentes funciones para que pueda interactuar con la matriz del punto de memoria para poder leer o escribir.

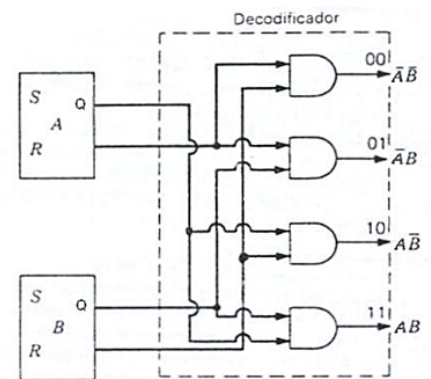
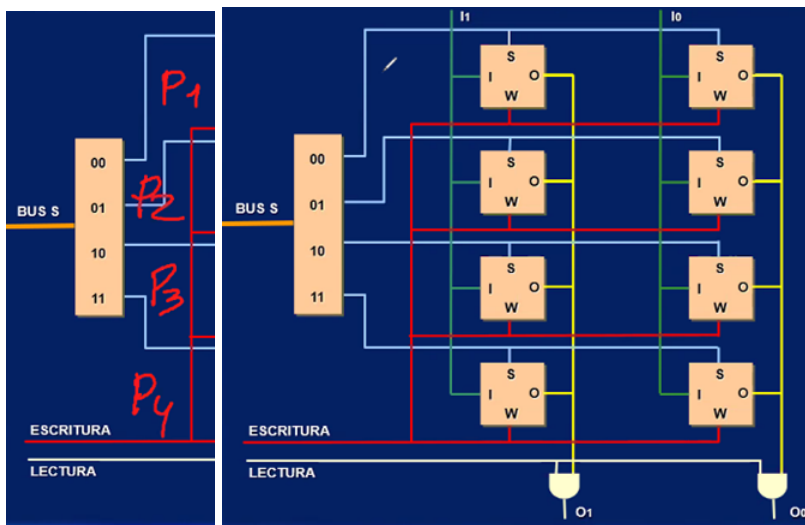
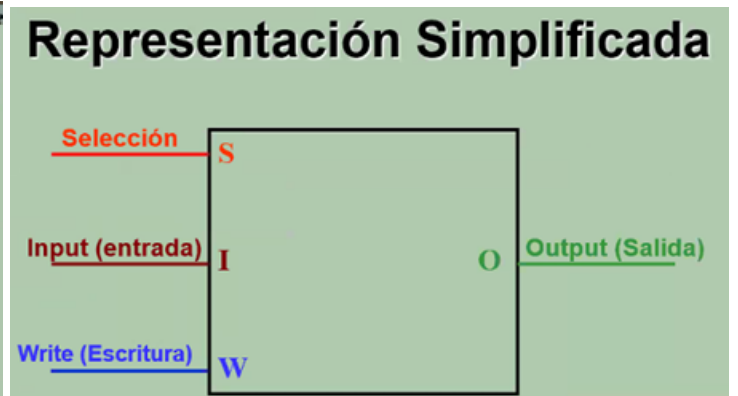
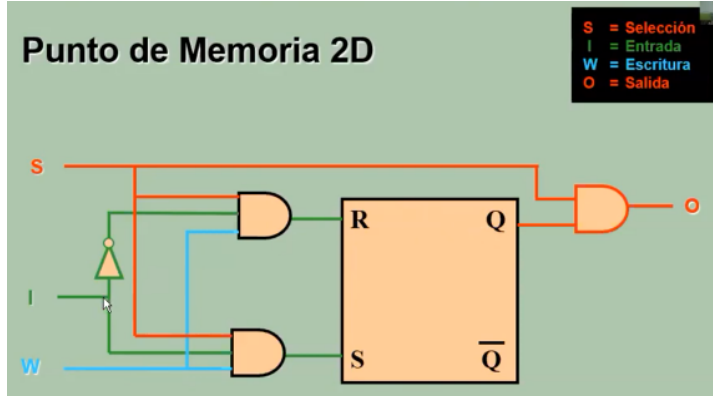
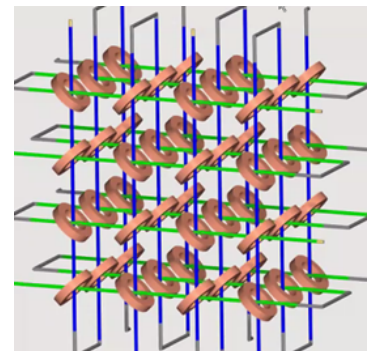


Fig. 7.5. (a) Decodificador de cuatro salidas.

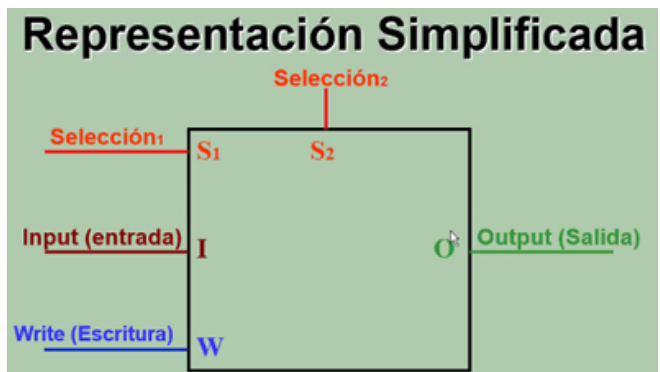
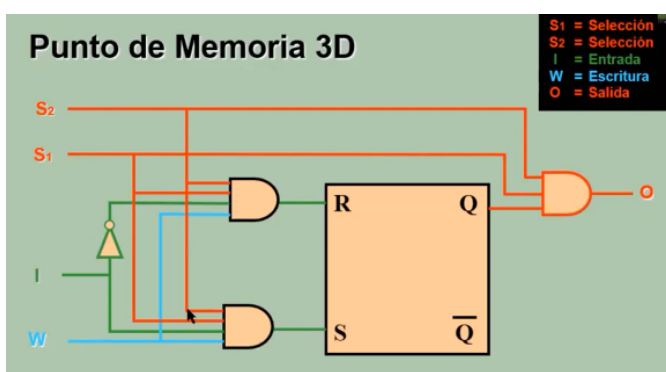
Implementación 3D

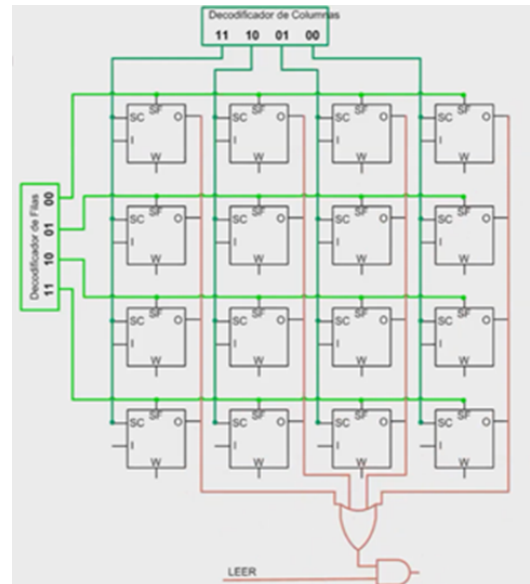
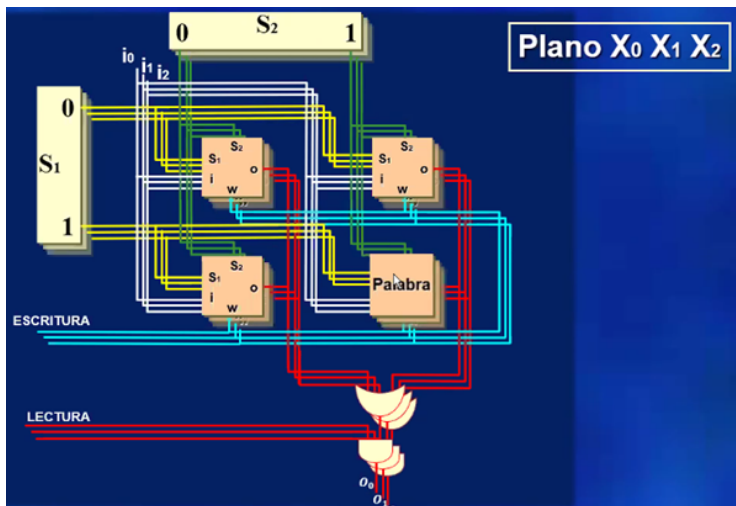
Con el fin de disminuir aún más la cantidad de hilos, la 3D plantea superponer planos colocándolos alternados, uno al derecho y otro al revés. Finalmente se conectan en serie los hilos de selección homólogos uniéndose en lados alternos, de modo que se forme una especie de zigzag con cada hilo. Un decodificador de direcciones X (Horizontal) e Y (Vertical) logran a través de corrientes coincidentes, seleccionar una palabra espacial.



PUNTO DE MEMORIA EN 3D

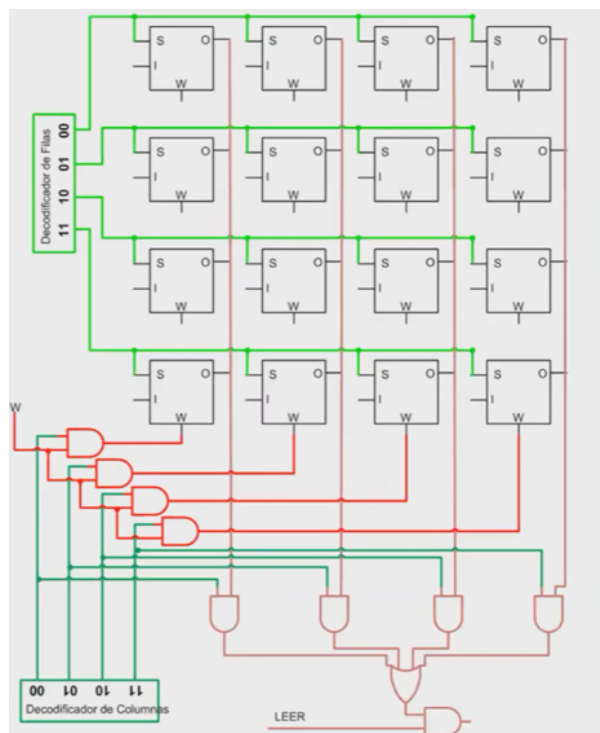
Plantea planos en distintas posiciones espaciales que cada plano representa un tamaño x de memoria. Implica un punto más complejo en lo que se refiere a las entradas, contempla una línea más de entrada S.





PUNTO DE MEMORIA EN 2½D

Es una organización 3d construida a partir de puntos de memoria 2d. Aprovecha los dos puntos fuertes de cada uno. La estrategia consiste en que el decodificador de fila selecciona la fila por la entrada S, el decodificador de columna como NO tiene S, selecciona de forma combinacional.



Direccionamiento

Directo / Absoluto

La referencia al dato que especifica el campo DATA queda sin alteraciones y el tiempo de captación del dato depende sólo del tiempo de un acceso a memoria.

Es la técnica por excelencia y la más sencilla, la palabra seleccionada su posición coincide con el campo DIR.

Inmediato	Se dice que la arquitectura hace un dir. Inmediato cuando obliga al DIR a no ir a S, si no ir a la ALU a operar ese contenido como si fuese un operando; puede contener el operando mismo y se calcula de manera directa con la ALU. Puede contener el operando mismo en DIR.
Indirecto	La dirección del operando no es la dirección efectiva $\Rightarrow$ hace referencia a una palabra de memoria que contenga la dirección del operando $\rightarrow$ 1º busca la dirección 2ª encuentra el operando (tiene dos ciclos)
Base y Desp.	Plantea que la dirección efectiva surge de la operación de registro base y del contenido de DIR (entre desplazamiento y base).
Referencia al programa	Plantea que la dirección efectiva de la palabra de memoria surge de sumar o restar el campo DIR (desplazamiento) con el contador de programa. Esta técnica permite, basándose en el valor del CP tiene un valor fijo de DIR $\Rightarrow$ es muy utilizada para hacer barridos de palabras consecutivas.
Paginación	Concatena el contenido de los bits de mayor peso del contador de programa con el valor DIR. El contador de programa mantiene fijos los de mayor peso y acelera los de menor $\Rightarrow$ la memoria se organiza en 2^p tal que p es la cantidad de bits de mayor peso del contador de programa. 2^p=cantidad de páginas (manera virtual de ver agrupamiento de palabras en una unidad de información) Desplazamiento indica la posición de la palabra dentro de una página. Es fundamental porque permite la multiprogramación, el procesador puede hacer procesos concurrentes, hay distintos procesos que corren en distintas páginas.
Indexado	Consiste en muchos registros bases modificables por el programa. La dirección efectiva surge del operar del contenido del índice con el desplazamiento. En un programa se utilizan algoritmos que involucran índices, por ejemplo, para sumar datos en posiciones contiguas de memoria.

Unidad de Control

La UC extrae de la memoria central la nueva instrucción a ejecutar, la analiza y establece las conexiones eléctricas correspondientes dentro de la ALU, extrae de la memoria central los datos implicados por la instrucción, desencadena el tratamiento de dichos datos en la ALU y, eventualmente, almacena el resultado en MC. La ALU opera con los datos que recibe siguiendo órdenes de la UC.

Al conjunto UC y ALU se lo denomina Unidad Central de Procesamiento (UCP) o Procesador Central (cuando están integradas en una sola pastilla -chip- se denomina microprocesador).

La UC necesita dos registros para operar:

1. Contador de instrucciones o **contador de programa**, que contiene la dirección de la próxima instrucción por ejecutarse. Este registro aumenta su contenido en uno para pasar a la siguiente instrucción.

2. **Registro de instrucción**, que contiene la instrucción extraída de la MC. Este registro tiene (al menos en Abacus) dos partes:

- a. El código de operación, que indica que tratamiento se debe seguir.
- b. La dirección del operando, dónde se encuentra almacenado.

La UC tiene un órgano denominado Secuenciador que, luego de analizar el código de operación, distribuye las órdenes al conjunto de unidades del computador para hacerles ejecutar las distintas fases de la instrucción en curso.

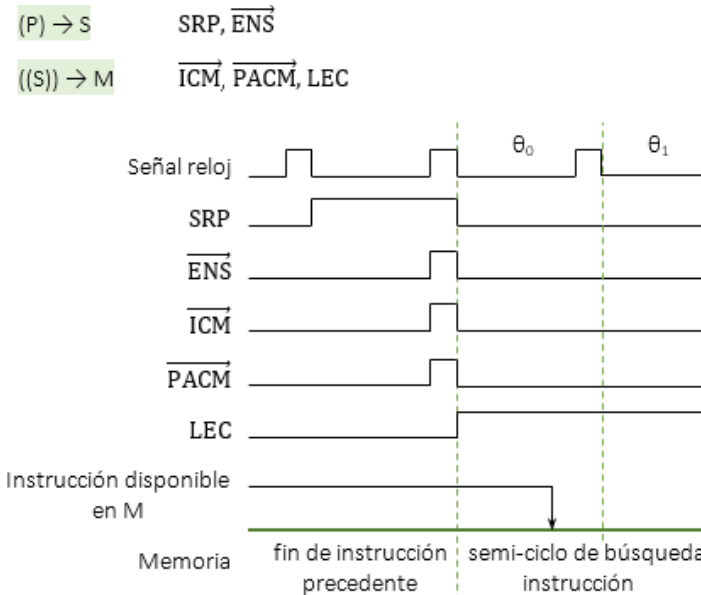
Suma en Abacus

Búsqueda de la Instrucción

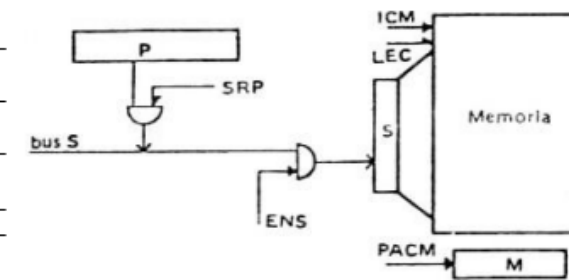
(P) → S. Durante el último θ de la instrucción precedente, la señal **SRP** mantiene abierta la puerta de salida del registro P, por lo que la información de P está latente en el bus S bajo la forma de niveles lógicos. Bastará con validar esta información al primer batido de reloj de la instrucción por medio de la señal de muestreo (de impulso) **ENS** para introducirla en el registro S.

((S)) → M. Ahora el contenido del contenido de S ((S)) (es decir, ubicación del operando) para a M $\Rightarrow$ Será necesario lanzar un ciclo de memoria mediante la señal impulsional **ICM** (inicio de ciclo de memoria), indicar que se trata de una lectura, manteniendo por ejemplo la señal **LEC** durante todo el intervalo del ciclo (y en cualquier caso hasta el inicio de la regeneración) y poner a cero el registro M antes de la lectura de los núcleos (**PACM**) *¿Por qué?* Porque estamos trabajando con núcleo magnético, por lo que no se pueden transferir 0; se desmagnetiza la palabra completa $\Rightarrow$ podemos hacer **LEC** (siempre se necesitan esos dos para la lectura de Abacus).

$\Rightarrow$ **¿Cuánto tiempo dura la lectura en operación suma en abacus?** 4 batidos: 2 para leer la instrucción y luego 2 para leer el operando. Porque está basado en memoria de núcleo magnético, donde la lectura destruye y es requerido un segundo batido para regenerar.



Cronograma de la búsqueda de la instrucción



Búsqueda de la instrucción

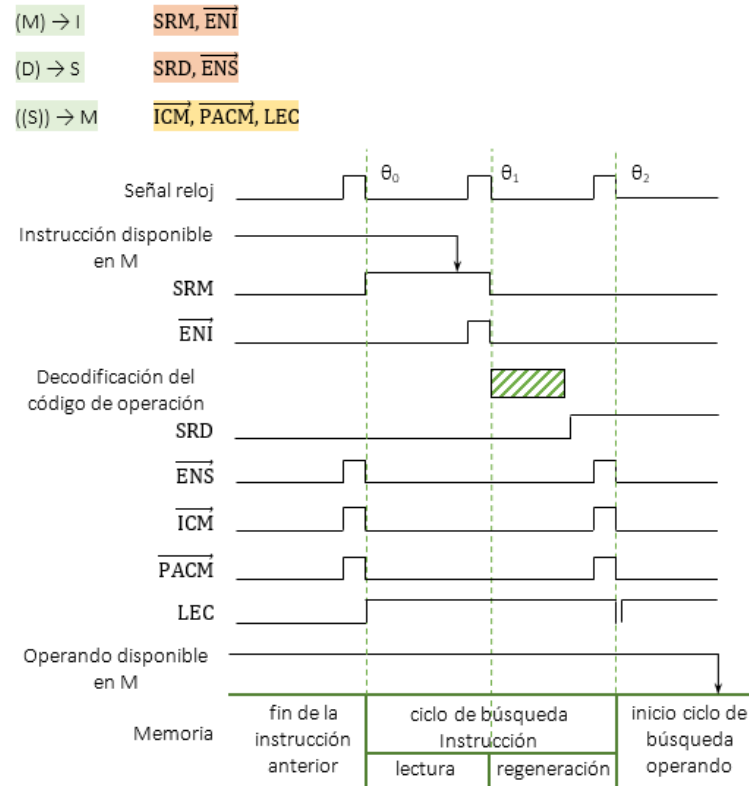
Búsqueda del Operando

Hay instrucciones que no requieren búsqueda del operando (salto condicional, almacenamiento; son como de toma de decisión).

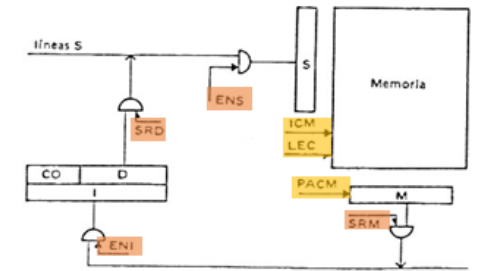
(M) → I. La instrucción se encuentra en el registro M antes del fin del primer θ del ciclo $\Rightarrow$ para ello debemos habilitar salida registro M, con **SRM**, al bus M durante este primer θ , para que pueda forzarse su introducción en I gracias a la señal **ENI**, sincronizada con el segundo batido del reloj del ciclo de memoria.

(D) → S. Aquí la AC debe decodificar el código de operación de la instrucción (el tiempo de esto es suuuper corto) Por consiguiente, es preciso enviar la parte de dirección del registro I al registro S abriendo la puerta de nivel (señal **SRD**) para mantener la información en el bus S. Al siguiente batido se abrirá la puerta impulsional de entrada en S (señal **ENS**).

((S)) → M. Se borrará el registro M (señal **PACM**) simultáneamente con la introducción en S y se lanzará un ciclo de memoria por medio de un impulso **ICM**, indicando que se trata de una lectura gracias al mantenimiento durante todo el nuevo ciclo del nivel **LEC**.



Cronograma de búsqueda del operando



Transferencia de la instrucción y búsqueda del operando

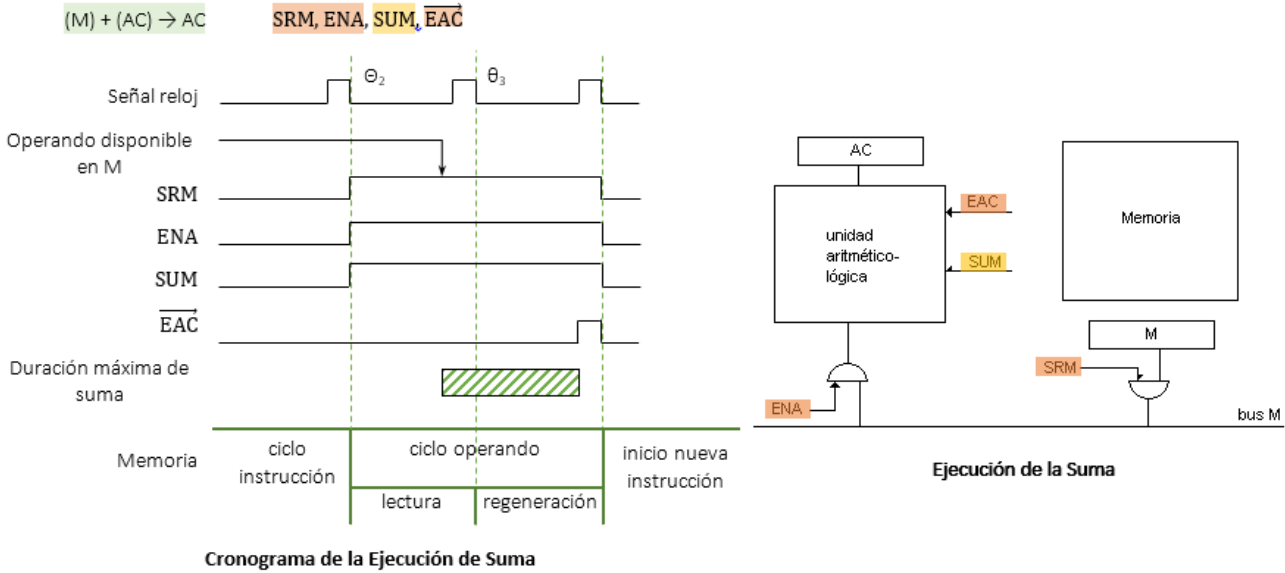
3

Ejecución de la Suma

Se trata de sumar el contenido de M con el contenido de AC: **(M) + (AC) → AC**. Suma entre el contenido de AC y el contenido del contenido de Dir

Los niveles lógicos correspondientes a los registros M y AC durante todo el tiempo de trabajo del sumador, consagrado esencialmente a la propagación del arrastre. Se consigue por las señales **SRM, ENA, SUM**. Terminada la suma, una señal **EAC** introduce el resultado en el AC.

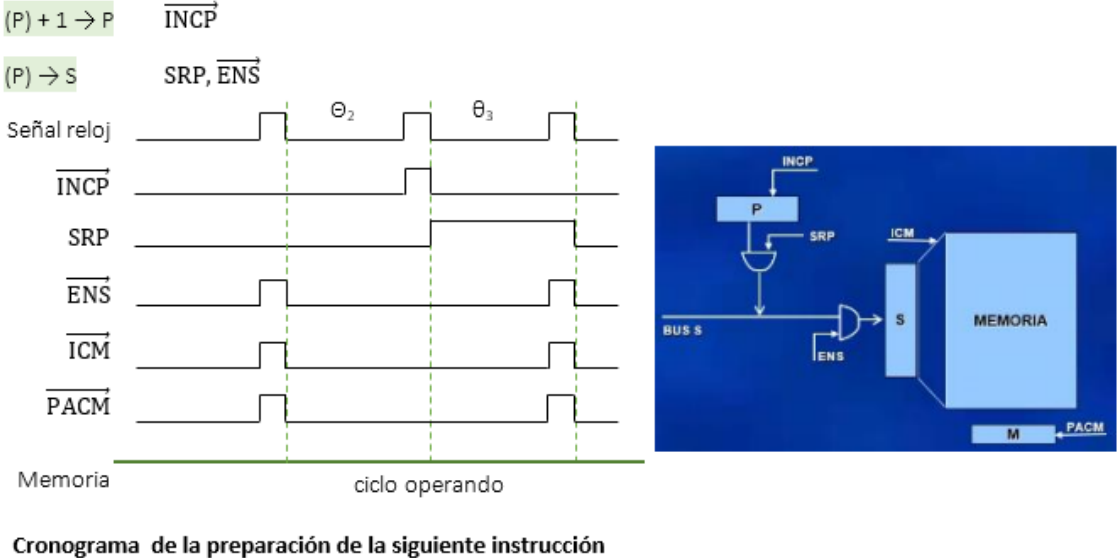
SRM, ENA, SUM son todas de dos batidos porque en un solo batido no se puede hacer todo simultáneamente, no alcanza el tiempo.

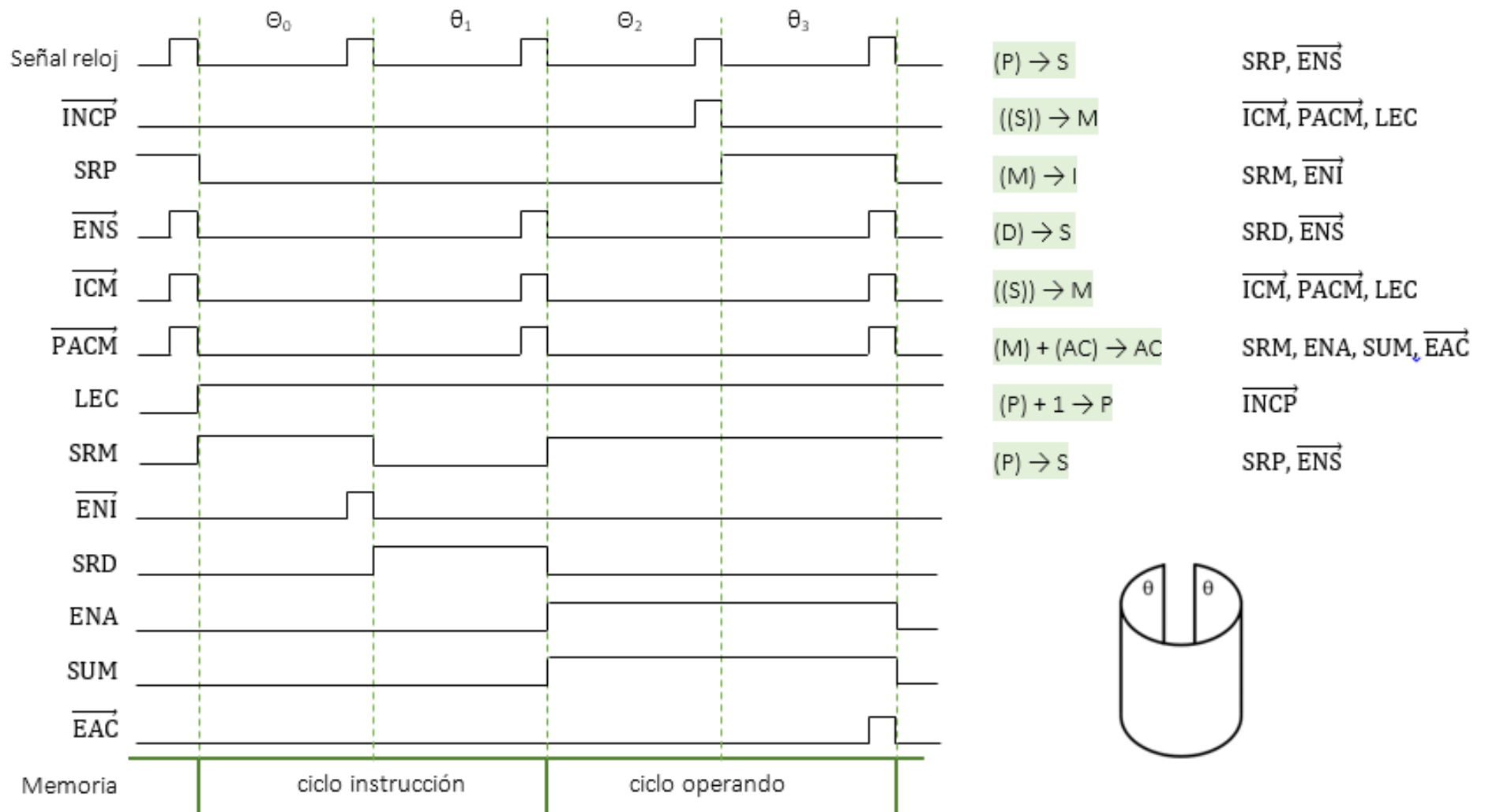


4

Preparación de la próxima instrucción

Esta fase se dedica esencialmente a incrementar el contador de programa en 1: $(P) + 1 \rightarrow P$, enviándole un impulso de cuenta INCP para que al final de la instrucción contenga la dirección de la próxima instrucción. Esta última reanudará nuevamente el proceso: $(P) \rightarrow S$, ciclo de búsqueda de la instrucción, etc.





Cronograma de la suma en Abacus

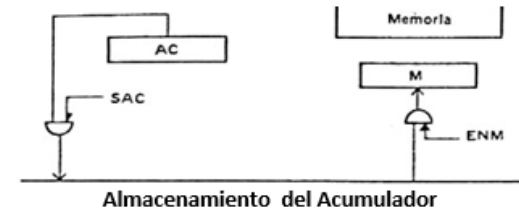
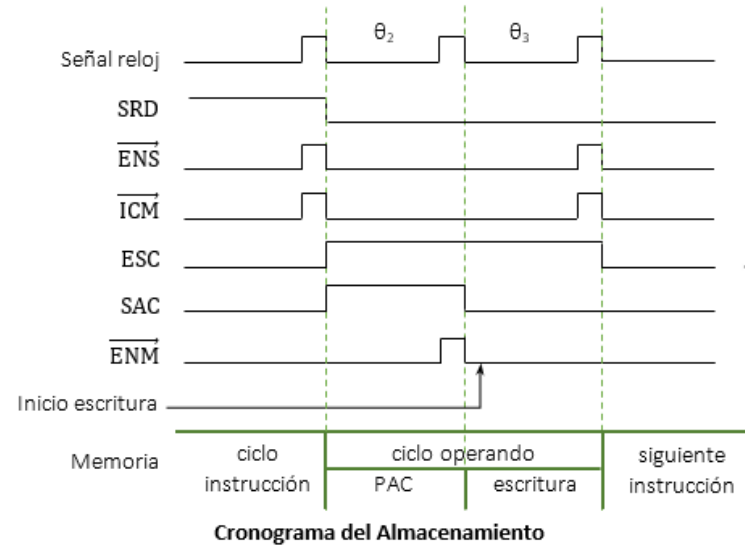


Almacenamiento en Abacus

Almacena el contenido del AC en la palabra de memoria designado por la dirección del registro de instrucción. El primer ciclo es idéntico al de la suma (búsqueda de la instrucción) mientras que el segundo ciclo se dedica seleccionar la palabra designada por la dirección de la instrucción, para luego escribir en ella el valor del acumulador $\Rightarrow$ el segundo ciclo estabiliza la información contenida en el acumulador sobre el bus de datos M (señal de gobierno en nivel **SAC**) para que luego se ha almacenado en la palabra de memoria seleccionado por el campo dirección, a través del registro palabra de memoria M

El valor del AC tiene que habilitarse y salir de ese mismo batido al bus M $\Rightarrow$ se lanza la escritura antes para poner a 0 la memoria, es como hace un lugarcito al lugar donde se graba en la memoria.

(D) $\rightarrow$ S $\overrightarrow{\text{SRD}}, \overrightarrow{\text{ENS}}$
 (M) $\rightarrow$ (S) $\overrightarrow{\text{ICM}}, \text{ESC}$
 (AC) $\rightarrow$ M $\text{SAC}, \overrightarrow{\text{ENM}}$



Salto Incondicional en Abacus

Es una estrategia que permite que desde un programa lenguajes de máquina se le permita indicar a través de SAI la dirección a donde deba saltar el programa y efectuar la ruptura de secuencia lineal $\Rightarrow$ se puede lanzar un salto a OTRA parte del programa (es el famoso Go to)

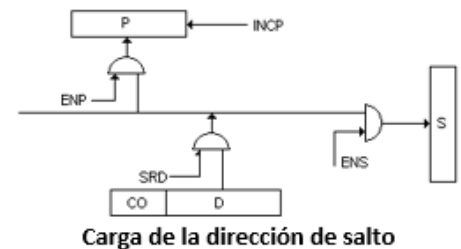
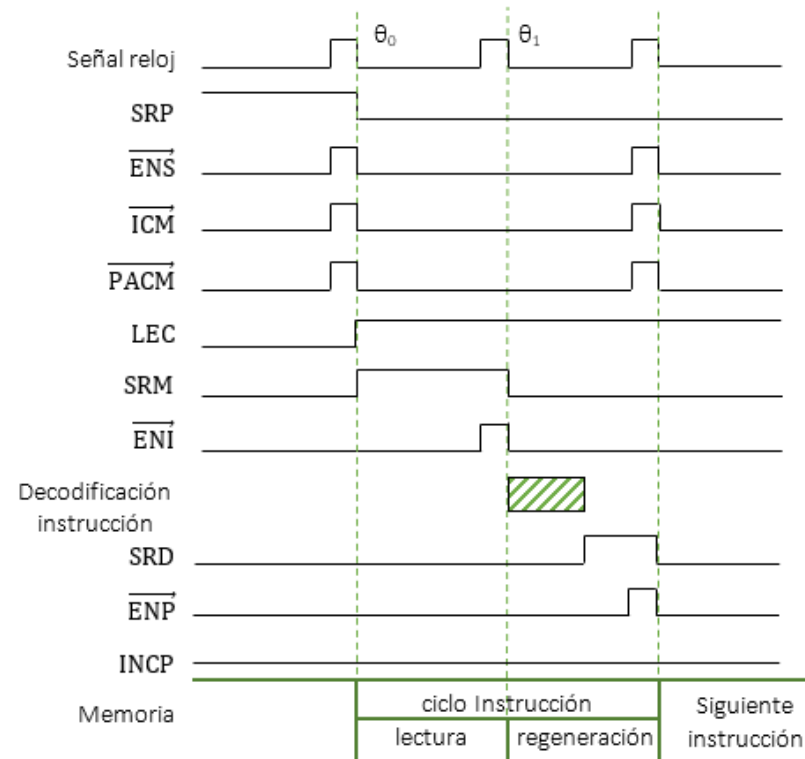
Consiste en que mueve el contenido de D a S y sobrescribe el valor de P para que no quede desactualizado. La información de la parte de la Dirección del registro de instrucción, sobrescribe el valor que lleva el registro CP, de manera tal que la siguiente instrucción sea la indicada por la instrucción en curso.

La información de la parte de dirección de la instrucción debe ser transferida simultáneamente a S para buscar la próxima instrucción y a P para actualizar el contador: (D) $\rightarrow$ S, (D) $\rightarrow$ P.

Obsérvese que esta instrucción no dura más que un ciclo de memoria y que no hay que incrementar en 1 el contador como ocurre en las otras instrucciones.

Las primeras tres es la búsqueda de la instrucción y la parte que resta está dedicada a alterar la secuencia del Programa basando el contenido de P en el valor del contenido de D $\Rightarrow$ El contenido de D va a P y a S. No se incrementa P porque ya tiene el valor de SAI.

(P) $\rightarrow$ S	SRP, $\overrightarrow{\text{ENS}}$
((S)) $\rightarrow$ M	$\overrightarrow{\text{ICM}}$, $\overrightarrow{\text{PACM}}$, LEC
(M) $\rightarrow$ I	SRM, $\overrightarrow{\text{ENI}}$
(D) $\rightarrow$ P	SRD, $\overrightarrow{\text{ENS}}$
(D) $\rightarrow$ S	SRD, $\overrightarrow{\text{ENP}}$, $\overrightarrow{\text{ENS}}$



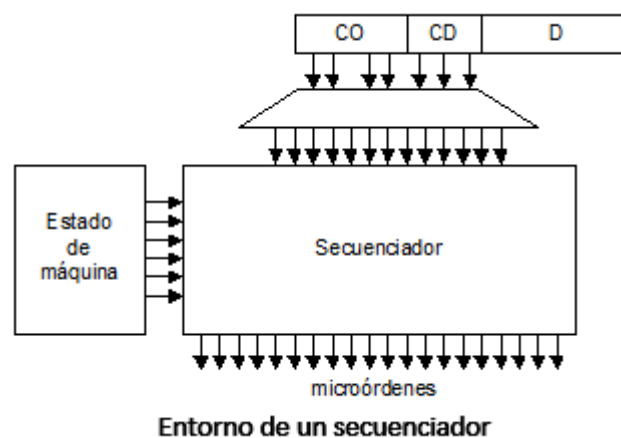
Secuenciador

Este se encarga de **generar las micro-órdenes necesarias para ejecutar una instrucción de lenguaje de máquina**, las cuales a través de la ruta de datos genera las habilitaciones distribuidas de manera racional (para no consumir tanto ancho de banda en los buses) y eficiente (para no exceder en micro-órdenes para una misma acción) a lo largo de la ruta de datos de dicha Arquitectura (que son distribuidas a lo largo de la ruta de datos para activarla y gobernar sus diversos elementos constituyentes). **¡Es como el cerebro del computador! El corazón sería aquellos que sincronizan al computador.**

El secuenciador se encuentra entre la Microprogramación y la Lógica Digital, ya que Abacus no tiene la capacidad de Microprogramación, por ello decimos que es Secuenciador en lugar de eso.

Toma como información la instrucción y genera las señales para que se resuelva ordenadamente la instrucción mandada. Lee la instrucción y decodifica la instrucción para generar el conjunto de micro-órdenes (señal de gobierno impulsional o de nivel que viaja en la ruta de datos) que deben distribuirse por la ruta de datos según cronogramas precisos a causa de los tiempos de respuesta de los órganos gobernados.

Las informaciones de entrada al secuenciador son suministradas, de una parte, por la instrucción (código de operación, condiciones de direccionamiento y las direcciones de registros), de otra parte, por el estado de la máquina, que agrupa un cierto número de informaciones (indicadores de error, demandas de interrupción, indicadores del estado de la memoria y de las unidades funcionales, etc.)



Secuenciador Cableado y MicroProgramado

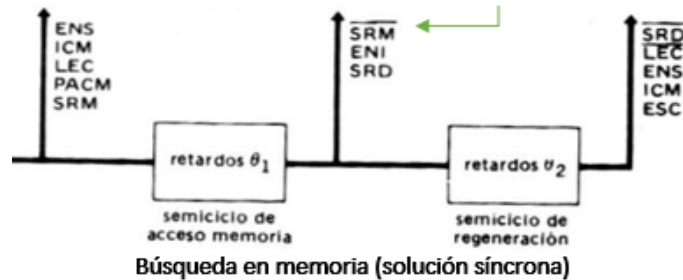
Micro-programado → posee una Memoria de control, son memorias de nivel 0 y se puede re-programar con el fin de mejorarla o corregirla; entonces permite que el programador se aleje del hardware. Corresponden a la introducción en la UC de una memoria con un pequeño programa llamado microprograma para cada instrucción.

Cableado → es mucho más rápido que el micro-programado porque no debe realizar ninguna interpretación, pero es muy caro y es mucho más grande. Realizados en forma de circuito secuencial electrónico. El secuenciamiento de las micro-órdenes en el tiempo puede llevarse a cabo de dos maneras distintas:

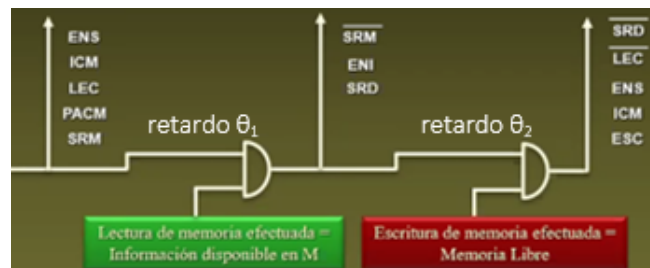
- **Ordenador Síncrono:** el secuenciador conoce los tiempos de respuesta de los diferentes órganos gobernados. Entonces hace intervenir los retardos apropiados entre las

diferentes órdenes sucesivos de manera que tengan tiempo de ejecutarse las operaciones. Genera las m-órdenes distanciadas en el tiempo. Son retardos fijos.

SRM negada significa
que se anula la señal
SRM del ciclo anterior

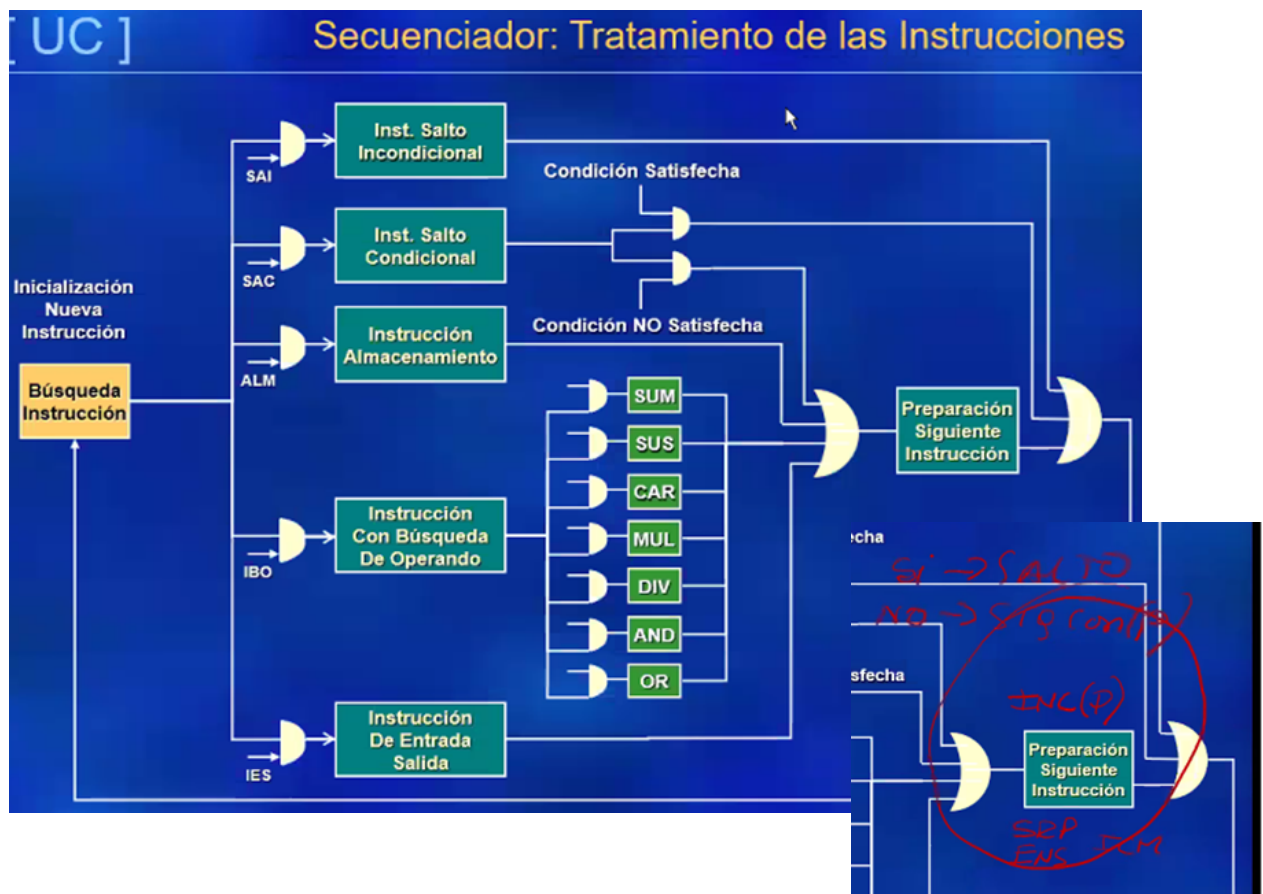


- **Ordenador Asíncrono:** el secuenciador recibe de los diferentes órganos del computador señales indicando que han terminado las operaciones indicadas y que se liberan, aquel puede eventualmente comprobar su estado. No lanzará una nueva operación más que después de haber quedado advertido de que las operaciones precedentes han sido ejecutadas completamente y de haberse asegurado que el órgano que debe realizarla está libre.

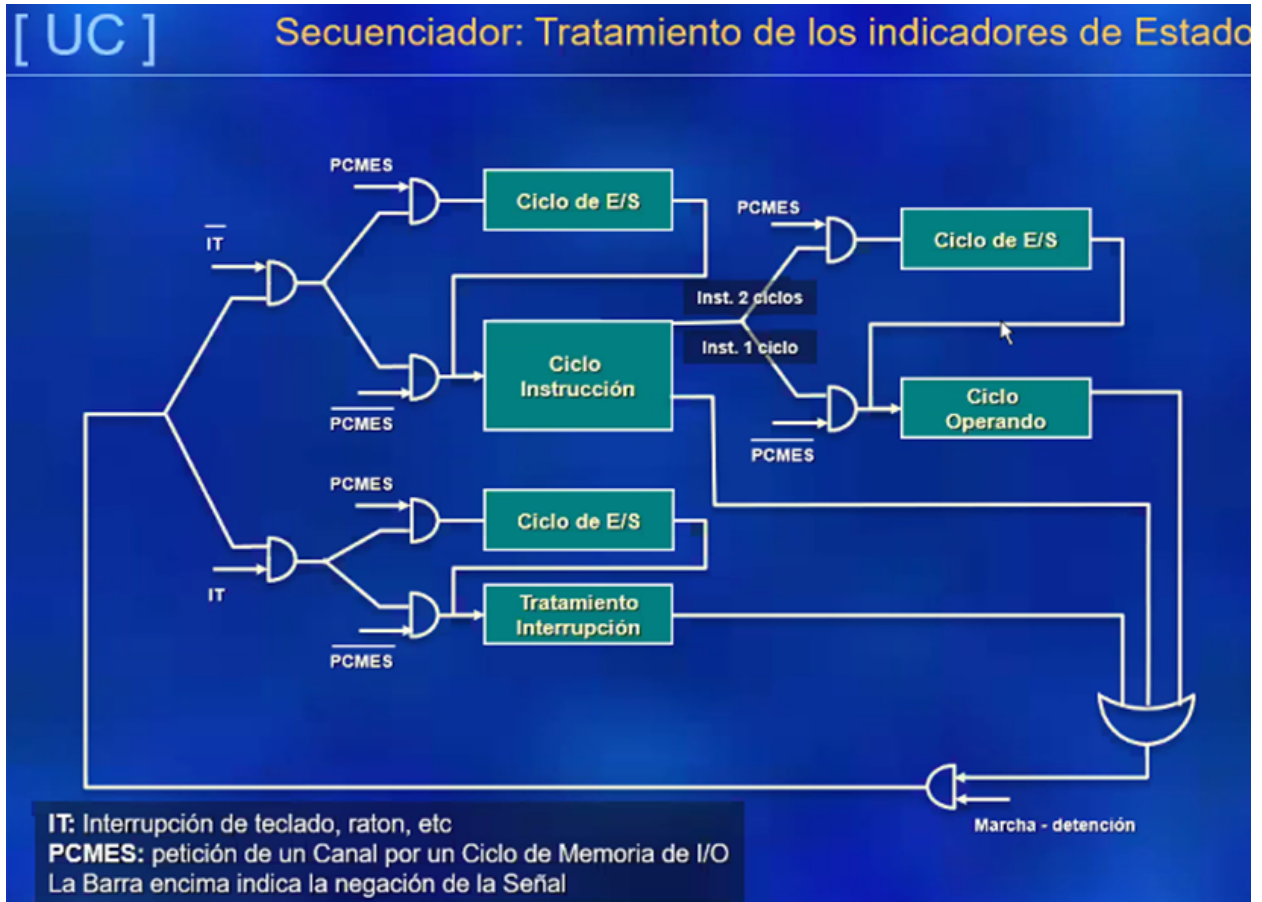


Las compuertas AND de la figura representan simbólicamente que solo se pasan a las siguientes micro-operaciones siempre que el conjunto anterior haya finalizado.

Tratamiento de las Instrucciones

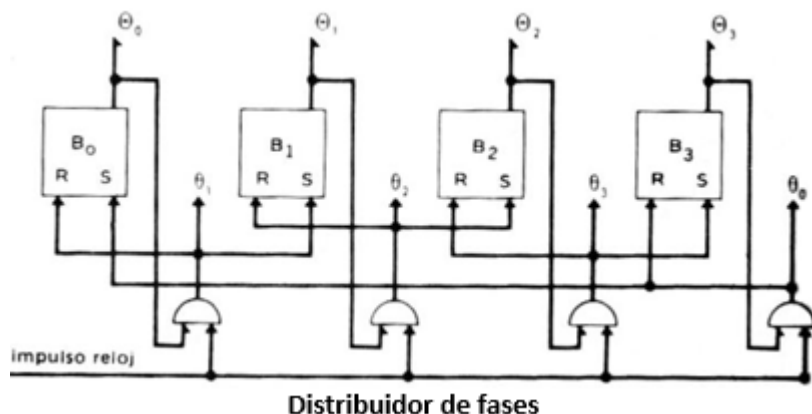


Tratamiento de los Indicadores de Estado



Distribuidor de Fases

En la mayoría de los casos se sincronizan las distintas micro-órdenes con señales temporales regularmente espaciadas, provenientes de un mismo reloj. A cada impulso de reloj debe ser generado un cierto número de micro-órdenes $\Rightarrow$ produce señales de impulso o de nivel acordes a las diferentes fases del ciclo de memoria o de la instrucción.

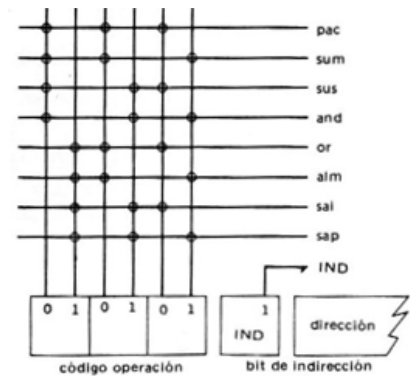


Decodificador del CO

La instrucción consta de diferentes campos de información de interés para el secuenciador, como son el código de operación, las condiciones de direccionamiento, la dirección de registros o de unidades periféricas, etc. El decodificador de instrucción se encarga de realizar esta decodificación.

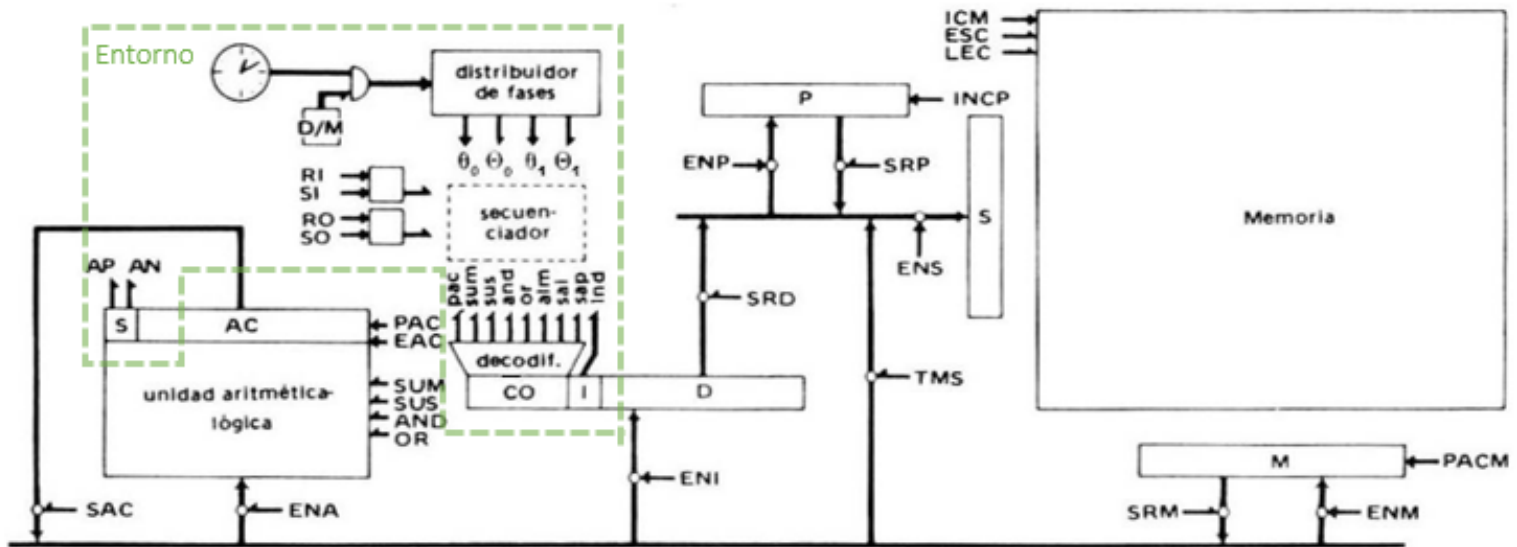
Puede utilizarse una matriz de diodos o de transistores.

Nombre Nemotécnico	Código Operación	Efecto de la instrucción
PAC	000	Puesta a cero del acumulador
SUM	001	Suma con el acumulador
SUS	010	Sustracción del acumulador
AND	011	Intersección con el acumulador
OR	100	Reunión con el acumulador
ALM	101	Almacenamiento del acumulador
SAI	110	Salto incondicional
SAP	111	Salto si (AC) >= 0



Decodificador del código de operación de Abacus

Esquema General de Abacus

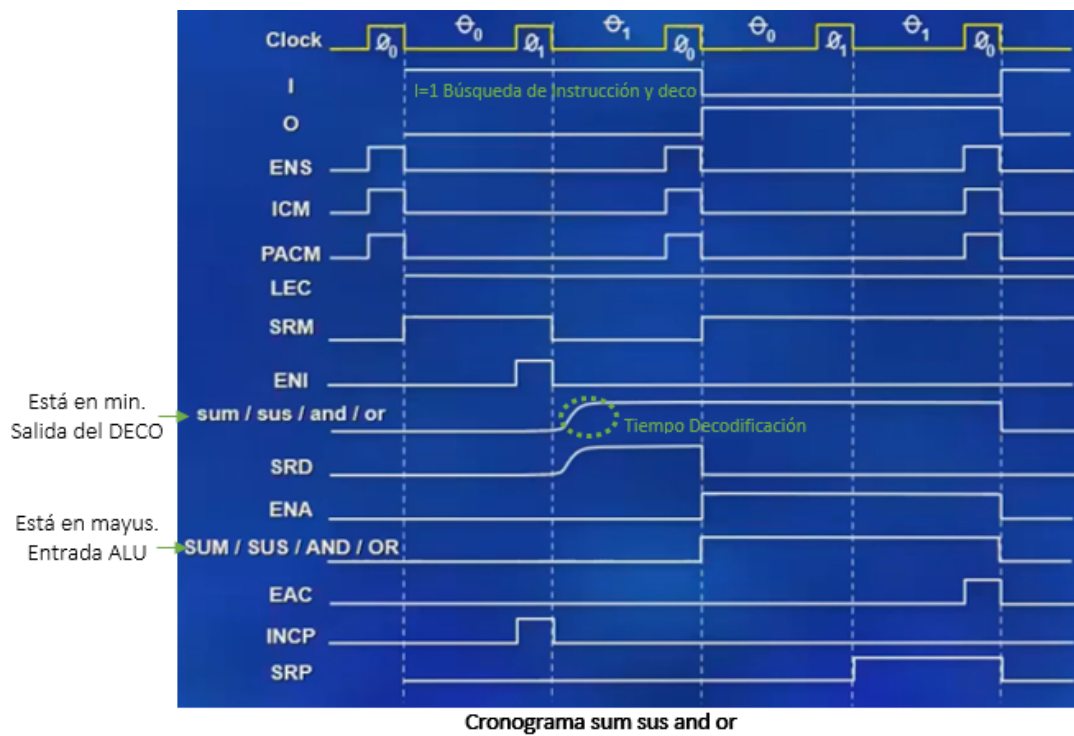


Trazado de Cronogramas

A fin de lograr la realización de los cronogramas se deberá tener en cuenta los tiempos de respuesta de los diferentes operadores implicados en la ejecución de cada microoperación y de él se deducen las micro-órdenes a generar a cada batido de reloj. Para poder explicar la forma de realizar esta tarea tomaremos el ejemplo de Abacus, para el cual definimos lo siguiente:

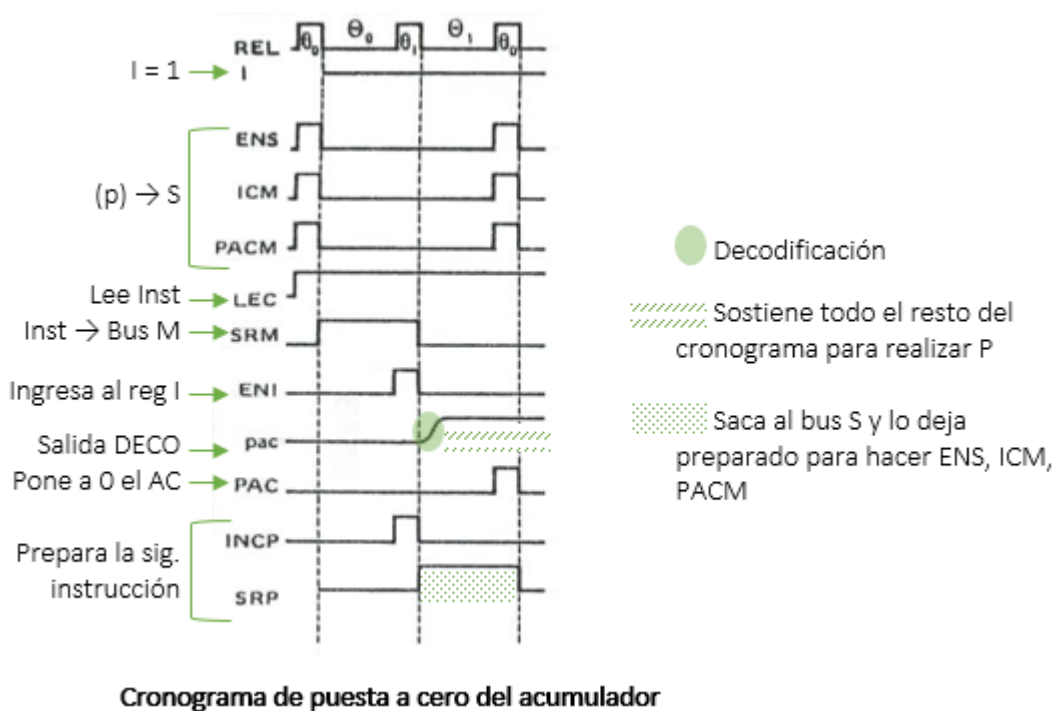
Abacus está provisto:

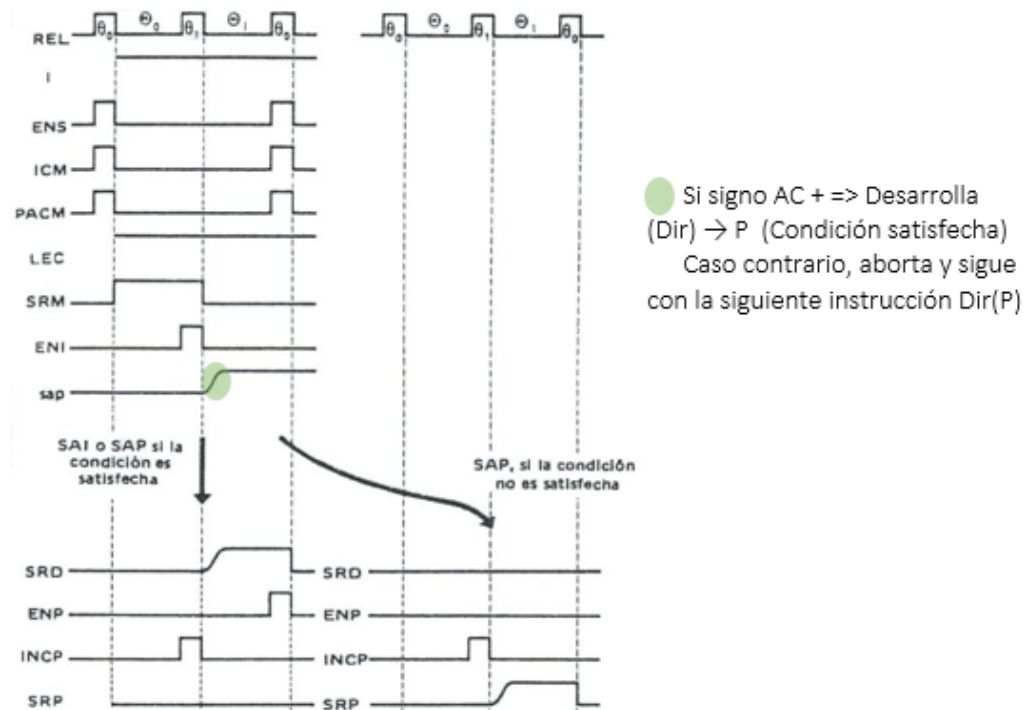
1. De un distribuidor de fases de dos fases
2. De un decodificador de instrucciones descrito anteriormente.
3. De los biestables de estado definidos anteriormente. Se supondrá que el funcionamiento del reloj está condicionado por la señal de detención-marcha, y que la puesta en marcha inicializa el distribuidor de fases.



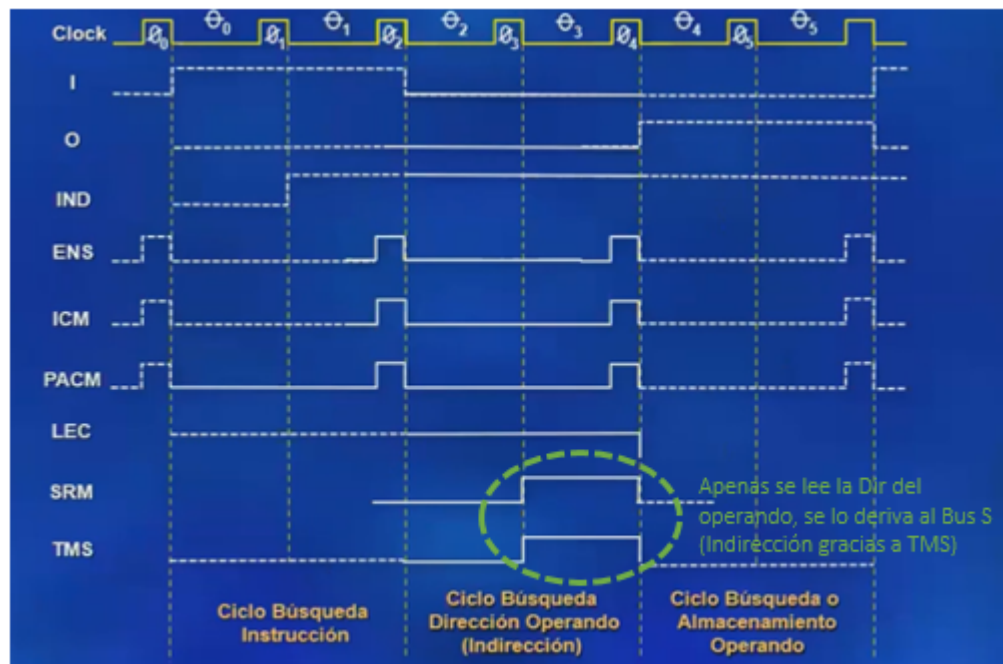
Cuando recién arranca el circuito el cronograma pone en 1 a I, y en 0 a O para que el secuenciador no se pierda y entienda que corresponde a la búsqueda de instrucción y decodificación.

Luego en la parte de sum/sus/and/or, eso va a ser la salida del decodificador, está en minúscula porque es una señal de habilitación del decodificador. Después en mayúsculas están las entradas de la ALU (operaciones IBO).





Cronograma salto incondicional y condicional si signo AC es +



Cronograma Instrucción con ciclo de búsqueda del operando (dir indirecto)

Ecuaciones Lógicas

Una vez realizados todos los cronogramas, el paso siguiente consiste en, micro-orden por micro-orden, buscar todas las veces que debe ser activada y de ellas deducir la expresión lógica en relación con las informaciones de entrada y de estado del secuenciador. El conjunto de estas expresiones constituye las ecuaciones lógicas del ordenador.

$$\text{ENS} = \theta_0 \quad \text{ICM} = \theta_0 \quad \text{PACM} = \theta_0$$

Deben activarse siempre en θ_0 , θ_2 y θ_4 , pero debido a que Abacus trabaja con un Distribuidor de Fase de dos Fases, se deduce que deben estar activos en el instante θ_0 de todos los cronogramas, tanto para la fase de instrucción como para la del operando.

$$ENI = \theta_1 \cdot I \quad INCP = \theta_1 \cdot I$$

$$\text{Si } I = \text{verdadero} \Rightarrow \theta_1 \cdot I \equiv \theta_3$$

Son señales de tipo impulsional que solo deben generarse en el ciclo de Instrucción, para la etapa θ_1

$$SUM = \text{sum} \cdot O$$

$$SUS = \text{sus} \cdot O$$

$$AND = \text{and} \cdot O$$

$$OR = \text{or} \cdot O$$

Son señales de Nivel que deben sostenerse a lo largo de $\theta_0 + \theta_1$ y para cada operación en particular, dentro del ciclo del operando. La expresión $\theta_0 + \theta_1$ se elimina ya que la operación dura las dos fases completas

$$LEC = I + O \cdot + IND \cdot \theta_1 \cdot ()$$

La señal de lectura es de nivel y debe durar todo el espacio de tiempo entre ciclo de Instrucción y de Operando (menos para almacenamiento) o bien fuera de los ciclos de Operando e Instrucción.

$$SRM = \theta_0 \cdot I + O \cdot + IND \cdot \theta_1 \cdot ()$$

La salida del Registro M de Nivel debe ser activada:

1. En la búsqueda de instrucción para θ_0 , es decir ($\theta_0 \cdot I$)
2. En el ciclo de operando para sum, sus, or, and y durante el tiempo que le lleve a la ALU realizarlo $O \cdot (\theta_0 + \theta_1) \cdot (\text{sum} + \text{sus} + \text{or} + \text{and})$. Debido a que $(\theta_0 + \theta_1)$ cubre todo el espacio de tiempo que dura O, es inútil expresarlo. Además a que sum, sus, or y and requieren de un ciclo completo de Operando (con excepción de alm), podemos expresarlo de manera más simple como $O \cdot$
3. En el caso del Direccionamiento Indirecto, en el ciclo de búsqueda de la dirección y durante todo el ciclo θ_1 .
4. Para Direccionamiento Indirecto, es necesario contemplar la habilitación de la señal IND en θ_1 dentro del "Ciclo de Almacenamiento del Operando" (fuera del ciclo de instrucción y el del operando)

Son señales del tipo impulsional que solo deben generarse en el ciclo de la Instrucción, para la etapa θ_1 .

$$SI = \theta_0 \cdot O$$

El propio secuenciador deberá tener la tarea de posicionar a los Biestables I y O, por tal motivo I debe ser puesto a Uno al final del ciclo del operando.

$$RI = \theta_0 \cdot I \cdot (\text{sum} + \text{sus} + \text{or} + \text{and} + \text{alm})$$

I debe ser Puesto a Cero al final del ciclo de aquellas instrucciones que requieren ciclo de operando, es decir, para ciclo de instrucción dentro del espacio θ_0 , para la suma, resta, or, and y alm, el biestable I deberá ser reseteado ya que se encuentra en la fase de resolución del Operando.

$$SO = \cdot \theta_0 \cdot I (\text{sum} + \text{sus} + \text{or} + \text{and} + \text{alm}) + IND \cdot \theta_0 \cdot$$

O debe ser activado ya sea después de un ciclo de instrucción, cuando es con operando y sin direccionamiento indirecto, o bien al final del ciclo de búsqueda de la dirección efectiva, en el caso de direccionamiento Indirecto y por fuera de la instrucción y operando.

$$RO = \theta_0 \cdot O$$

O debe ser puesto a Cero al final del Ciclo de operando. A continuación se observan todas las ecuaciones para formar el secuenciador de Abacus:

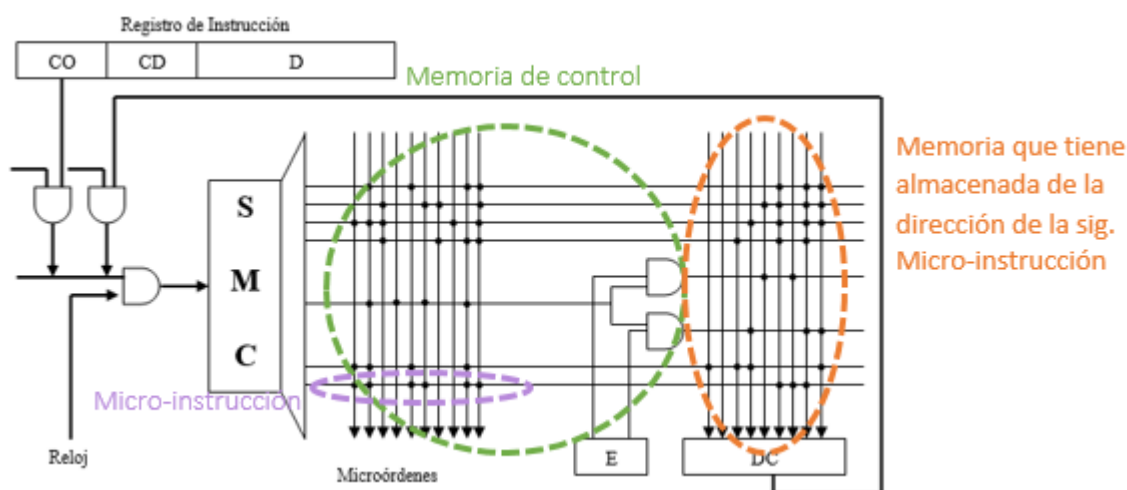
ENS	$= \theta_0$
ICM	$= \theta_0$
PACM	$= \theta_0$
LEC	$= I + O \cdot \overline{alm} + (\bar{I} \cdot \bar{O})$
ESC	$= O \cdot alm$
SRM	$= I \cdot \theta_0 + O \cdot \overline{alm} + IND \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O})$
TMS	$= IND \cdot \bar{I} \cdot \bar{O} \cdot \theta_1$
ENI	$= \theta_1 \cdot I$
SRD	$= \theta_1 \cdot I \cdot (sum + sus + and + or + alm + sap \cdot AP + sai)$
SRP	$= \theta_1 \cdot I \cdot (pac + sap \cdot AN) + \theta_1 \cdot O$
INCP	$= \theta_1 \cdot I$
ENP	$= \theta_0 \cdot (sap \cdot AP + sai)$
SUM	$= sum \cdot O$
SUS	$= sus \cdot O$
AND	$= and \cdot O$
OR	$= or \cdot O$
ENA	$= O \cdot \overline{alm}$
EAC	$= O \cdot \overline{alm} \cdot \theta_0$
PAC	$= pac \cdot \theta_0$
SAC	$= O \cdot \theta_0 \cdot alm$
ENM	$= O \cdot \theta_1 \cdot alm$
SI	$= SI = \theta_0 \cdot O$
RI	$= RI = \theta_0 \cdot I \cdot (sum + sus + or + and + alm)$
SO	$= SO = \overline{IND} \cdot \theta_0 \cdot I \cdot (sum + sus + or + and + alm) + IND \cdot \theta_0 \cdot \bar{I} \cdot \bar{O}$
RO	$= RO = \theta_0 \cdot O$

Es de destacar que el ordenador Abacus es una máquina simple con un conjunto de instrucciones reducido para facilitar su estudio.

Modelo de WILKES

Wilkes presenta este modelo innovador donde trata de introducir una mecánica de micro-órdenes de forma micro-programadas. Es un modelo muy flexible el cual nos permite prescindir de ecuaciones lógicas complejas y pasar a una memoria de control.

Dada una instrucción en lenguaje de máquina, compuesta como mínimo de cod de operación, el cod de operación está relacionado con la dirección de la primera micro-instrucción (microprograma). Un micro programa es un conjunto de microinstrucciones.



VENTAJAS:

- Reducimos la cantidad de componentes dentro de la unidad de control.
- Flexibilidad, menos tiempo de desarrollo, menor COSTO.
- Flexibilidad en el sentido de que si quieres cambiar algo puedes regrabar ponele, en un cableado tenes que tirar todo nomá(?)

DESVENTAJAS:

- ¿Qué pasa con la longitud de la memoria? Es complicado porque depende de la ruta de datos. Si tenes muchas señales de gobierno, vas a necesitar más bits y eso se va al pingo
- También hay un problema de velocidad por el 'pensamiento', el ritmo con el cual genera las instrucciones por el acceso a memoria.

Relación de la memoria de Wilkes al tamaño de la Arquitectura.

Es ineficiente porque no prevé ninguna codificación que pueda achicar un toque el tamaño, y encima lo pone más complejo por todos los bits que necesita para convertirse en la dirección de la siguiente microInstrucción. La longitud de palabras en Wilkes está relacionada con la cantidad de compuertas de gobierno en la ruta de datos ⇒ En Abacus tenes pocas, tipo 20 ó 25 bits que es manejable, pero arquitecturas de hoy tenes centenares.

La tabla siguiente muestra otras características de una unidad de control micro-programada que también se encuadran dentro de espectros de posibilidades, y que en general depende del espectro del grado de codificación.

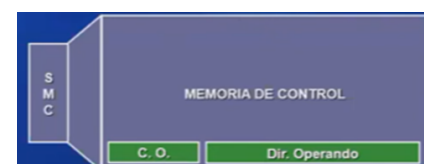
Características	
Microinstrucción no codificada	Microinstrucción muy codificada
Muchos bits	Pocos bits
Visión detallada del hardware	Visión global de hardware
Difícil de programar	Fácil de programar
Concurrencia explotada completamente	Concurrencia no explotada completamente
Poca o ninguna lógica de control	Lógica de control compleja
Ejecución rápida	Ejecución lenta
Optimización de las prestaciones	Optimización de la programación

Terminología	
Desempaquetada	Empaquetada
Horizontal	Vertical
“Hard”	“Soft”

Codificación de las Microinstrucciones

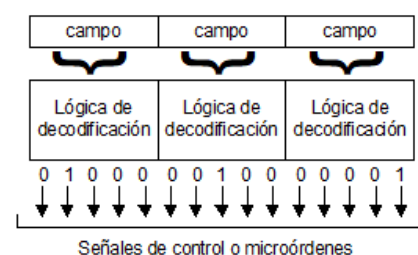
Existen dos métodos extremos para pasar de la configuración binaria de la micro-instrucción a las diversas micro-órdenes correspondientes. El **primer método** consiste en asociar un bit de la microinstrucción a cada micro-orden (mayor longitud de la micro-instrucción y difícil programación, permite ejecutar varias micro-órdenes simultáneas). En el otro extremo, el **segundo método** consiste en codificar el conjunto de las posibles micro-órdenes con el número mínimo de bits (menor longitud de microinstrucción y mayor decodificación, se necesitaría una micro-instrucción por cada micro-orden).

1. Codificación tipo instrucción. Este tipo de codificación da a la micro-instrucción una estructura semejante a la de una instrucción, con código de operación y dirección de operando (en registro o en memoria local).



La decodificación de las micro-instrucciones para obtener las correspondientes micro-órdenes es relativamente compleja, siendo compensado por la pequeña longitud de la micro-instrucción. Pone la Micro-Programación al alcance del programador de lenguaje de máquina.

2. Codificación por campos. Se dividen las distintas micro-órdenes en grupos independientes de modo tal que sería imposible que, dentro de cada grupo, se den micro-órdenes simultáneas. Cada grupo, además, corresponde a un determinado tipo de función: apertura de puertas de diversos registros a un mismo bus, gobierno de una unidad funcional, gobierno de la memoria local, etc.



Direccionamiento de las Micro-Instrucciones

Secuencial

Este modelo es básicamente similar al de Wilkes, ya que en condiciones normales la dirección de la próxima microinstrucción la calcula el SMC con capacidad de incrementar la última dirección de la micro-instrucción y de manera secuencial. El registro SMC tiene capacidad de cuenta, de manera tal que la primera dirección de la microinstrucción la da el código de operación, pero luego se incrementa SMC para apuntar a la siguiente en posición física.

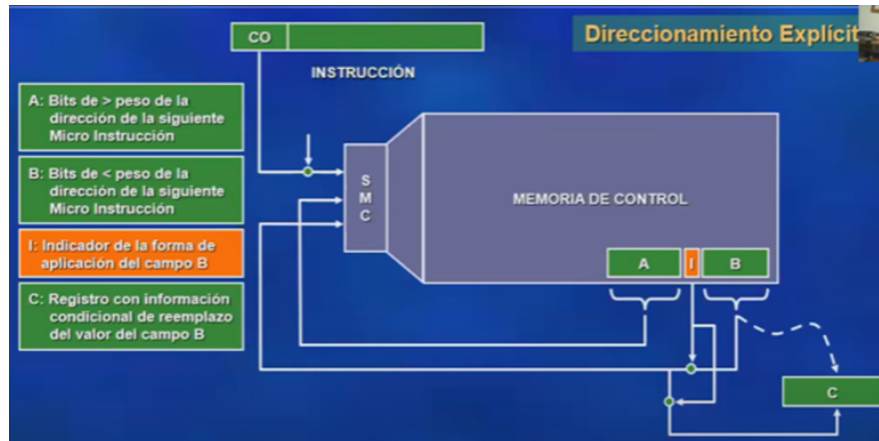


Ventaja: simple de programar y no alarga (no encarece) el tamaño de la microinstrucción.

Desventaja: un salto ocupa una micro-instrucción, por sí solo dependiendo el tiempo. Se lo usa cuando la concepción de la micro-programación de la máquina no requiera muchos saltos

Explícito

Esquema preconizado por Wilkes para evitar cablear un salto condicional. Hay una parte de codificación por campo y la dirección de la siguiente instrucción. Tiene un nivel importante de codificación por campo para reducir el tamaño.



En condiciones normales, la dirección de la siguiente microinstrucción surge de yuxtaponer las direcciones de los campos A y B. En un salto condicional:

- Si $I=0$, el campo B podrá contener los bits de menor peso de la dirección de salto si este fuese conocida
- Si $I=1$, B contiene la dirección de un registro C que almacena los bits de menor peso de la dirección de salto

CISC (Complex instruction set computer)

La denominación CISC se debe a que los procesadores que utilizan esta tecnología pueden ejecutar desde instrucciones simples, hasta instrucciones muy complejas. **Las instrucciones simples**, luego de su decodificación, pueden ejecutarse en un pulso de reloj, mientras que **las complejas** requieren un número de pulsos que dependen de la secuencia de pasos necesarios para su ejecución.

Cada paso se lleva a cabo mediante una **combinación binaria (micro-código)**, que aparece en las líneas de control de la UC con cada pulso de reloj, la cual activa los circuitos que intervienen en ese paso. Las sucesivas combinaciones (micro-códigos) que requiere la ejecución de una instrucción compleja, deben ser **provistas por una memoria de control que las almacena y que forma parte de la UC**.

Por lo tanto, un procesador CISC necesariamente debe contener una memoria de control con los micro-códigos, para poder ejecutar las instrucciones complejas.

La existencia de muchos “modos de direccionamiento” para realizar la operación que ordena una instrucción, es otra de las características de complejidad de los procesadores CISC $\Rightarrow$ presenta un número elevado de códigos de instrucción. Un set de instrucciones CISC posee alrededor de 210 instrucciones.

En la concepción CISC se busca lograr el “salto semántico”, que es obtener la menor disparidad entre los lenguajes de alto nivel y el lenguaje de máquina. De existir muchas equivalencias entre sentencias en alto nivel e instrucciones de máquina, el programa traductor entre

ambos niveles (compilador) sería más sencillo de fabricar, y los tiempos de compilación disminuirían, que lógicamente son los objetivos de la arquitectura CISC.

Consume muchos ciclos de instrucción y es difícil de modificar, porque no es flexible, en donde los tiempos de ejecución son mayores (es más lento)

ENTONCES

- Son muchas instrucciones y complejas (cada instrucción permite realizar más de una operación, como ser la división, multiplicación, lo cual penaliza al consumo de energía eléctrica y disipación de calor y vida útil).

- Todo sistema orientado a ventana tiene un costo, por el manejo de las mismas. Es un programa que corre de background, donde el procesador trata de dar soporte a todo el manejo gráfico. Pensar en un risk era imposible, es por ello que era necesario un cisc, por su complejidad.

- Muchas de las instrucciones disminuyen la brecha semántica entre los lenguajes de alto nivel y la máquina → Cisc considera que entre más compleja una instrucción le damos un valor agregado a los lenguajes de alto nivel, es por ello que incorpora muchos modos de direccionamiento y tan grande como sea posible.

- Lograr altas velocidades de ejecución no es fácil ⇒ cisc aumenta la superficie de silicio, aumentando las instrucciones.

- Posee muchos registros de propósito específico. (Una risc se puede manejar con 4 registros mas, pero a la hora de programar es horrible (sería 10 veces más largo que en una cisc)).

- **Control microprogramado** → utiliza un secuenciador microprogramado porque tiene una banda de instrucciones (por ende señales) que tiene que manejar, sin esto el precio se dispara por las nubes.

RISC (Reduced Instruction set computer)

Buscando optimizar la performance de los procesadores, se realizaron estadísticas de las instrucciones de máquina más usadas. Resultó que las instrucciones más simples (que son solo el 20% del repertorio de instrucciones de un procesador CISC) constituían el 80% de los programas típicos ejecutados.

Planificar procesadores con un set de instrucciones simples que presentan **muy pocos modos de direccionamiento**. Por ser simples, estas instrucciones se ejecutan en un solo pulso de reloj, luego de haber sido decodificadas.

El número total del set de instrucciones de un procesador RISC es reducido (entre 70 y 150 instrucciones, según el modelo). Todas las instrucciones son de formato fijo, siendo que en un CISC ocupan distinta cantidad de bytes. **Esto nos brinda mayor sencillez y velocidad de procesamiento** ⇒ Cada vez que hay que agregar una instrucción a rics, hay que pensar mil veces antes de hacerlo.

A fin de traducir un lenguaje de alto nivel a este tipo de instrucciones, empleando un mínimo de ellas, **RISC requiere de un programa compilador inteligente, muy elaborado**. O sea que es necesario un compilador (software) complejo, como contrapartida de un hardware más simple.

Los bits del código de operación de cada instrucción que llega a RI para ser ejecutados por un procesador RISC, sirven de base para que un circuito de la UC los convierta directamente en la combinación de bits que deben aparecer en las salidas de la UC (microcódigo) para que en próximo pulso de reloj se ejecute la instrucción en cuestión.

Por constar de instrucciones cuya fase de ejecución requiere mayormente de un pulso, o a lo sumo dos, **no se requiere de una memoria de control que sirva de base para la ejecución del micro-código** ⇒ por un lado permite ganar mayor velocidad de procesamiento, pues se evita el acceso a la memoria de control, y por otro lado beneficia mucho el diseño del chip que contiene un procesador RISC.

Otra característica de los procesadores RISC es **la incorporación de un cache interno para datos y otro para instrucciones, que reducen los accesos a memoria principal.**

ENTONCES

- Las instrucciones no monopolizan los tiempos de la cpu. Mientras se está ejecutando algo se puede estar buscando la siguiente instrucción. Pero para lograr esto debe desechar el concepto de la micro-programación para lograr altas velocidades.
- Requiere muchas instrucciones de bajo nivel, en comparación a las de alto nivel.
- Su repertorio es usado por los **compiladores optimizadores** (analiza el código y ve la posibilidad de reducirlo) porque puede fraccionar, factorizar y re-factorizar este pobre set de instrucción y tiene la posibilidad de mezclar de forma conveniente para que la compilación sea más óptima.
- Risk necesita tenerla cerca a la ram, al tenerla lejos hay cierta exigencia que la puede hacer más lenta ⇒ **la unidad cableada es necesaria.**
- Si vas a diseñar una risk tenes que estudiar todo el universo de las operaciones para ajustar el set de instrucción que va a manejar → es penalizado el sobrante de instrucciones.

CISC vs RISC

Característica	CISC	RISC
Memoria de control	Sí	No
Set de instrucciones	Grande. Tiene inst. complejas (multi reloj)	Reducido Inst. simples (reloj simple)
Ciclos de reloj por instrucción	No existe un número definido	Uno o dos
Accesos a MP mediante microinstrucciones	Sí	No, utiliza registros auxiliares. (excepto LOAD – STORE)
PIPELINE	Ineficiente	Eficiente
Compiladores	Simple	Complejos
Permite modularidad y el crecimiento del set de instrucciones	Sí	No
Formato de Instrucciones	Variable	Fijo
Implementación de paralelismo en la ejecución de las instrucciones	Compleja	Simple
Velocidad de procesamiento	Reducida por los accesos a memoria de control	Optimizada por el paralelismo de ejecución
Predicción de bifurcaciones	No	Sí
Énfasis en ...	HW	SW
Costo de producción	Alto	Bajo

Canales

Es un órgano encargado de conectar la UCP con dispositivos externos. Evitando impactar en la performance del procesador en lo que se refiere a la ejecución del programa en curso, de modo que se mantenga una comunicación sostenida con el periférico y que este pueda ser utilizado por el programa. El canal debe gestionar la transmisión de información entre la memoria RAM y los periféricos.

Las controladoras tienen una doble finalidad: entender el hardware del periférico y dialogar con el canal.

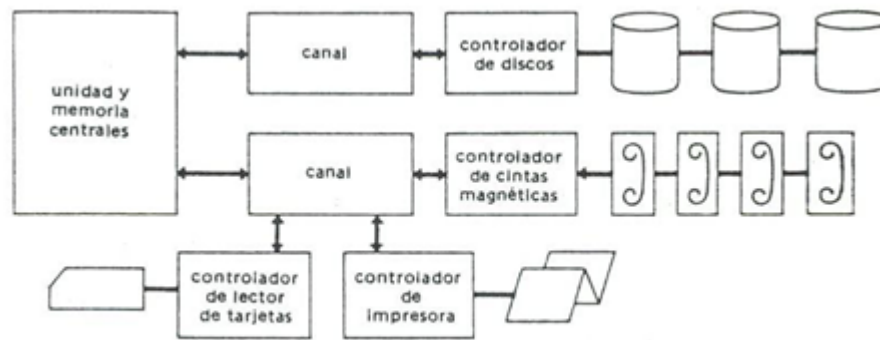


Fig. 1. Configuración ilustrativa de la función de los canales y de los controladores

Modos de operación

Toda comunicación con el exterior consume ciclos de CPU y accesos a memoria. Es importante que el canal logre una simultaneidad entre los programas en curso y las operaciones de e/s.

Modo bloqueado

Encadena la CPU al ritmo del periférico → Básicamente esperar que el periférico termine de hacer lo suyo bloqueando a la CPU. Se hace un problema en tanto aumenta la velocidad de la máquina, es decir, a medida se aumenta esta diferencia de tiempo de respuesta (asimetría entre ambos).

Modo por Prueba de Estado

Incluye en la arq una instrucción PRE (prueba de estado). Se pregunta si el periférico está libre: se realiza la rutina de "impresión", sino se continua con la rutina. Le da la oportunidad al programador evitar poner el programa en estado de bloqueo.

El problema es que queda todo a cargo del programador y weno, alto laburito. Aparte, a medida que los periféricos se hacen más rápidos cada vez tenés que andar preguntando más y se te arma una ensalada de código.

Modo por Interrupción de Programa

Son los periféricos los que interrumpen para avisar al computador que se encuentran disponible ⇒ esta interrupción detiene momentáneamente el programa en curso para permitir que el programa de transferencia sea ejecutado con prioridad.

Modo Automático por Suspensión de Programa o Modo Canal

El canal es ahora una unidad que hace una transferencia de un BLOQUE DE INFORMACIÓN. Se tiene una instrucción que le da toda la info a la CPU para que el canal pueda gestionar todo un bloque de memoria

Si el canal necesita realizar una transferencia, ya no genera una irrupción → hace una solicitud de servicio para que se le conceda un ciclo de memoria ⇒ La instrucción provee al canal la siguiente info:

- Número y emplazamiento de info a transferir.
- Dirección del periférico.
- Condición de finalización de la transferencia.

Básicamente se inicializa el canal con la info, entonces el man empieza a trabajar según la disponibilidad de la memoria central. La transferencia se va a realizar al ritmo del periférico y el canal va a robarle ciclos al procesador y a avisar cuando termina.

Encadenamiento Automático de las Transferencias

El canal puede encadenar la transferencia de los distintos bloques $\Rightarrow$ Se utiliza un conjunto de instrucciones de canal para encadenar distintos bloques: es decir un PROGRAMA CANAL.

Básicamente agarras el método anterior y armas paquetes de tantas instrucciones de canal como bloques de memoria no contiguos que tengas.

Interface

Se denomina Interface a las especificaciones de conexión entre un canal y cualquier tipo de unidad periférica. Es la caja negra que permite adaptar las señales del periférico a las especificaciones de la interface ordenador $\Rightarrow$ establece el protocolo de comunicación entre la controladora de la familia de periféricos y sus periféricos.

La interface, está formada físicamente por un cierto número de conectores, salidos del canal, en los cuales pueden ir a enchufarse. Es necesario precisar además las especificaciones de formato y de tiempo de las señales dadas o esperadas por el canal.

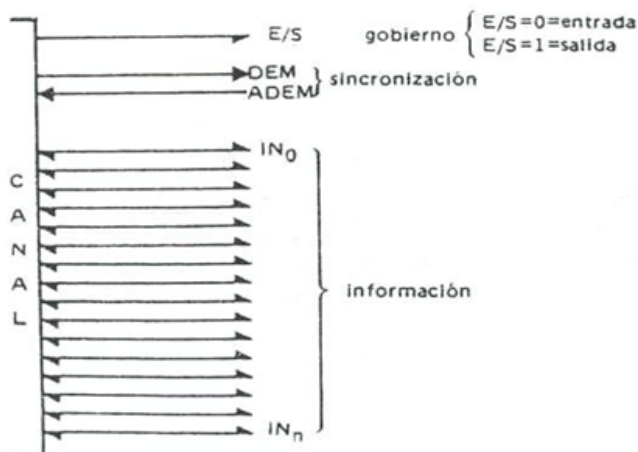


Fig. 3. Esquema de interface

DEM demanda de transferencia.

ADEM aceptación de demanda.

Se requieren técnicas para conectar un canal con más de un controlador

Básicamente, la interface se compone de dos tipos de hilos:

1. **Hilos de información**, que llevan en paralelo la información por transmitir, en forma de niveles;
2. **Hilos de gobierno y sincronización**, que capacitan para el diálogo de las dos unidades interconectadas.

Concentración por multiplexación

Se tiene un decodificador que sensibiliza un solo periférico del grupo de controladores y el multiplexor lo que le indica con quién debe establecer la comunicación con la memoria.

El multiplaje consiste en desviar la información transportada por un bus hacia otro escogido entre varios o en concentrar en un solo bus las informaciones procedentes de varios buses diferentes. Permite concentrar varias vías de entrada en una sola o distribuir una vía de salida en varias vías. Los sistemas de multiplaje permiten conectar varias unidades externas a un mismo canal.

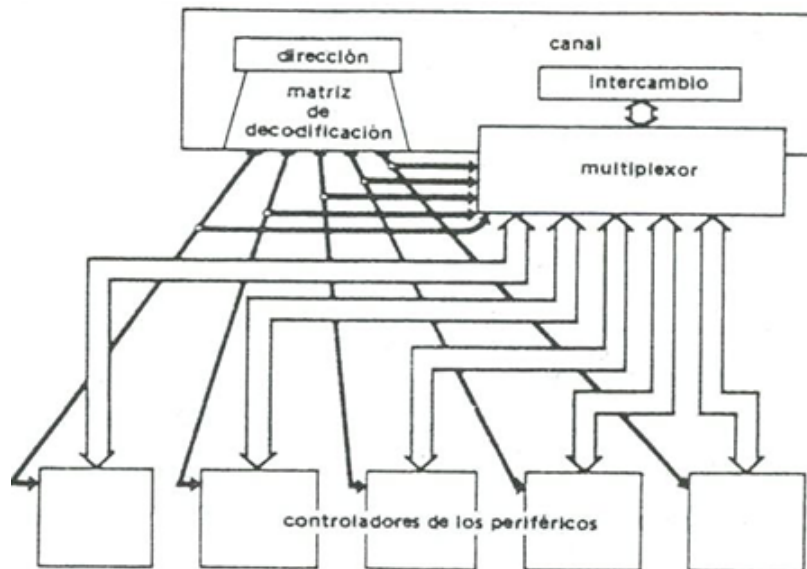


Fig. 5. Concentración por multiplexaje

Ventaja: Permite conectar el único canal y la única memoria con múltiples controladoras.

Desventaja: Muchas conexiones y buses, un multiplexor. Caro.

Concentración por línea ómnibus

Para conectar varias unidades externas en un canal, puede también emplearse una línea ómnibus que las una. Barata y más lenta. 1 Solo bus $\Rightarrow$ tarda más cada paquete de información viaja por ese bus, en el anterior podían ser paquetes simultáneos.

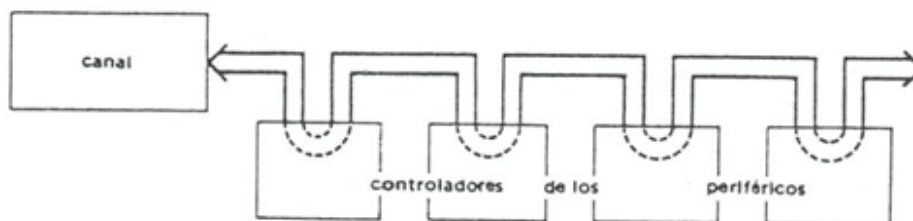


Fig. 6. Concentración por líneas ómnibus

El paquete de información va acompañada por una número de periférico y pasa por todas las controladoras conectadas hasta que se identifica con una, que indica que ese es el periférico a comunicar.

Técnicas de Ejecución de una Transferencia Elemental

Llamaremos **transferencia elemental** a la transferencia de una información (carácter o palabra) desde una posición de memoria central a una unidad externa (o viceversa, según se trate de una entrada o una salida). La ejecución de una transferencia elemental supone que se conozcan las informaciones siguientes:

- Información pidiendo el lanzamiento de la transferencia: DEM;
- Indicación del sentido de la transferencia (entrada o salida): SEN;
- Dirección de almacenamiento en memoria de la información transferida: DIR;

Transferencia Programada

Se ejecuta cuando el programa llega a una instrucción de transferencia. Es el ordenador quien inicializa la demanda de transferencia (DEM). Las informaciones complementarias son suministradas por la instrucción, que puede adoptar dos formas:

ENT DIR	Entrar información y almacenarla en la dirección DIR.
SAL DIR	Leer información almacenada en la dirección DIR y hacerla salir.

Transferencia por Instrucción Forzada

Es el periférico que genera una instrucción de E/S y fuerza su ejecución prioritaria en cuanto la unidad central ha concluido la instrucción en curso. El programa queda suspendido durante el tiempo de ejecución de esta instrucción.

Transferencia por Robo de Ciclo

Se inicia a petición de la unidad exterior. La unidad central termina el ciclo de memoria en curso y concede el próximo ciclo a la unidad exterior. El programa en curso queda en suspenso durante un ciclo de memoria.

Transferencia por Acceso Directo a Memoria

Se supone que la memoria posee dos vías de acceso (que podrían llamarse muelles de acceso). Una de las vías está reservada al ordenador, la otra a la unidad exterior.

Las peticiones de ciclo de memoria son dirigidas a la unidad de control de acceso a memoria, tanto si proceden de la unidad central como si de la unidad exterior. Esta unidad de control gestiona las prioridades de acceso de la unidad central y de la unidad externa.

TÉCNICA EMPLEADA	TIEMPO MÁXIMO DE ESPERA DE LA UNIDAD EXTERIOR	TIEMPO RESTADO AL PROGRAMA EN CURSO
Programada	Hasta el final de la instrucción en curso, más el inicio del programa solicitado de interrupción.	Procesamiento de un programa de interrupción.
Por instrucción forzada	Duración de la instrucción más larga del repertorio.	Una instrucción generalmente es ejecutada en uno o dos ciclos de memoria.
Por robo de ciclo	Un ciclo de memoria.	Un ciclo de memoria.
Por acceso directo	Un ciclo de memoria.	Nada, o un ciclo de memoria.

Tipos de canales

Enlace Programado ABACUS

No es propiamente un canal, sino un **medio de acceso controlado por el programa**. En efecto, cada transferencia elemental está gobernada por una instrucción del programa en curso. Por el hecho de estar controlados por el programa, tales enlaces **pueden utilizarse tanto en modo bloqueado como, mucho más a menudo, en modo por interrupción**. Casi siempre aparecen integrados en la unidad central.

Son necesarias 4 instrucciones de entrada – salida para realizar entradas – salidas en modo programado:

Instrucciones de gobierno de transferencias de información	SAL M	Para extraer el contenido de la dirección M hacia las líneas de salida de información de la interface.
	ENT M	Para almacenar en la dirección M una información presente en las líneas de entrada de información a la interface.
Instrucciones de control general de las operaciones de entrada – salida	GCP	Permite direccionar una unidad de control de periférico y especificar la operación por ejecutar.
	PRE	Permite adquirir una información descriptiva del estado del periférico anteriormente direccionado.

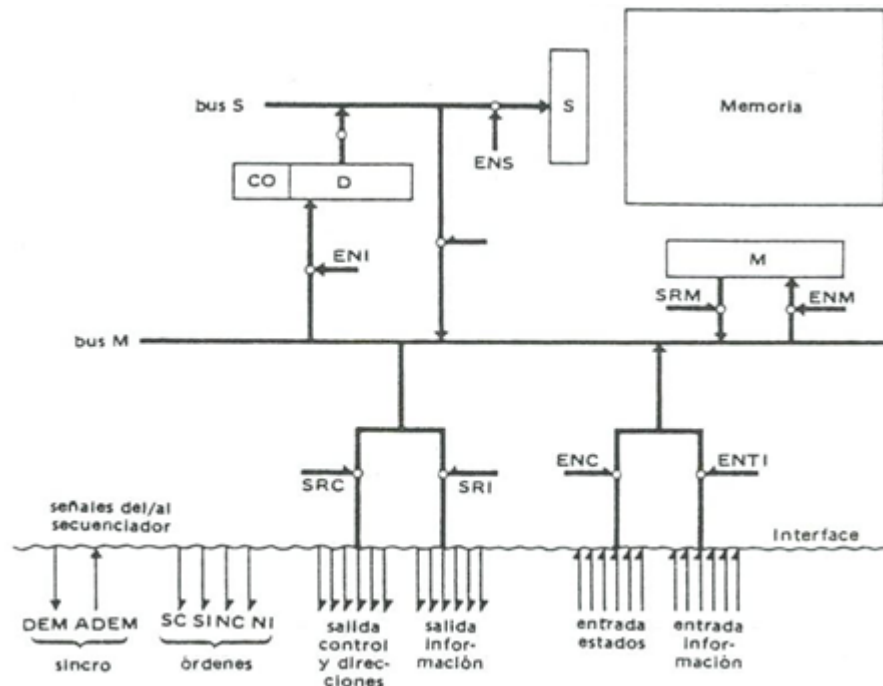


Fig. 12. Enlace programado de Abacus

Canal automático ABACUS

Acá ya se puede hacer la transferencia de un bloque de info desde/hacia un periférico. El canal incluye un conjunto de registros/señales de Gob para manejar las transferencias a través de instrucciones de transferencia.

- El registro T se encarga de aplacar las diferencias de tiempos de acceso entre la memoria central y los periféricos. Se libera al computador de tener que bloquear procesos para esperar que se procese la info que está por entregar/recibir

- El registro DEC (Direccion en Curso), es la dirección de cada palabra que son transferidas
- CDP (Palabra en curso) lleva la cuenta de cuántas palabras se transfirieron (descontador)

En este modelo, se requiere que la información esté ordenada y contigua en memoria.

Si DEC hace la petición de transferencia y trabaja directamente con el bus S ¿Que tipo de transferencia es? → Robo de ciclo ¿Esto afecta al programa en curso al dejarlo en una espera del ciclo? → Sí pero no. Está el registro T que minimiza el efecto de robo de ciclo. Apenas ocurre, se encarga de retener la información y liberar a la memoria para la próxima instrucción. La transferencia queda a cargo del canal, quien trabaja al ritmo del periférico.

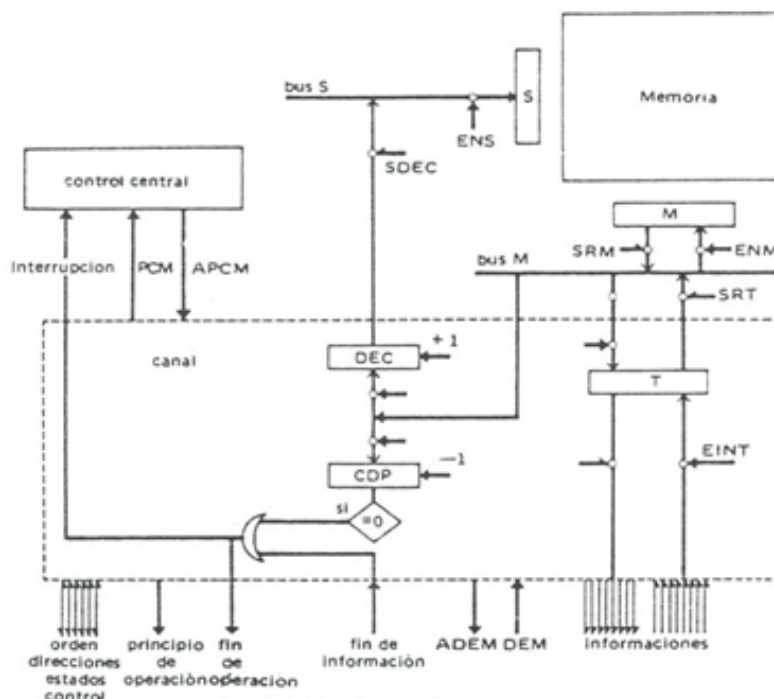


Fig. 17. Canal automático para Abacus

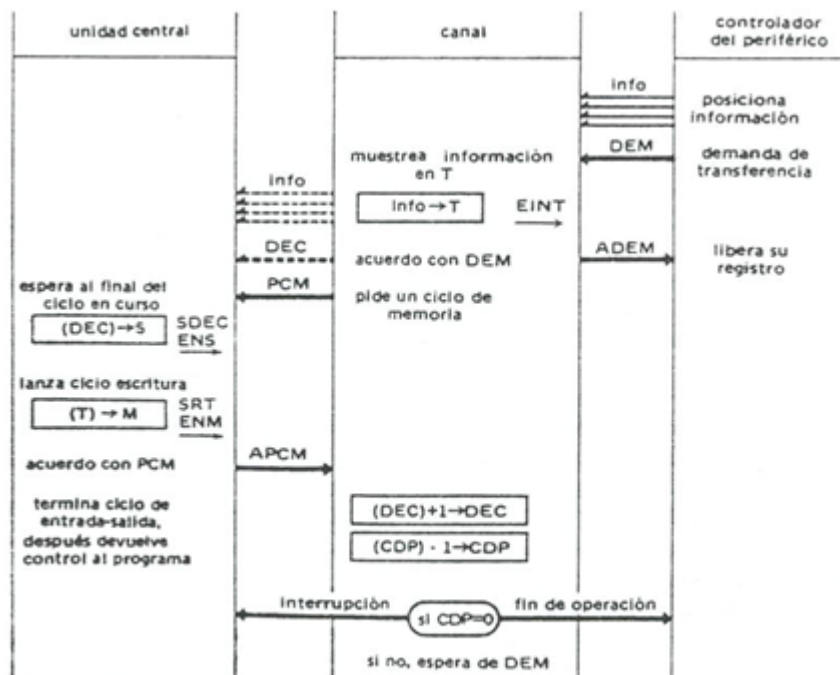


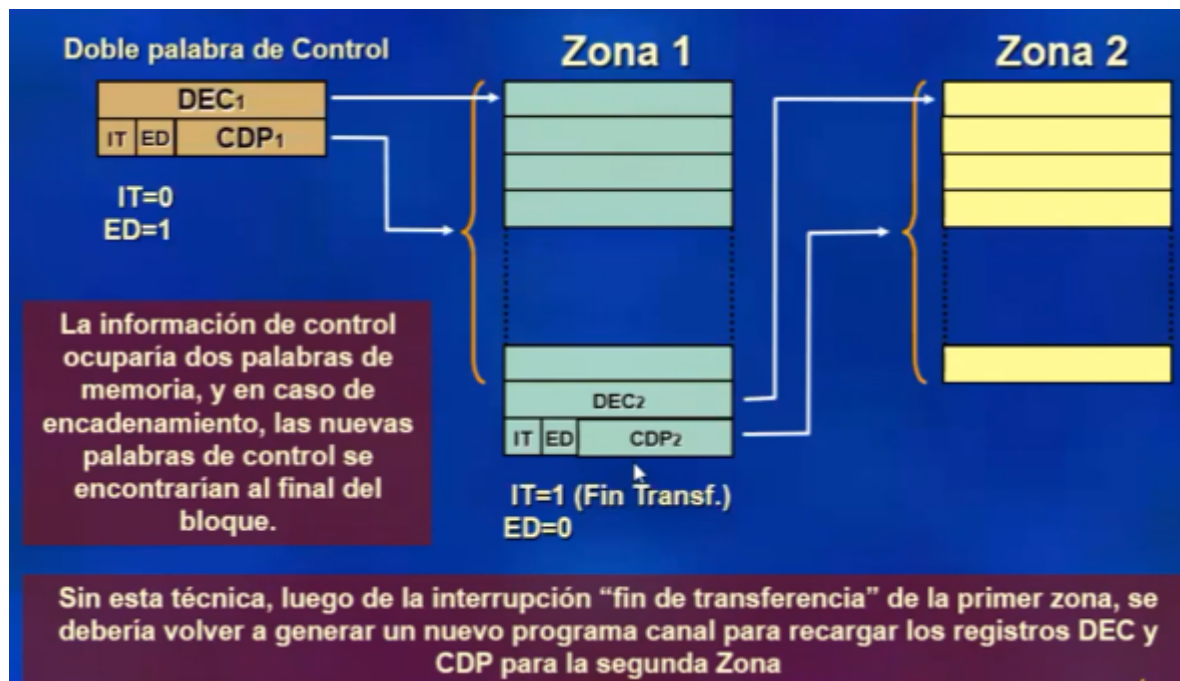
Fig. 18. Esquema de una entrada de información a través del canal automático de Abacus

¿Cuál es la diferencia entre el enlace programado y el canal automático? → El primero no es ni siquiera un canal. No tiene el gobierno de la transferencia y solo hace una transferencia elemental. En el segundo es el canal el que toma el control y con robos de ciclo genera las transferencias.

Encadenamiento de datos

Con esto la idea es poder cambiar de zona/bloque de memoria sin necesidad de reiniciar el canal y perder un ciclo de memoria haciéndolo ⇒ Agarramos la última palabra de cada bloque para actualizar la información y que indique la dirección del siguiente.

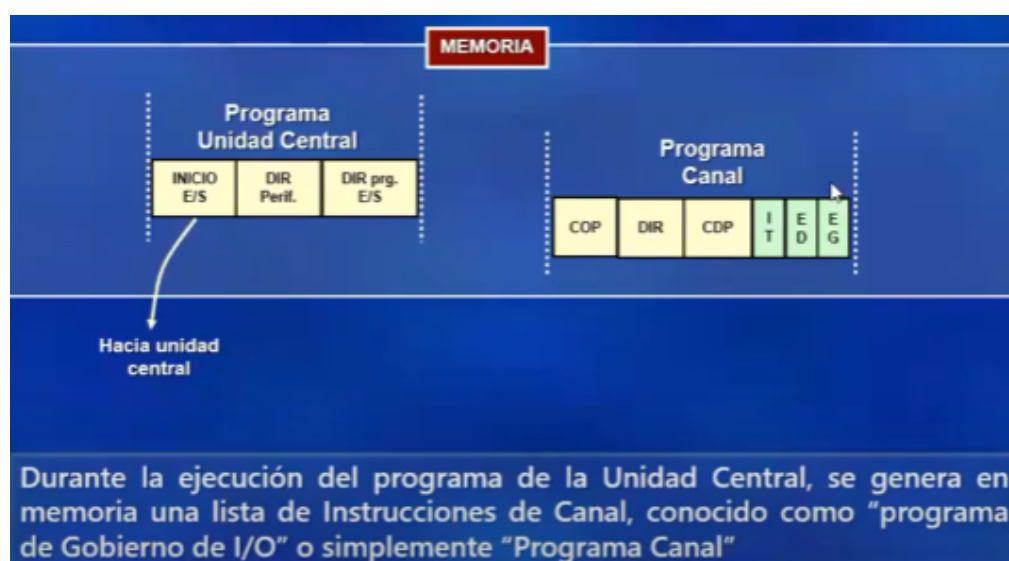
IT:	= 0	No interrupción	<table><tr><td colspan="3">DEC</td></tr><tr><td>IT</td><td>ED</td><td>CDP</td></tr></table>	DEC			IT	ED	CDP
DEC									
IT	ED	CDP							
	= 1	Interrupción							
ED:	= 0	No encadenamiento							
	= 1	Encadenamiento							



Noción del programa canal

Se tiene un programa llamado programa canal en memoria central, al cual se le va a proporcionar los mismos derechos que a otros programas y el canal información necesario para hacer encadenamiento de datos y tareas complejas.

Durante la ejecución del programa de la unidad central se genera en memoria una lista de instrucciones de canal (programa de gobierno de E/S // programa canal)



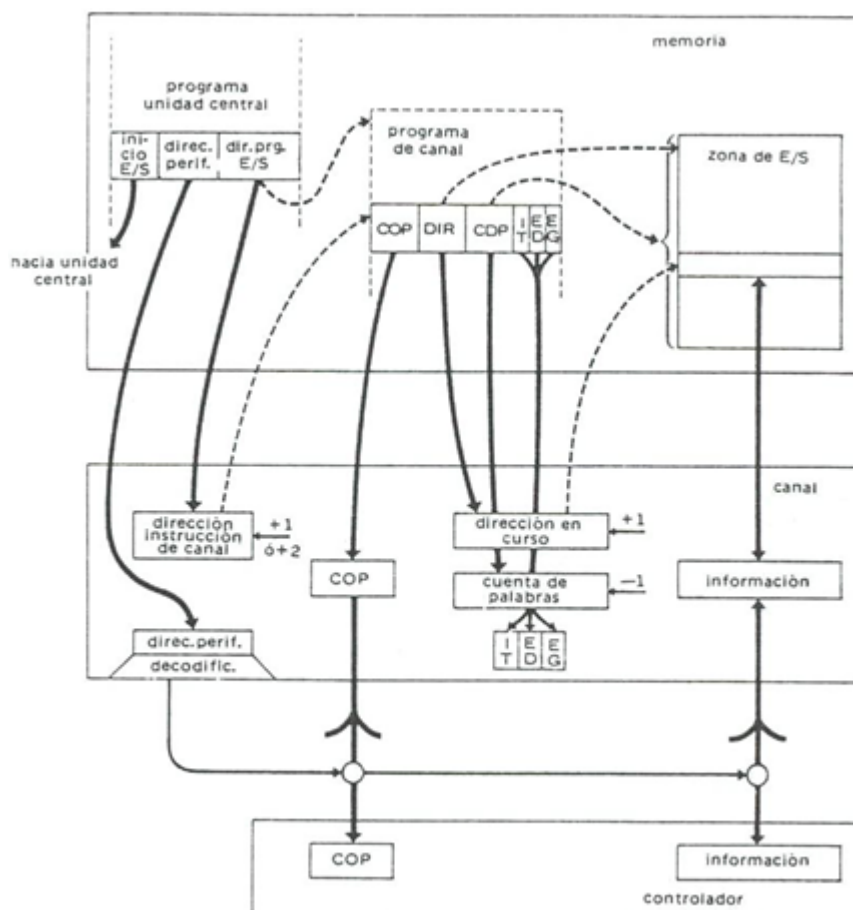
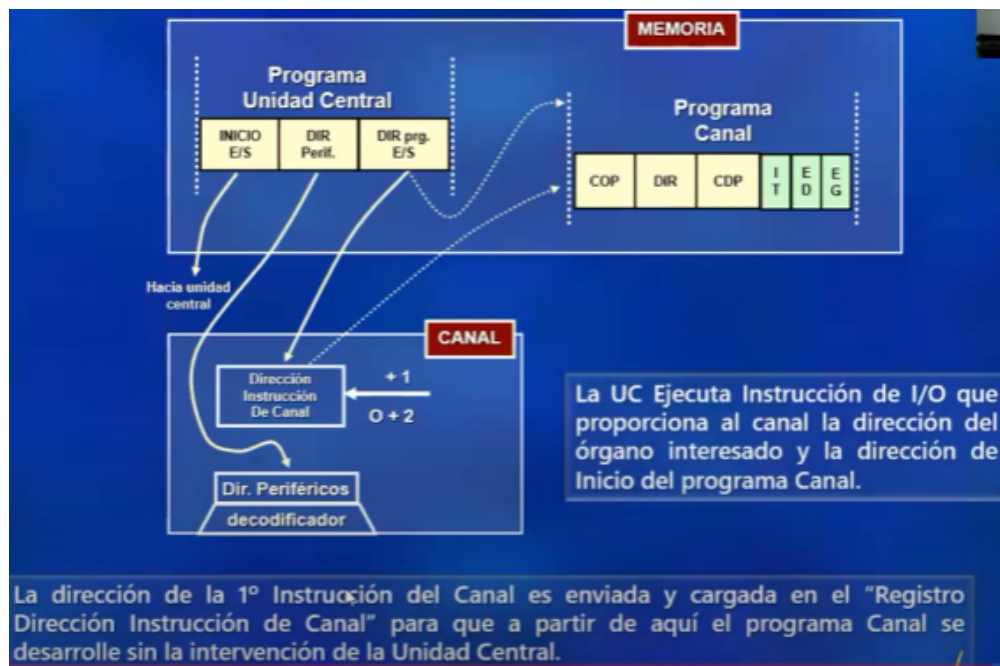


Fig. 21. El programa de canal

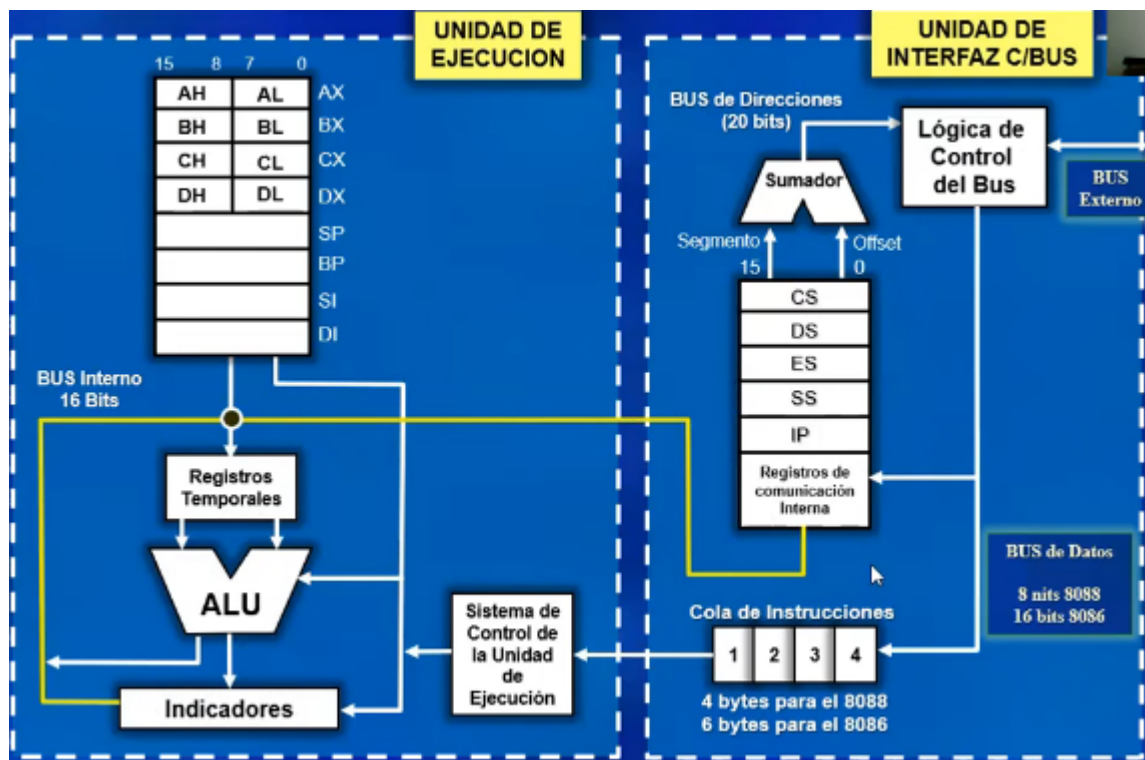
Lenguaje de Máquina

Lenguaje definido por la lista de las instrucciones de un ordenador y su representación interna bajo forma binaria. Es el lenguaje directamente ejecutable por el ordenador.

El LM es específico de cada arquitectura, aunque su conjunto de instrucciones pueden ser similares ⇒ Puedo tener dos arquitecturas diferentes, donde sus LM son diferentes, **donde el lenguaje assembler es el mismo en ambas. La ventaja que tiene assembler es que se puede programar para dos arquitecturas diferentes.** No significa que el assembler sea un lenguaje genérico, pero intenta ser el primer paso para un lenguaje de alto nivel.

El ensamblador es un programa encargado de traducir al lenguaje de máquina las instrucciones escritas en lenguaje simbólico.

8086



Ciclo FETCH (búsqueda) de la arquitectura 8086

- 1) La instrucción ingresa a la Lógica de Control del Bus o Controladora del Bus, encargado de comunicar al procesador con el exterior., en este caso con la Memoria RAM. Recibe todas las instrucciones del Bus externo y las envía a la Cola de Instrucciones
- 2) La cola de Instrucciones es una estructura FIFO de 6 palabras encargada de ir almacenando en orden de llegada cada instrucción de la memoria central (conocido como ciclo prefetch).
- 3) Sistema de control de la unidad de la UE se encarga de ir sacando las instrucciones de la cola, decodificarlas y ejecutarlas.
- 4) Las operaciones de la ALU generan una serie de eventos, registrados en el Registro Indicadores de eventos o bits bandera (resultado cero, negativo, acarreo, etc), que almacenan 1 si se produce el evento o cero si no se produce.
- 5) Para realizar las Operaciones Aritméticas, la ALU utiliza a los Registros de Propósito General, lo que almacenan los valores temporales que la instrucción necesita.
- 6) Si la instrucción requiere acceso a operando en memoria, pasan al entorno del Sumador, el que obtiene la dirección efectiva del operando sumando el valor del registro DS (Segmento de Datos) mas el valor de desplazamiento

Registros de Direcciones de Segmentos

Es el proceso en el que la memoria principal de la computadora se divide lógicamente en diferentes segmentos y cada segmento tiene su propia dirección base.

Básicamente se utiliza para mejorar la velocidad de ejecución del sistema informático, de modo que el procesador pueda recuperar y ejecutar los datos de la memoria de forma fácil y rápida.

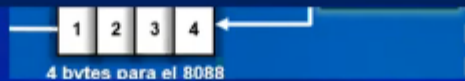
La unidad de interfaz de bus (BIU) contiene cuatro registros de propósito especial de 16 bits denominados registros de segmento:

Registro de segmento de código (CS): se utiliza para direccionar la ubicación de la memoria en el segmento de código de la memoria, donde se almacena el programa ejecutable.

Registro de segmento de datos (DS): apunta al segmento de datos de la memoria donde se almacenan los datos.

Registro de segmento Extra (ES): también se refiere a un segmento en la memoria de la zona alta disponible para almacenar información.

Registro de segmento de pila (SS): se utiliza para direccionar el segmento de pila de la memoria. El segmento de pila es el segmento de memoria que se utiliza para almacenar datos de pila.



Los registros de Propósito General

Registro AX es el acumulador principal, utilizado para operaciones que implican entrada/salida y la mayor parte de la aritmética. Por ejemplo, las instrucciones para multiplicar, dividir y traducir suponen el uso del AX. Algunas operaciones generan código más eficiente si se refieren al AX en lugar de a los otros registros.



Registro BX conocido como el registro base ya que es el único de propósito general que puede ser índice para direccionamiento indexado y emplearse para cálculos. Se utiliza para almacenar el offset de una estructura de datos (normalmente indica el número de posiciones de memoria sumadas a una dirección base para conseguir una dirección absoluta específica)



Registro CX conocido como el registro contador. Usado para cálculos, pero su función principal es la de controlar la cantidad de ciclos de una estructura loop.



Registro DX conocido como el registro de datos. Algunas operaciones de entrada/salida requieren su uso, y las operaciones de multiplicación y división con cifras grandes suponen al DX y al AX trabajando juntos.



Los registros de Direcciones de Datos (Apuntadores)

Registro SP (Puntero de Pila): Aunque es un registro de uso general, debe utilizarse sólo como puntero de pila, la cual sirve para almacenar las direcciones de retorno de subrutinas y los datos temporarios (mediante las instrucciones PUSH y POP). Al introducir (push) un valor en la pila a este registro se le resta dos, mientras que al extraer (pop) un valor de la pila este a registro se le suma dos.

Registro BP (Puntero de Base): Generalmente se utiliza para realizar direccionamiento indirecto dentro de la pila

Registro SI o Índice Fuente: Sirve como puntero fuente para las operaciones con cadenas. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde comienza los datos que se quieren leer dentro de un segmento de datos.

Registro DI o Índice Destino: Sirve como puntero destino para las operaciones con cadenas. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde terminan los datos que se quieren leer dentro de un segmento de datos.

Registros de Direcciones de Segmentos

Es el proceso en el que la memoria principal de la computadora se divide lógicamente en diferentes segmentos y cada segmento tiene su propia dirección base.

Básicamente se utiliza para mejorar la velocidad de ejecución del sistema informático, de modo que el procesador pueda recuperar y ejecutar los datos de la memoria de forma fácil y rápida.

La unidad de interfaz de bus (BIU) contiene cuatro registros de propósito especial de 16 bits denominados registros de segmento:

Registro de segmento de código (CS): se utiliza para direccionar la ubicación de la memoria en el segmento de código de la memoria, donde se almacena el programa ejecutable.

Registro de segmento de datos (DS): apunta al segmento de datos de la memoria donde se almacenan los datos.

Registro de segmento Extra (ES): también se refiere a un segmento en la memoria de la zona alta disponible para almacenar información.

Registro de segmento de pila (SS): se utiliza para direccionar el segmento de pila de la memoria. El segmento de pila es el segmento de memoria que se utiliza para almacenar datos de pila.

Registro de Banderas (FLAG)

Contiene información de estado y de control de las operaciones del microprocesador. De los 16 bits del registro se utilizan solo 9 y cada uno de estos representa a una bandera. Permiten que instrucciones de bifurcación puedan respaldarse en resultados temporales de estos indicadores

Seis banderas de estado que registran el estado del procesador, normalmente asociado a una comparación o a una instrucción aritmética:

CF - (Carry Flag) Bandera de acarreo. Indica acarreo en las instrucciones aritméticas.

OF - (Overflow Flag) Bandera de desbordamiento aritmético.

ZF - (Zero Flag) Bandera de resultado cero o comparación igual.

SF - (Sign Flag) Bandera de resultado o comparación negativa.

PF - (Parity Flag) Bandera de paridad, utilizada en la verificación de la transferencia de bytes entre dos componentes de la computadora.

AF - (Auxiliar Flag) Bandera auxiliar. Indica si hay necesidad de ajuste en las operaciones aritméticas con números BCD.

Modos de Direccionamiento de Memoria

Los modos de direccionamiento indican la manera de obtener los operandos de memoria. El tipo de direccionamiento se determina en función de los operandos de la instrucción, siendo el mas usado MOV destino, origen:

La instrucción MOV realiza transferencia de datos desde un operando origen a un operando destino. Su formato es el siguiente:

MOV destino, origen

- 1-Direccionamiento de registro
- 2-Direccionamiento inmediato
- 3-Direccionamiento directo
- 4-Direccionamiento indirecto mediante registro
- 5-Direccionamiento indirecto por registro base
- 6-Direccionamiento indexado
- 7-Direccionamiento indexado respecto a una base

Periféricos

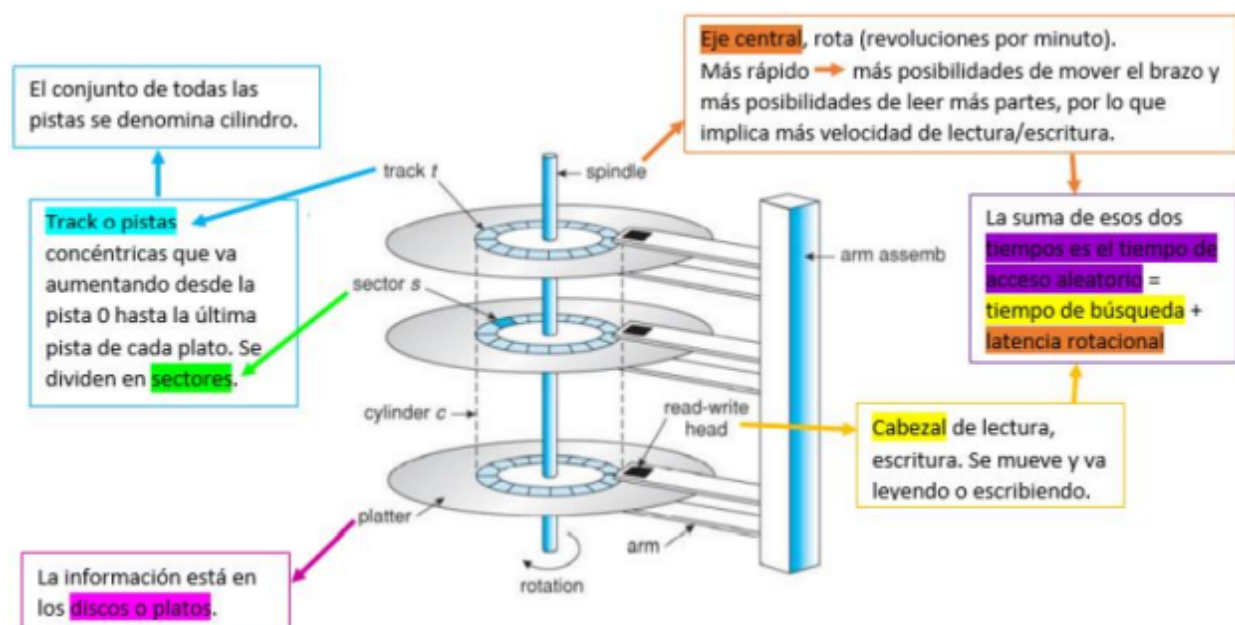
Disco Rígido

Los discos rígidos constituyen la unidad de almacenamiento principal de la computadora. En ellos se guardan una gran cantidad de datos y programas. Constituyen la memoria de almacenamiento masivo. Estos datos y programas no los puede procesar directamente el microprocesador, sino que en un paso previo deben ser transferidos a la memoria principal donde sí se los puede manejar.

Las unidades de disco rígido contienen uno o más platos discos (platos) apilados sobre un eje central y aislados completamente del exterior.

Los elementos móviles de los discos rígidos están mucho mejor contruidos que los de los disquetes, el sistema gira a más velocidad y los discos son de mayor densidad. Cuando el usuario o el software pide al sistema operativo que lea o escriba un determinado sector del disco, el sistema operativo ordena a la controladora del disco rígido que mueva la cabeza de lectura/escritura sobre la pista que contiene ese sector. Los cabezales de lectura/escritura no tienen más que esperar que ese sector pase exactamente por debajo de ellos, para leer o escribir sobre él.

Para grabar la información en el disco hay que codificarla para escribirla y decodificarla para leerla. Lo que persigue una buena técnica de codificación es manejar una misma información con un mínimo número de pulsos, y una serie de no pulsos no muy grande.



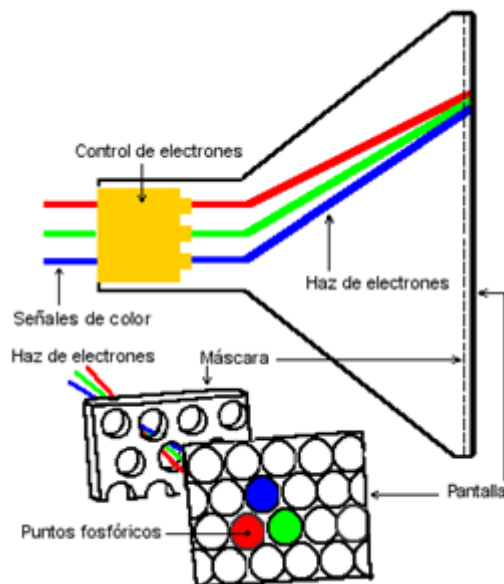
Monitor

El monitor es una de las partes más importantes de la computadora, ya que es el primer medio por el que el usuario conoce los resultados de su trabajo. El monitor se conecta a la tarjeta gráfica, estando dicha tarjeta integrada en la placa base, o conectada a un bus de expansión.

En los monitores, la imagen en la pantalla está formada por un grupo de líneas horizontales que se van explorando secuencialmente. Un haz de electrones barre cada línea de la pantalla sucesivamente de izquierda a derecha desde la parte superior izquierda. A medida que el haz barre cada línea, el color y brillo de cada uno de los cientos de puntos que componen la pantalla (píxeles) se modifican por la información suministrada por la tarjeta gráfica hasta formar una imagen coherente procedente de dicha

tarjeta. Esta imagen de vídeo se refresca de una forma cíclica, a una frecuencia que envía la tarjeta gráfica.

Una vez que el haz de electrones ha barrido todas las líneas horizontales, la señal de activación de pantalla se sitúa en "off" (no aparece nada en pantalla), y el controlador del tubo de rayos catódicos genera una señal de sincronismo vertical, que le dice al monitor que desvíe el haz de electrones desde el final de la pantalla hacia el borde superior izquierdo.



Teclado

Gracias al cual es posible comunicarse con la computadora, permitiéndote enviar datos o instrucciones en forma de texto, cifras, etc. El teclado de contacto puede ser de dos tipos:

- **Mecánico:** usa pequeños interruptores individuales para cada tecla. Cuando se presiona una tecla se cierra el interruptor y permite el paso de corriente. Tiene el inconveniente de ser bastante vulnerable a la suciedad, de forma que después de un tiempo de uso es posible que alguna de sus piezas empiece a fallar.

- **De membrana:** las teclas llevan una membrana de goma que al pulsar hace que se unan dos pistas conductoras en la parte inferior y permite el paso de corriente: son similares a las que llevan las calculadoras. Estas membranas pueden ser individuales para cada tecla o bien una lámina completa con membranas que se mueven con cada tecla.

El teclado contiene un pequeño procesador que se encarga de comprobar si se ha pulsado alguna tecla, se sigue un proceso hasta que su código queda almacenado en la memoria, y es el siguiente:

- Se pulsa una tecla [Si se pulsan varias teclas simultáneamente, el controlador de teclado las almacena en un buffer interno (de unos 10 caracteres). Este buffer no se llena nunca ya que la transmisión es demasiado rápida para que el usuario pueda mantener el ritmo.

- El controlador de teclado detecta la pulsación.
- Genera el código de muestreo (code) de tecla pulsada y lo envía al puerto del teclado.
- Activa una solicitud de interrupción y genera un vector de interrupción.
- El microprocesador accede a la tabla de vectores de interrupción a la dirección correspondiente al vector solicitud.
- Se ejecuta la rutina del controlador de teclado.

- Cuando se suelta la tecla, el controlador de teclado genera un código de tecla solicitada (break code).

El carácter que aparezca, se corresponde con la tecla pulsada y depende de la página de códigos cargada en el sistema operativo.

Mouse

- **Mecánicos:** tienen una gran bola de plástico, de varias capas, en su parte inferior para mover dos ruedas que generan pulsos en respuesta al movimiento de éste sobre la superficie.

- **Ópticos:** la bola y las ruedas de codificación son sustituidas por unos componentes ópticos: diodos emisores de luz y fototransistores. Su funcionamiento se basa en el reflejo de un haz de luz es una superficie reflectante que utiliza como soporte, que incorpora unas líneas formando cuadros que sirven para medir el desplazamiento del ratón y con ello la posición del cursor en la pantalla. Los diodos producen dos rayos dirigidos hacia abajo. Al estar situado el ratón sobre una superficie reflectante que contiene finas rayas horizontales y verticales, cuando se desplaza el ratón se reenvían hacia arriba y los fototransistores detectan las variaciones de luz que resultan de ello. Los componentes electrónicos del ratón transforman estos impulsos luminosos en señales eléctricas.

- **De láser:** detecta el movimiento deslizándose sobre una superficie horizontal, pero el haz de luz de tecnología óptica se sustituye por un láser con resoluciones a partir de 2000 ppp, lo que se traduce en un aumento significativo de la precisión y sensibilidad.

- **Trackball:** se debe mover el puntero, de tal forma que cuando se coloque la mano encima se pueda mover mediante el dedo pulgar.

Impresora

Es un dispositivo electromecánico capaz de plasmar sobre un soporte físico (papel, transparencia, etc.) la información (textos, gráficos, etc.) creados en una computadora. A través de la conexión (ya sea utilizando un puerto paralelo o un puerto USB) la computadora envía una serie de códigos hexadecimales (o códigos ASCII) que representan los caracteres, signos de puntuación y movimiento de la impresora.

Cuando el buffer de la impresora se llena, el controlador de la impresora envía un código de control para indicar a la computadora que interrumpa la transmisión de datos. En el momento en que se vacía en buffer, la impresora envía un nuevo código que reanuda el envío de datos.

El procesador de la impresora tomará toda la información, calculará el posicionamiento más eficaz de las distintas partes encargadas de la impresión, y enviará las señales de control a los distintos elementos de movimiento, para que todo salga a la perfección. Al mismo tiempo, se dispone de sensores que detectan los movimientos y posición de cada elemento, con el fin de que el procesador contraste los datos enviados con los reales y corrija las deficiencias detectadas.

- **La impresora de agujas**

Las impresoras de agujas, eran ruidosas y ofrecían un resultado de calidad mediocre, pero eran mucho más rápidas y baratas. Producen una imagen en una hoja de papel con ayuda de una serie de agujas que golpean dicha hoja. Entre las agujas y la hoja se pone una cinta entintada, de forma similar a como se hace en una máquina de escribir.

Las **ventajas** más importantes de este tipo de impresoras son su fiabilidad y bajo coste de mantenimiento (el coste de la cinta es insignificante, aunque también su duración es bastante menor). Los principales **inconvenientes** son su baja velocidad y, sobre todo, el enorme ruido que producen las agujas al golpear sobre el papel.

• La impresora de inyección de tinta

Es una impresora silenciosa, de alta o muy alta calidad y de bajo precio, más rápida que la de agujas aunque más lenta que la láser, y que por sus características se ha impuesto en el sector doméstico de forma generalizada. Imprime línea a línea con ayuda de un cabezal de impresión móvil, de forma similar a como lo hace una impresora de agujas. Para crear la imagen impresa, se utiliza un sistema de no impacto, por el cual el cabezal no llega a tocar la hoja en ningún momento, sino que pulveriza pequeños chorros de tinta sobre el papel.

El cabezal de impresión de una impresora de chorro de tinta está montado en un carro móvil, como en el caso de las impresoras de agujas. Contiene un cierto número de orificios, a través de los cuales se expulsa la tinta líquida formando un minúsculo punto negro en el papel. Según el dato a imprimir, se expulsará la tinta por unos orificios u otros, formando la imagen en el papel.

• La impresora láser

La impresora láser es una impresora silenciosa, de muy alta calidad, precio elevado (aunque hay modelos a buen precio), más rápida que las de inyección a tinta y que debido a estas características se ha impuesto en el sector profesional.

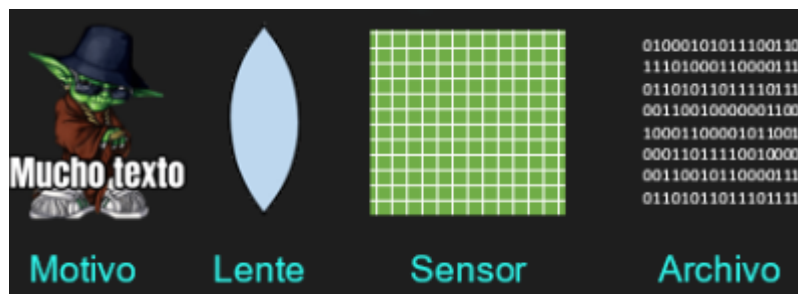
En una impresora láser, la imagen se obtiene por un procedimiento electrostático, similar al de las fotocopadoras. La impresión no imprime el papel línea por línea, sino por páginas completas. Todos estos datos se almacenan en una memoria RAM propia de la impresora, y cuando los datos de la página están completos tomará una hoja de la bandeja e imprimirá la hoja completa, pasándola entre dos rodillos.

El rodillo de la impresora va girando y entra en contacto con un polvo muy fino llamado tóner que se adhiere a las zonas cargadas eléctricamente, las zonas donde ha incidido el rayo láser son descargadas y corresponden a las zonas que van en blanco.

Escáner

Un escáner de ordenador es un periférico que se utiliza para "copiar", mediante el uso de la luz, imágenes impresas o documentos a formato digital

Percibe el color y la intensidad y los convierte en señales electrónicas conforme recorre una imagen. Permite convertir los caracteres escritos o gráficos impresos en un formato digital, compuesto por un lenguaje binario de 0 y 1 comprensible por la computadora. A partir de esta transformación, la imagen que queda registrada en un archivo en la computadora puede ser manipulada por el usuario.



Para el caso de texto digitalizado la imagen se puede convertir en forma simbólica a través de software de reconocimiento óptico de caracteres (**OCR**, por sus siglas en inglés) y se almacena como archivo de texto.

Básicamente, su funcionamiento se basa en la captación mediante espejos, donde la luz es reflejada hacia un dispositivo llamado CCD y transforma esa luz en señales eléctricas. El proceso

continúa transformando estas señales eléctricas a un formato digital en un DAC (Conversor Analógico-Digital) que transmite el caudal de bits resultante al ordenador.

Hace rebotar luz blanca de la imagen, y convierte los rayos reflejados en electricidad, esto permite que el escáner detecte la diferencia entre los millones de sombras de color. El cinturón motorizado mueve el cabezal del escáner plano por toda la superficie de la imagen, la idea principal acá es que el ccd pueda convertir la luz en una carga eléctrica luego la envía a la computadora como datos digitales. Las áreas más oscuras las páginas reflejan menos luz que las áreas en blanco.

El primer rayo pasa por un filtro rojo, el segundo rayo pasa por un filtro verde y el tercero con un filtro azul. Todos filtran todos los colores menos el suyo. El resultado es la cantidad exacta de cada uno de esos tres colores presente en todos y cada uno de los puntos de la imagen

